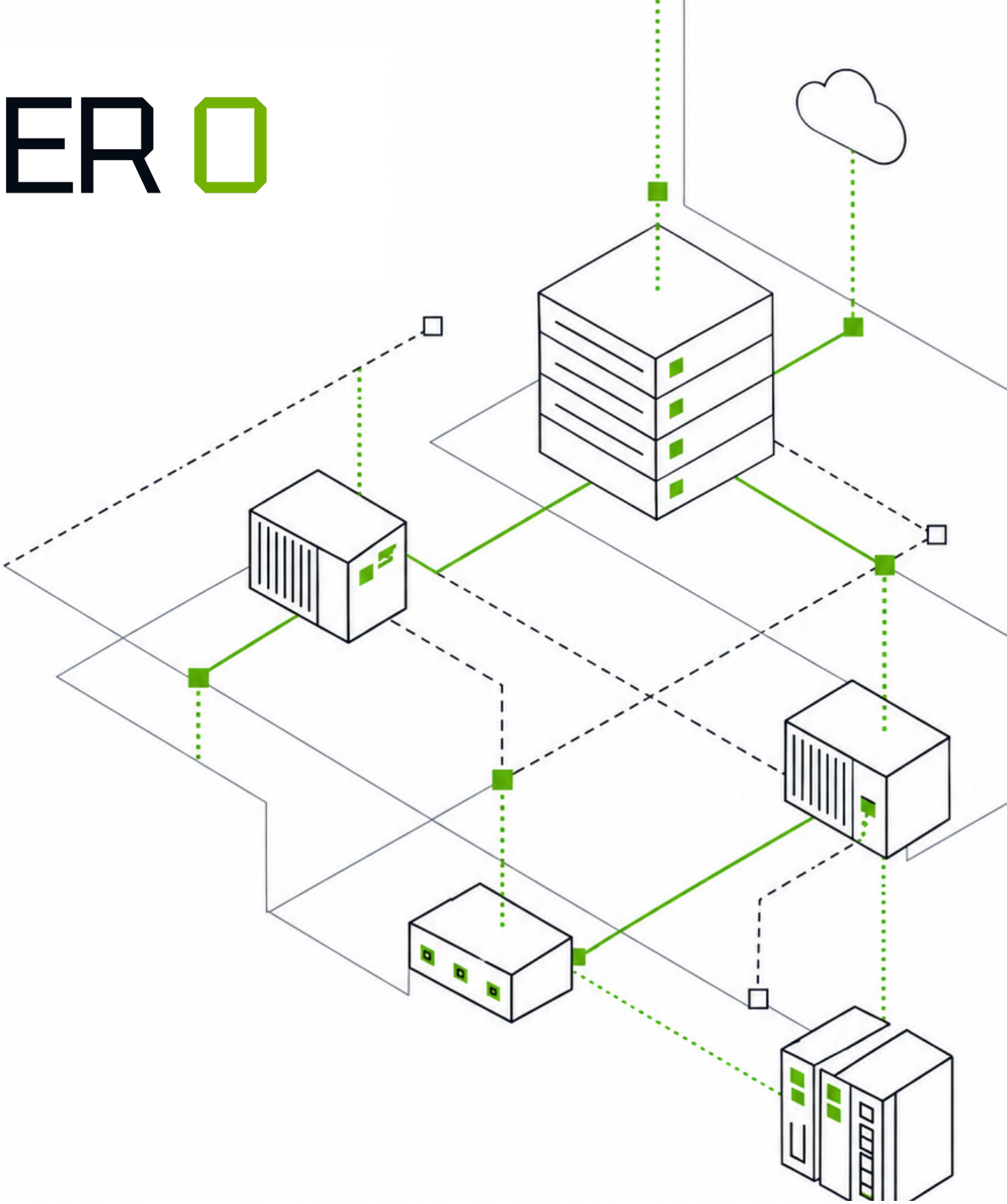
TIER 0



Tier0 技術ガイド

目次

はじめに

1. 最適なバージョンを選ぶ
2. インストール
3. Tier0 で Demo Factory を試す

TIER0 の使い方

1. 工場データ基盤
2. データ基盤を準備する
3. センターからエッジのデータを管理する
4. 工場データを扱う
5. アプリを構築する
6. Digital Twin を構築する
7. データを分析する
8. Agents で Tier0 を操作する

ベストプラクティス

1. 工場データをモデリングする
2. 産業プロトコルの接続
3. セキュリティとレジリエンス
4. 分析アプリの構築
5. Agent でデータワークフローを構築する
6. Digital Twin でデータを表示する

リファレンス

1. API リファレンス
2. CLI コマンドリファレンス
3. 組み込み Builder Skills
4. ユーザー管理
5. Tier0 Cloud のネットワークとデータのセキュリティに関する説明
6. ISA-95 設備階層
7. 標準デプロイ計画
8. 高可用性デプロイ計画
9. SLA と高可用性の境界
10. 標準ポート一覧
11. 運用 Runbook

付録

1. OpenAPI リファレンス完全版

はじめに

最適なバージョンを選ぶ

Edge、Cloud、Enterprise：位置づけ、プラン、機能マトリクスから Tier0 エディションを選びます。

Tier0 には 3 つのエディションがあります。各エディションには、それぞれ異なる機能、プラン、対応範囲があります。

■ オープンソース

Edge

1 台のマシン上で動く UNS 基盤。すべて自分で管理できます。

> 技術評価、PoC、自社でオープンソースソフトウェアを運用できるチーム向け。

Apache-2.0 無料

UNS データ統合、履歴保存、単一マシン Docker デプロイ。

GitHub でクローン ->

■ マネージド SAAS

Cloud 多くのチームはここから始めます

フルプラットフォームを運用込みで利用できます。

> 初日から apps、notebooks、launchpad を利用でき、インフラ運用は不要です。

Builder $199/SEAT/MO

アプリ生成のみ。

Growth ★ $20,000/YR

最大 5 edges。少数のアプリで素早く始めたい単一工場向け。

Scale $38,000/YR

最大 10 edges。複数工場と多数のアプリを持つユーザー向け。

7 日間トライアルを開始 ->

■ プライベートデプロイ

Enterprise

フルプラットフォームを自社要件に合わせて利用できます。

> データ主権、スケール、ガバナンス、エンタープライズ監督が必要な環境向け。

Base $10,000/YR

統合データ基盤。少数の単目的アプリ。

Plus $20,000/YR

シングルインスタンス。単一工場のデータ統合と複数ユースケースのアプリ。

Cluster ★ $39,900+/YR

マルチインスタンス。複数工場、多数のアプリ、集中型プライベートクラウド管理。

チームに相談 ->

Add-ons：要件に応じて、既存プランに edge nodes と applications を追加できます。詳細は tier0.app/pricing を参照してください。

ⓘ 用語説明

Cloud プランでは、edge はクラウド Tier0 と通信する接続ノードです。Edge Tier0、ゲートウェイ、または産業用 PC にできます。

機能マトリクス

機能	Edge	Cloud	Enterprise
UNS / データモデリング	✓ 単一マシン UNS	✓ Growth / Scale	✓ Base / Plus / Cluster
産業プロトコル	✓ MQTT	✓ Growth / Scale：MQTT、REST、i3X、OPC UA	✓ Base：MQTT；Plus / Cluster：MQTT、REST、i3X、OPC UA
UNS Agent	×	✓ Growth / Scale	×
Notebook（高度分析）	×	✓ Growth / Scale	✓ Plus / Cluster
Vision	×	✓ Scale	✓ Plus / Cluster
Anchor	×	✓ Scale	✓ Cluster
App Builder + Template Library	×	✓ Builder / Growth / Scale	×
LaunchPad / My Apps	×	✓ Builder / Growth / Scale	✓ Base / Plus / Cluster
監査 / アプリとシステムログ	×	✓ Growth / Scale	✓ Plus / Cluster；Cluster では SIEM
HA / マルチインスタンス / ガバナンス	×	×	✓ Cluster
運用	自分で運用	FREEZONEX	自分で運用、サポート付き

Edge ハードウェア要件

> **Edge を使いたい場合**
> Edge は技術評価向けで、運用経験が必要です。 使用前に、環境が以下のハードウェア要件を満たしていることを確認してください。

	最小	推奨
CPU	4 cores	8 cores
Memory	8 GB	16 GB
Disk	100 GB (1000 IOPS)	1 TB
OS	Ubuntu 24.04、Windows 10/11 (Docker)	-

判断ツリー

まだどれを選ぶべきか迷う場合は、パスをたどって判断してください。



次へ

- **UNS 上にアプリを構築** - UNS データで産業アプリケーションを構築します。
- インストール - 7 日間 Cloud トライアルはフルプラットフォームです。

はじめに

インストール

エディション別に Tier0 をインストールします。

Tier0 には 3 つのエディションがあります。

- **Edge** - オープンソースのデプロイ版で、Docker を使ってローカルで実行します。
- **Cloud** - マネージド SaaS で、インフラの準備は不要です。
- **Enterprise** - エンタープライズ環境向けのプライベートデプロイ版です。

要件に最も合うエディションを選ぶには、最適なバージョンを選ぶ を参照してください。

Tier0 のインストール

タブ: CLOUD

Tier0 Cloud を試す前に、トライアルアカウントを申請してください。

1. Tier0 の登録 でアカウントを登録します。
2. ポップアップ画面で、メールに送信された確認コードを入力します。
3. **Login** ページにリダイレクトされます。登録したアカウントでログインします。
4. 関連情報を入力し、**Continue** をクリックします。
5. **Create a New Workspace** を選択します。
6. workspace を作成してトライアルを開始します。

タブ: ENTERPRISE

要件

- 単一ノードデプロイ
  - 最低：4C / 8G / 100G
  - 推奨：8C / 16G / 500G
- 標準デプロイ
  - 最小：4 台の VM

    - VM タイプ別の必要ベースライン：
        - App：8C / 32G / 100G
        - Database：4C / 8G / 500G
        - Witness：2C / 4G / 40G
- 高可用性デプロイ
    - 最小：6 台の VM
    - VM タイプ別の必要ベースライン：
        - App (Swarm)：8C / 32G / 100G x 3
        - Database：4C / 8G / 500G x 2
        - Witness：2C / 4G / 40G

> ハードウェア要件は変わる場合があります
> Enterprise のハードウェア要件はデプロイシナリオによって変わります。

> ⓘ ノート
> 詳細なデプロイ情報は 標準デプロイ計画 と 高可用性デプロイ計画 を参照してください。

操作

1. ベンダーからパッケージを入手し、検証します。

```
sha256sum -c tier0-enterprise-offline-standard-
YYYYmmddTHHMMSS.tar.gz.sha256
```

2. パッケージを展開します。

```
tar -xzf tier0-enterprise-offline-standard-YYYYmmddTHHMMSS.tar.gz
cd tier0-enterprise-offline-standard-YYYYmmddTHHMMSS
```

3. デプロイ内容に合わせて `install.env` を編集し、デプロイを開始します。

```
vi install.env
```

```
bash install.sh
```

よく使うメンテナンス操作

- Enterprise をインストールせずにデプロイ環境を確認します。

```
./install.sh --check-only
```

- デプロイ失敗後にすべてのノードをクリアします。

```
./full-cleanup.sh --env install.env --yes
```

> このコマンドは、VM、OS、インストール済み Docker ソフトウェアを残したまま、Tier0 のランタイムデータとデータベースバックアップを削除します。

- インストール後にデプロイレポートが生成されます。

```
runtime/install-reports/<timestamp>/deployment-report.md
```

> レポートに **FAIL** 項目がある場合は、プロジェクトを引き渡さず、先にサポートに連絡してください。

メンテナンス参照

- 標準ポート一覧
- メンテナンスとトラブルシューティングガイド
- HA 境界の説明

タブ: EDGE

要件

- 最低 4 コア CPU、8 GB RAM、100 GB ディスク
- Ubuntu 24.04、または Docker と Git をインストールした Windows 10/11

操作

1 GitHub から Tier0-Edge リポジトリをサーバーにクローンし、プロジェクトに移動します。

```
git clone https://github.com/FREEZONEX/Tier0-Edge
cd Tier0-Edge/deploy
```

2 環境に合わせて、下表のパラメータを `.env.default` に設定します。

```
cp .env.default .env
vi .env
:wq!
```

ⓘ 設定が必要なパラメータ

フィールド	型	説明	例
VOLUMES_PATH	string	プロジェクトデータの保存パス	Linux では `/srv/tier0/data`、Windows Bash では `D:/tier0/data` または `/d/tier0/data`
ENTRANCE_DOMAIN/ENTRANCE_PORT	string	Tier0 のアクセスエンドポイント（IP/ドメイン + ポート）	`192.168.1.10` / `8088`
LANGUAGE	enum	zh-CN / en-US	`en-US`
ADMIN_INIT_PASSWORD	string	管理者ユーザーの初期パスワード	強力な非公開パスワード

フィールド	型	説明	例
COMPOSE_PROJECT_NAME/PORT_OFFSET	string	同じホスト上で実行する複数インスタンスを分離	`edge_lab` / `100`

3 インストールスクリプトを実行し、インストール状態を確認します。

```
bash bin/install.sh
bash bin/compose.sh ps # コンテナの状態を確認
bash bin/compose.sh logs -f backend # backend コンテナのログを確認
curl -fsS http://127.0.0.1:8088/readyz # backend サービスの readiness を確認
```

> ⚠ 注意
> `deploy/.env.runtime` は編集しないでください。生成済みの secrets と解決済み runtime 値が含まれます。

4 設定した endpoint とデフォルト管理者アカウントで Tier0 にアクセスします。

- Username: `tier0`
- Password: `.env` の `ADMIN_INIT_PASSWORD` の値。未設定の場合は `tier0`。

次へ

- Demo Factory を試す - 事前ロード済みの工場を探索します。
- 最適なバージョンを選ぶ

はじめに

Tier0 で Demo Factory を試す

Cloud トライアル workspace で、namespace、flows、notebooks、apps を含む事前設定済みの稼働する工場を探索します。

Tier0 を理解する最短の方法は、すでに動作している工場を探索することです。

ビジネス背景

Tier0 の工場は ERP から注文を受け取り、注文を分割し、生産をスケジュールし、その計画を生産部門に送信して、作業の進行に伴い生産進捗データを受け取ります。



プロセスを理解する

1 Tier0 で **UNS** に移動し、ERP から来た注文の詳細を確認します。

- `DemoFactory/ERP/ProductionOrders/State/UpsertProductionOrder`：注文。
- `DemoFactory/ERP/ProductionOrders/State/OrderList`：現在の注文リストのスナップショット。

> 🚀 データはどのように **UNS** に収集されますか？
> **Flows** > **Source Flow** > **DemoFactory-Flow** に移動し、データ収集の詳細を確認します。

2. （任意）**Launchpad** に移動し、**Work Order Management** アプリケーションを開いて、**Production Orders** ページで注文を作成します。
3. **Work Order Management** で **Plans** ページを使い、注文を分割して、分割された作業指示の生産計画をスケジュールします。
4. 計画を生産部門へ送信し、**UNS** の以下の topics で計画詳細を確認します。
   - `DemoFactory/ERP/WorkOrderPlan/Metric/SplitCount`：分割後の作業指示数。
   - `DemoFactory/ERP/WorkOrderPlan/State/PlanStatus`：現在の計画ステータス。
   - `DemoFactory/ERP/WorkOrderPlan/State/WorkOrderList`：生産計画のスケジュール後の作業指示リスト。

> ⓘ ノート
> アプリケーションは plans と workorders を直接 **UNS** に書き戻します。

5. アプリケーションの **Plan Details** ページで生産進捗データを確認します。

> 🚀 詳細データはどこから来ますか？
> 生産進捗データは生産部門から送られ、**Source Flow** の **DemoFactory-Flow** によって収集されて **UNS** に公開され、**Plan Details** ページに表示されます。
>
> - `DemoFactory/Site_01/Production/Line_01/WorkOrderExecution/State/CurrentWorkOrder`：処理中の注文。
> - `DemoFactory/Site_01/Production/Line_01/WorkOrderExecution/State/WorkOrderStatus`：現在の注文の実行ステータス。

- `DemoFactory/Site_01/Production/Line_01/WorkOrderExecution/Metric/Target_Qty`：現在の注文の目標生産数量。
- `DemoFactory/Site_01/Production/Line_01/WorkOrderExecution/Metric/Produced_Qty`：現在の注文で完了した製品数量。
- `DemoFactory/Site_01/Production/Line_01/WorkOrderExecution/Metric/Defect_Qty`：現在の注文の不良数量。
- `DemoFactory/Site_01/Production/Line_01/WorkOrderExecution/Metric/Completion_Rate`：現在の注文の完了率。

次へ

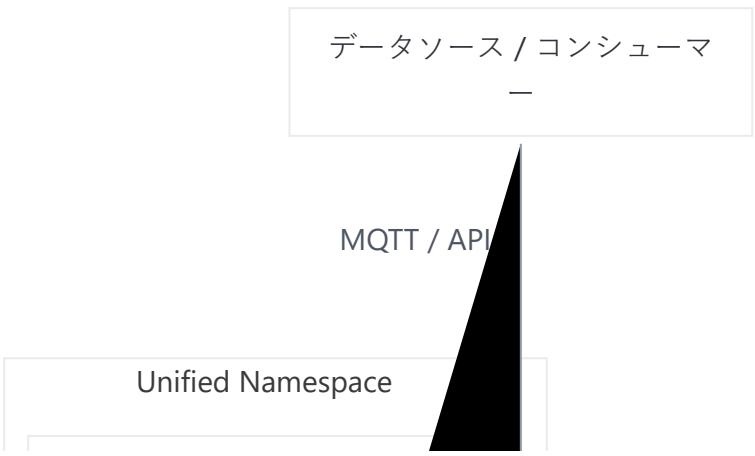
- 最適なバージョンを選ぶ - 各バージョンを比較し、最適なものを選択します
- **UNS の概念** - Unified Namespace におけるデータモデリングを理解します

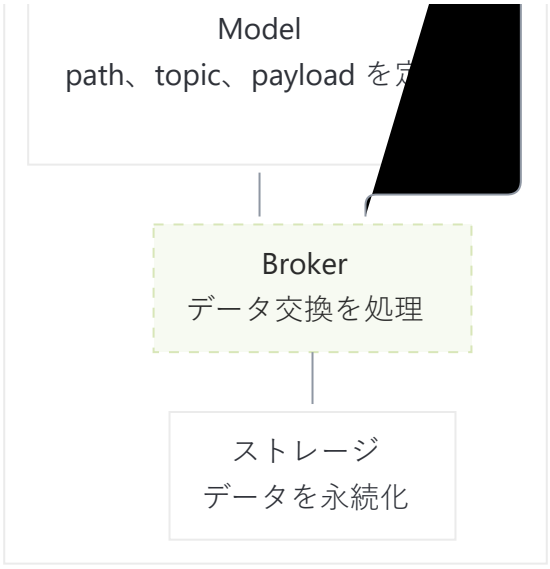
工場データ基盤

Unified Namespace モデル：Metric、State、Action、および UNS が工場データを保存する方法。

Tier0 では、工場データは UNS（Unified Namespace）の考え方に基づき、明確なツリー構造のモデルとして整理され、統一されたデータ基盤を形成します。

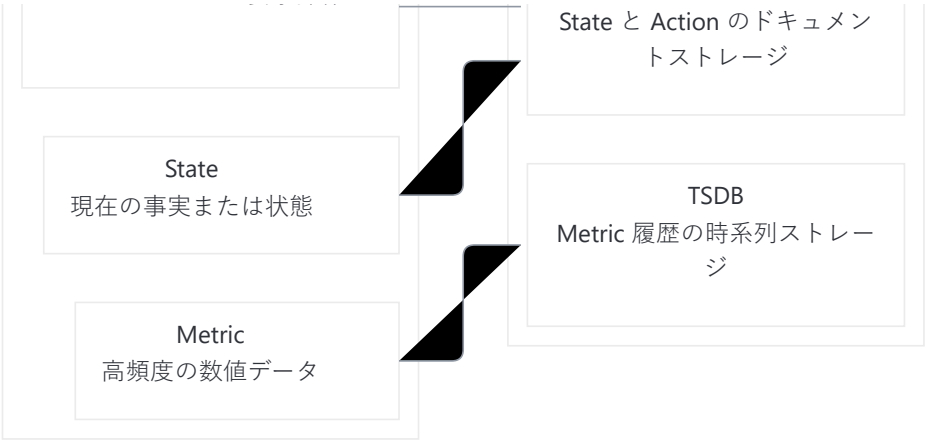
- Broker は model topic を通じてデータを受信、送信します。





- Model は **Metric**、**State**、**Action** の 3 種類に分類されます。
- データは model type に応じて異なるデータベースに保存されます。
  - **Action/State**: JSONB (PostgreSQL)
  - **Metric**: リアルタイムデータ（TSDB）





Broker

UNS broker は、データモデルに基づいて MQTT または OpenAPI 経由でデータを受信、送信するプラットフォームコンポーネントです。

- **MQTT の場合**：model tree 内の各 node の完全な path が topic address として機能し、message payload を渡します。
- **OpenAPI の場合**：HTTP request を通じて data model に対する CRUD コマンドを実行します。

> **API リファレンス**
> API の詳細は **API リファレンス** を参照してください。

Model

> **モデルを作るとき、どのガイドに従うべきですか？**
> データが何を表しているかに基づいて、モデリング方法を選びます。
>
> - **Physical Data**：物理機械からのリアルタイムデータです。通常は **ISA-95** 標準に基づいて整理され、場所の階層に強く依存します。

- **Business Data**：生産要件、スケジュール、タスク、ディスパッチに関係するデータです。通常は **business domain** と **object** によって整理されます。

Metric, State, Action

産業データは変化頻度と用途が異なるため、Tier0 は UNS model を 3 つの固定 type に分けます。

Type	Object	Example
`METRIC`	継続的に収集され、時系列保存が必要な高頻度データ向けに設計されています。	Machine temperature/pressure/voltage
`STATE`	変化頻度が比較的低く、object の現在状態または事実を表すデータ向けに設計されています。	Machine operational status
`ACTION`	外部システムまたは operator が実行すべき内容を記録する低頻度の command data 向けに設計されています。	Start batch, stop machine

Path/Topic, Payload

各 UNS model には path、topic、payload が含まれます。これらが一緒になって、business context を持ち産業データを運ぶ **tree** 構造を形成します。

- **Path/Topic**：path はデータの所属を表し、topic は message payload を運びます。これらが ***model address*** を定義します。
- **Payload**：topic が保持する message payload です。**key-value** 形式の JSON で、***model data*** を定義します。

たとえば、UNS 内の model は次のような tree になります。

タブ: PHYSICAL DATA

```
Factory_A
└── Site_01
    └── SMT_Line_1
        └── Metric
            └── Machine_001
```

- **Path**: `Factory_A/Site_01/SMT_Line_1/Metric`
- **Topic**: `/Machine_001` .
- **Payload**

```
"temperature": 85,
"vibration": 2.8,
```

タブ: BUSINESS DATA

```
Factory_A
└── ERP
    └── ProductionOrders
        └── State
            └── OrderList
```

- **Path**: `Factory_A/ERP/ProductionOrders/State`
- **Topic**: `/OrderList` .
- **Payload**

```
"orders": "0123569876",
"updated_at": 2026-7-13,
```

🚀 モデルを構築するには
データモデルを構築するを参照してください。

Storage

Tier0 は PostgreSQL を使ってデータを保存します。データ type によって変化速度と保存要件が異なります。

Data Type	Storage	Description
`METRIC`	TSDB	温度や圧力などの高頻度測定値に最適化されています。各 topic には default field として `timestamp` と `quality` が含まれます。
`STATE`	PostgreSQL	machine または system state を柔軟な `JSONB` 形式で保存します。
`ACTION`	PostgreSQL	command または action を柔軟な `JSONB` 形式で保存します。

次へ

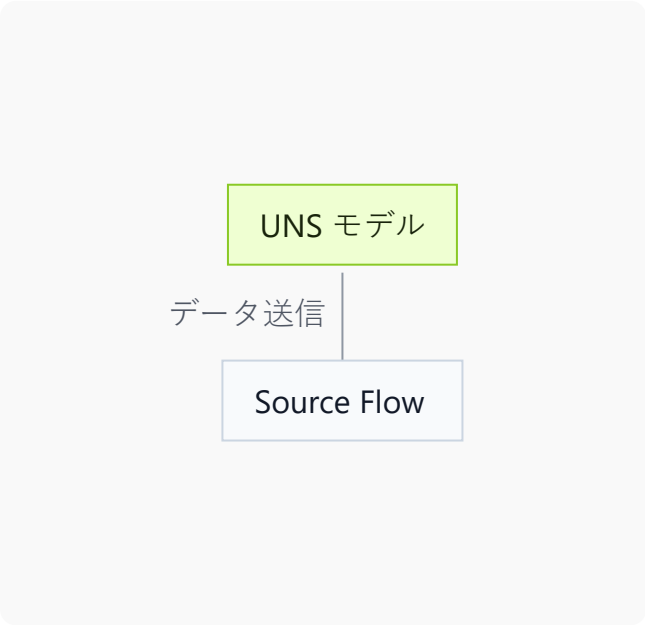
- UNS にデータを接続する - 工場を UNS にモデル化し、データをモデルへ接続する方法。
- 工場データを扱う - MQTT と API を通じて UNS model を操作する方法。

TIER0 の使い方

データ基盤を準備する

namespace モデルを構築し、Source Flow でデータを取り込みます。すべてのエディションで利用できます。

まず **UNS** でモデルを構築し、その後 **Source Flow** でデータをモデルへ接続して、後続利用のためのデータ基盤を作ります。



データモデルを構築する

次の内容に基づいて、データ階層をツリーマップとして定義します。

- データが発生する物理的な場所（ISA-95 を参照）；
- データが従う業務ロジック。

モデルを手動で構築する

1. モデル構造を設計し、topic を追加する前に順序どおりに path を追加します。
2. データ特性に基づいて、**Metric**、**State**、**Action** から **Topic Type** を設定します。
3. **Mock Data** を選択してモデルへシミュレーションデータを送信し、**Enable History** を有効にしてデータをデータベースへ保存します。

> ⓘ ノート
> シミュレーションデータをモデルへ送信する flow が **Flows** に作成されます。

4. **Auto Parsing** または **Pre-defined** を使って、正しいデータ型のフィールドを topic に追加します。
   - **Auto Parsing**：JSON テキストからフィールドを一括で自動変換します。
   - **Pre-defined**：topic のフィールドを 1 つずつ手動で設定します。

モデルをインポートする

> 🚀 ヒント
> ChatGPT などの LLM を使ってモデルのインポートを支援できます。

1. **Import** からテンプレート JSON を AI に送り、次のような prompt を使用します。

```
テンプレートに基づき、xx 工場の xx 用途で使う UNS モデルを生成してください。xx 設備とデータソースを含めてください。
```

2. 生成された結果を UNS にインポートします。

データを UNS に接続する

データフローの特徴

- Node-RED をベースにしています。
- すべての flow は `mqtt out` ノードで終わります。このノードは MQTT client として UNS broker にデータを publish します。
- UNS broker は、flow と同じ名前の `mqtt out` と `mqtt in` ノードに組み込まれています。
- UNS モデルを topic として使用すると、データは **UNS** の対応するモデルへ直接入ります。

手順

1. データソースの種類に応じてノードを使って flow を構築し、最後を `mqtt out` ノードにします。

   > 🚀 必要なノードがない場合
   > 要件に合わせて Node-RED コミュニティからノードをインストールできます。公式コミュニティノードは、多くの場合 `node-red-contrib` で始まります。

2. `mqtt out` ノードの **Server** が UNS broker に設定され、topic が **UNS** のモデルになっていることを確認します。

Flow のバージョン管理

データ flow のバージョンを保存すると、以前のバージョンへロールバックしたり、履歴バージョンから新しい flow を開始したりできます。

> ⚠ 注意
> Edge edition ではバージョンロールバックはサポートされていません。

次へ

- 工場データでアプリを構築 - UNS データを使って産業アプリケーションを構築します。
- データを分析する - Marimo Notebook と Python で UNS データを分析します。

センターからエッジのデータを管理する

センターノードから、接続されたエッジノード上のデータをシームレスに管理します。

実際の工場では、ネットワーク帯域幅の使用量を抑え、データセキュリティを向上させるため、データをエッジデバイスで収集し、センターノードから一元管理することがよくあります。 Tier0 は、センターノードからエッジノード上のデータを管理する Fleet Management ソリューションを提供します。

エッジノードをセンターノードに接続する

1. まず、センターノードで Fleet ノードを作成します。

   > ⓘ ノート
   > 接続できるエッジノードの数はライセンスによって異なります。

2. センターノードで生成されたトークンを使用して、各エッジノードを接続します。

エッジとセンターの間でデータを同期する

- UNS データは、エッジノードからセンターノードへの方向にのみ選択して同期できます。
- センターノードからエッジノードにユーザーを割り当て、ノードを管理できます。

次へ

- 工場データをモデリングする — UNS で工場データをモデリングする方法を確認します。
- データ基盤を準備する — namespace をモデリングし、ソースデータを接続します。

TIER0 の使い方

工場データを扱う

MQTT と API で UNS データを操作し、API リファレンスを参照します。

MQTT と API を通じて UNS データを操作できます。

MQTT

MQTT clients は、Tier0 で生成された credentials を使って UNS broker に接続できます。

> ⓘ ノート
> **MQTTX**、**MQTT Explorer**、**Node-RED** などの MQTT clients を使って broker に接続し、messages を publish/subscribe できます。

MQTT Credentials を生成する

Tier0 の **Edge** タブで新しい credential を作成し、記録します。

MQTT Clients を接続する

> 🚀 ヒント
> 以下の clients は、credentials を使って UNS broker に接続する方法を示す例です。

タブ: MQTTX

1. MQTTX をインストールして開きます。
2. UNS broker の情報と生成された credentials を使って connection を追加します。

```
{
  "Host": "mqtt.tier0.dev",
  "Port": 8883,
  "Client ID": "<generated client ID>",
  "Username": "<generated username>",
  "Password": "<generated password>"
```

```
}
```

タブ: MQTT EXPLORER

1. MQTT Explorer をインストールして開きます。
2. UNS broker の情報と生成された credentials を使って新しい connection を追加します。

```
{
  "Protocol": "mqtt://",
  "Host": "mqtt.tier0.dev",
  "Port": 8883,
  "Username": "<generated username>",
  "Password": "<generated password>"
}
```

3. Change the client ID to the credential ID in **ADVANCED** before connecting.

タブ: NODE-RED

- **Event Flow** を使用 - Tier0 内の Node-RED ベースのデータ処理モジュールです。
  1. **Flows** で **Event Flow** を作成し、`mqtt in` node から開始します。
  2. node 内で UNS broker を **Server**（flow と同じ名前）として選択し、**Topic** を UNS model に設定します。
- 独立した **Node-RED Instance** を使用
  1. Node-RED instance をインストールして開きます。
  2. `mqtt in` node に、次の情報で server を追加します。

```
{
  {/* under Connection */}
  "Server": "mqtt://mqtt.tier0.dev",
  "Port": 8883,
  "Client ID": "<generated clientId>",
  {/* under Security */}
  "Username": "<generated username>",
  "Password": "<generated password>"
```

```
}
```

接続をテストする

1. client status が正常であることを確認します。
2. UNS model topic をコピーし、client から message を publish します。

API

REST API を使用して UNS データを操作します。

> **API の使い方**
> OpenAPI base path は `/openapi/v1` です。例: `https://your-workspace.tier0.app/openapi/v1`。API key は `x-api-key` header で渡します。

API Key を取得する

API keys は **Settings > API Keys** にあります。account type によって作成できる key type が異なります。

- **Service Key**: **Admin** と **Owner** のみ作成できます。
- **Personal Key**: 誰でも作成でき、permission scope は account と同じです。

> **Key permission vs Account permission**
> Account permission は key permission より優先されます。key permission を **Full Access** に設定しても、account がアクセスできないデータにはこの key でもアクセスできません。

例

- `/uns/create`

  UNS path または topic を作成します。`namespace` を使って node tree を定義します。

タブ: JAVASCRIPT

```
const BASE_URL = process.env.TIER0_BASE_URL;
const API_KEY = process.env.TIER0_API_KEY;

const response = await fetch(`${BASE_URL}/uns/create`, {
  method: 'POST',
  headers: {
    'content-type': 'application/json',
    'x-api-key': API_KEY,
  },
  body: JSON.stringify({
    namespace: [
      {
        name: 'DemoFactory',
        type: 'PATH',
        children: [
          {
            name: 'Site_01',
            type: 'PATH',
            children: [
              {
                name: 'Production',
                type: 'PATH',
                children: [
                  {
                    name: 'Produced_Qty',
                    type: 'TOPIC',
                    topicType: 'METRIC',
                    displayName: 'Produced Quantity',
                    description: 'Produced quantity from the production
line.',
                    enableHistory: true,
                    fields: [
                      { name: 'value', type: 'DOUBLE', unit: 'pcs' },
                    ],
                  },
                ],
              },
            ],
          },
        ],
      },
    ],
  }),
});

const result = await response.json();
```

```
console.log(result.data.results);
```

タブ: PYTHON

```
import os
import requests

base_url = os.environ["TIER0_BASE_URL"]
api_key = os.environ["TIER0_API_KEY"]

response = requests.post(
    f"{base_url}/uns/create",
    headers={"x-api-key": api_key},
    json={
        "namespace": [
            {
                "name": "DemoFactory",
                "type": "PATH",
                "children": [
                    {
                        "name": "Site_01",
                        "type": "PATH",
                        "children": [
                            {
                                "name": "Production",
                                "type": "PATH",
                                "children": [
                                    {
                                        "name": "Produced_Qty",
                                        "type": "TOPIC",
                                        "topicType": "METRIC",
                                        "displayName": "Produced
Quantity",
                                        "description": "Produced quantity
from the production line.",
                                        "enableHistory": True,
                                        "fields": [
                                            {"name": "value", "type":
"DOUBLE", "unit": "pcs"},
                                        ],
                                    },
                                ],
                            },
                        ],
                    },
                ],
            },
```

```
        ],
    },
    timeout=30,
)
response.raise_for_status()

result = response.json()
print(result["data"]["results"])
```

タブ: CURL

```
curl -X POST "$TIER0_BASE_URL/uns/create" \
  -H "content-type: application/json" \
  -H "x-api-key: $TIER0_API_KEY" \
  -d '{
    "namespace": [
      {
        "name": "DemoFactory",
        "type": "PATH",
        "children": [
          {
            "name": "Site_01",
            "type": "PATH",
            "children": [
              {
                "name": "Production",
                "type": "PATH",
                "children": [
                  {
                    "name": "Produced_Qty",
                    "type": "TOPIC",
                    "topicType": "METRIC",
                    "displayName": "Produced Quantity",
                    "description": "Produced quantity from the production
line.",
                    "enableHistory": true,
                    "fields": [
                      { "name": "value", "type": "DOUBLE", "unit": "pcs"
}
                    ]
                  }
                ]
              }
            ]
          }
        ]
      }
```

```
    ]
  }'
```

- `/uns/read`

  1 つ以上の UNS topics の最新値とメタデータを読み取ります。

タブ: JAVASCRIPT

```
const BASE_URL = process.env.TIER0_BASE_URL;
const API_KEY = process.env.TIER0_API_KEY;

const response = await fetch(`${BASE_URL}/uns/read`, {
  method: 'POST',
  headers: {
    'content-type': 'application/json',
    'x-api-key': API_KEY,
  },
  body: JSON.stringify({
    topics: [
      'DemoFactory/Site_01/Production/Produced_Qty',
    ],
    include_leaf_value: true,
    include_metadata: true,
  }),
});

const result = await response.json();
console.log(result.data.results);
```

タブ: PYTHON

```
import os
import requests

base_url = os.environ["TIER0_BASE_URL"]
api_key = os.environ["TIER0_API_KEY"]

response = requests.post(
    f"{base_url}/uns/read",
    headers={"x-api-key": api_key},
    json={
```

```
        "topics": [
            "DemoFactory/Site_01/Production/Produced_Qty",
        ],
        "include_leaf_value": True,
        "include_metadata": True,
    },
    timeout=30,
)
response.raise_for_status()

result = response.json()
print(result["data"]["results"])
```

タブ: CURL

```
curl -X POST "$TIER0_BASE_URL/uns/read" \
  -H "content-type: application/json" \
  -H "x-api-key: $TIER0_API_KEY" \
  -d '{
    "topics": [
      "DemoFactory/Site_01/Production/Produced_Qty"
    ],
    "include_leaf_value": true,
    "include_metadata": true
  }'
```

- `/uns/update`

  `path` で UNS node を更新します。`updateMask` を使用して変更するフィールドを指定します。

タブ: JAVASCRIPT

```
const BASE_URL = process.env.TIER0_BASE_URL;
const API_KEY = process.env.TIER0_API_KEY;

const response = await fetch(`${BASE_URL}/uns/update`, {
  method: 'POST',
  headers: {
    'content-type': 'application/json',
    'x-api-key': API_KEY,
```

```
  },
  body: JSON.stringify({
    path: 'DemoFactory/Site_01/Production/Produced_Qty',
    displayName: 'Produced Quantity',
    description: 'Updated production output quantity.',
    updateMask: ['displayName', 'description'],
  }),
});

const result = await response.json();
console.log(result.data);
```

タブ: PYTHON

```
import os
import requests

base_url = os.environ["TIER0_BASE_URL"]
api_key = os.environ["TIER0_API_KEY"]

response = requests.post(
    f"{base_url}/uns/update",
    headers={"x-api-key": api_key},
    json={
        "path": "DemoFactory/Site_01/Production/Produced_Qty",
        "displayName": "Produced Quantity",
        "description": "Updated production output quantity.",
        "updateMask": ["displayName", "description"],
    },
    timeout=30,
)
response.raise_for_status()

result = response.json()
print(result["data"])
```

タブ: CURL

```
curl -X POST "$TIER0_BASE_URL/uns/update" \
  -H "content-type: application/json" \
  -H "x-api-key: $TIER0_API_KEY" \
  -d '{
```

```
    "path": "DemoFactory/Site_01/Production/Produced_Qty",
    "displayName": "Produced Quantity",
    "description": "Updated production output quantity.",
    "updateMask": ["displayName", "description"]
  }'
```

- `/uns/delete`

  1 つ以上の UNS models を削除します。`hard_delete: false` を使用すると models をごみ箱へ移動します。

タブ: JAVASCRIPT

```
const BASE_URL = process.env.TIER0_BASE_URL;
const API_KEY = process.env.TIER0_API_KEY;

const response = await fetch(`${BASE_URL}/uns/delete`, {
  method: 'POST',
  headers: {
    'content-type': 'application/json',
    'x-api-key': API_KEY,
  },
  body: JSON.stringify({
    topics: [
      'DemoFactory/Site_01/Production/Produced_Qty',
    ],
    hard_delete: false,
  }),
});

const result = await response.json();
console.log(result.data);
```

タブ: PYTHON

```
import os
import requests

base_url = os.environ["TIER0_BASE_URL"]
api_key = os.environ["TIER0_API_KEY"]
```

```python
response = requests.post(
    f"{base_url}/uns/delete",
    headers={"x-api-key": api_key},
    json={
        "topics": [
            "DemoFactory/Site_01/Production/Produced_Qty",
        ],
        "hard_delete": False,
    },
    timeout=30,
)
response.raise_for_status()

result = response.json()
print(result["data"])
```

タブ: CURL

```bash
curl -X POST "$TIER0_BASE_URL/uns/delete" \
  -H "content-type: application/json" \
  -H "x-api-key: $TIER0_API_KEY" \
  -d '{
    "topics": [
      "DemoFactory/Site_01/Production/Produced_Qty"
    ],
    "hard_delete": false
  }'
```

API リファレンス

- Tier0 OpenAPI リファレンス

> ⚠ **Tier0 Edge の API key 制限**
> Tier0 Edge から取得した API key は UNS API と Flows API のみに有効です。

次へ

- UNS 上にアプリを構築 — UNS データで産業アプリケーションを構築します。
- データを分析する — Marimo Notebook と Python で UNS データを分析します。

TIER0 の使い方

アプリを構築する

AI との対話で、工場データ、アクセス制御、バージョン管理、デプロイ機能を備えた産業アプリを構築します。

AI との対話を通じて、カスタム産業アプリケーションを構築します。アプリケーションは工場データを利用でき、ロールベースのアクセス制御、バージョン管理、デプロイなどの機能を備えています。

Tier0 アプリを構築する

アプリケーションを構築する前に、project を作成します。

- Project はアプリケーションを分類し、アプリケーションのアクセス制御を管理するために使用します。
- 同じ project 内のアプリケーションは、複雑なシステムの関連する機能モジュールとして扱えます。
- 各 project のデータは、他の project から独立しています。

Prompt を書く

Agent が要件に合ったアプリケーションを生成できるように、prompt には*少なくとも*次の内容を含めてください。

- アプリケーションの目的：このアプリケーションは何のためのものですか？

  ```
  工場の技術者向けに、設備保全アプリケーションを作成してください。
  ```

- アプリケーションデータ：アプリケーションはどのデータを使用し、そのデータはどこにありますか？

  タブ: PLATFORM PLAN

  Prompt で UNS topic を指定すると、アプリケーションは UNS からデータを取得し、UNS にデータを公開できます。

```
このアプリケーションは、xxx UNS model の設備ステータス、アラーム記録、保全履歴を使用し、更新内容を xxx UNS topic に公開してください。
```

タブ: BUILDER PLAN

アプリケーションは UNS に接続しません。アプリケーションの使用中に生成されたデータはデータベースに保存されます。

- アプリケーション機能：アプリケーションは何を行いますか？

```
このアプリケーションでは、ユーザーが設備状態を監視し、保全依頼を送信し、修理進捗を追跡できるようにしてください。
```

> ⚠ **Builder を使用している場合**
> **Builder** plan は現在 UNS をサポートしていませんが、シミュレーションデータを利用できます。より具体的なデータ統合が必要な場合は、sales@freezonex.io までお問い合わせください。

> 🚀 **より速くアプリケーションを構築する**
> - 一般的な産業アプリケーションのテンプレートは **Library** に用意されています。テンプレートを利用し、その上で機能を変更すると、より速くアプリケーションを構築できます。
> - 組み込み **Builder Skills** を使ってアプリケーション構築を支援できます。
> - 会話が長くなりすぎた場合は、`/clear` でコンテキストをクリアし、`/compact` でコンテキストを圧縮します。

アクセス制御

ロールベースのアクセス制御は、projects とアプリケーションの両方に適用されます。

- Project access levels
  - **Owner**：project への完全なアクセス権を持ちます。

    - **Can Edit**：project の削除を除き、project への完全なアクセス権を持ちます。
    - **Can Launch**：**Launchpad** で project を表示し、許可されたアプリケーションだけを使用できます。
- Application roles
    - 明示的に定義されていない場合、アプリケーションのビジネスロジックに基づいて agent が自動生成します。責任範囲が一致する場合は、他のアプリケーションの既存ロールを再利用できます。
    - Project メンバーにのみ割り当てられます。
    - Prompt でロール不要と明示した場合、アプリケーションはすべての project メンバーに公開されます。

> 🚀 独自のロールを割り当てたい場合
> Prompt にロールと権限範囲を記載してください。Agent はそれを他のアプリケーションで生成済みの既存ロールと比較します。競合がある場合、agent が確認します。

> ⚠ ロール制御されたアプリケーションにはアプリケーションロールの割り当てが必須です
> **Launchpad** でロール制御されたアプリケーションにアクセスするには、アプリケーションロールが割り当てられている必要があります。Workspace/project owner であっても、アプリケーションロールがない場合はそのアプリケーションにアクセスできません。

バージョン管理

Tier0 上のアプリケーションには 3 つの状態があります。

- **Draft**：アプリケーションの最新状態です。デプロイしない限り、このバージョンへの変更は他のバージョンに影響しません。
- **Snapshot**：snapshot 作成時点のアプリケーションを保存します。Snapshot バージョンをデプロイすると、現在の **Active** バージョンが置き換えられます。
- **Active**：現在オンラインで稼働しているアプリケーションです。

> 🚀 バージョン切り替え
> - Draft をデプロイすると、active アプリケーションは新しいバージョンに更新されます。

- Snapshot をデプロイすると、active アプリケーションは以前のバージョンにロールバックされます。

外部へエクスポート

アプリケーションの準備ができたら、**Preview** ページから外部のサードパーティ環境または Tier0 Enterprise 向けにエクスポートできます。

- **Download Source Code**：二次開発用にアプリケーションのソースコードをエクスポートします。
- **Export Bundle**：**Tier0 Enterprise** でローカルインポートするための専用エクスポートです。

> 🚀 **Snapshot のソースコードをダウンロードする**
> Snapshot を保存してからダウンロードすると、その snapshot バージョンのソースコードを取得できます。

次へ

- データを分析する - Marimo Notebook と Python で UNS データを分析します。
- ベストプラクティス：産業プロトコルの接続 - 一般的なデータソース接続の例。

TIER0 の使い方

Digital Twin を構築する

ライブ UNS データを 3D モデルにバインドし、完全な工場シーンとして組み立てて Digital Twin 可視化を行います。

Tier0 に接続された工場データは、リアルな動きと環境を持つ Digital Twin として可視化できます。流れは、データを 3D モデルへマッピングし、その後モデルを組み合わせて完全な工場シーンを構築する形です。

> ⓘ **2D ダッシュボードを探していますか?**
> このページは **3D Digital Twin 可視化** —— UNS データを 3D モデルにマッピングし、工場シーンを組み立てる方法を扱います。 **2D 分析ダッシュボード**（チャート、テーブル、KPI パネル）を Notebook で構築する場合は Building Analytics Apps を参照してください。

データを 3D モデルにマッピングする

1. **Model** にモデルをアップロードしたあと、UNS データでリアルタイム動作を制御するノードを選択します。
2. 同じモデルに異なる UNS データを持つインスタンスを追加し、複数の機械をシミュレートします。
3. データをモデルにバインドし、**Preview** でデータの変化に合わせて移動や回転が行われるか確認します。
   - **Position**: モデルは選択した座標方向へ移動します。
   - **Rotation**: モデルは座標の原点を中心に回転します。
4. **Data Tagging** を使用して、静的情報表示用のタグを追加します。

データを可視化する

1. **Instance Library** から、シーン内に存在すべきインスタンスを追加します。
2. 上部のツールを使ってモデルを移動、拡大縮小、回転させ、シーンに合わせます。視点と座標位置も調整できます。
3. 右側の **Object** でモデル位置を確認できます。ここにはバインドされた UNS データが表示されます。

4 ビューポートとライティングは環境の詳細を調整するために使用します。

外部共有

共有できるのはインスタンスのみで、モデル単位またはシーン単位で共有できます。

- **Share Model**: リアルタイム UNS データ付きのモデルを表示する Tier0 プラットフォームリンクを生成します。
- **Share Scene**: リアルタイム UNS データと環境設定を含むモデルおよびシーンを表示する Tier0 プラットフォームリンクを生成します。

TIER0 の使い方

データを分析する

Tier0 Notebook で live namespace data をクエリし、インタラクティブ分析や機械学習アプリを構築します。Cloud と Enterprise で利用できます。

Tier0 では Marimo Notebook を使用し、UNS にモデリングおよび接続された工場データをもとに、**Python** スクリプトで高度なデータ分析を行います。

> 始める前に
> **Enable History** が有効な UNS データだけが、**Notebook** で利用できるデータソースとして表示されます。

1. UNS データで **VARIABLES** > **PostgreSQL** > **tenant_id** > **uns** を選択します。
2. **GENERATE WITH AI** をクリックし、**@** を使って分析対象の変数を選択します。
   Notebook は変数検索のために fuzzy match をサポートします。
3. 必要に応じて分析を続けるか、生成結果をロールバックします。

> **Notebook の AI 支援**
> - Tier0 Cloud では AI 支援を直接利用できます。
> - Tier0 Enterprise では独自の AI key を設定する必要があります。

次へ

- 工場データでアプリを構築 - UNS データで産業アプリケーションを構築します。
- 工場データ基盤 - Unified Namespace のデータモデリング基盤を理解します。

TIER0 の使い方

Agents で Tier0 を操作する

AI agents が Skills と CLI コマンドで Tier0 を操作する方法。

Tier0 は、AI agents がプラットフォームを操作するための Skills と CLI コマンドを提供します。

好みの **AI agent** で **Tier0** を操作します。

Claude Code　Codex　Gemini CLI　Cursor　GitHub Copilot

Skills と CLI のインストール

環境に合うインストーラーを選択します。

Node.js がある場合

Node.js 16 以降が利用できる場合は、`npx` で CLI と skills をインストールします。

```
npx -y @tier0/cli@latest install
```

このコマンドは完全なインストーラーを実行し、次の 2 つを行います。

- `tier0` CLI binary を `~/.tier0/bin/` にインストールします。
- 組み込みの Tier0 Skill baseline をインストールし、Codex、Claude Code など検出された agents に同期します。

Node.js がない場合

1. `tier0` CLI binary をインストールします。

タブ: CURL

```
curl -fsSL https://raw.githubusercontent.com/FREEZONEX/Tier0-cli/main/install.sh | bash
```

タブ: IWR

```
iwr https://raw.githubusercontent.com/FREEZONEX/Tier0-cli/main/install.ps1 | iex
```

2 最新の Tier0 Skills をダウンロードし、検出された agents に同期します。

```
tier0 skills update
```

3 インストール済み Skill のバージョンと health を確認します。

```
tier0 skills status
```

Skills の管理

- インストール済み Skill のバージョンと health を確認します。

```
tier0 skills status
```

- 欠落または破損した組み込み Skill baseline を修復します。

```
tier0 skills install
```

- 最新の Tier0 Skill に更新します。

```
```

```
tier0 skills update
```

Skills と CLI の連携

Tier0 CLI は実行可能なコマンドを提供し、Skills は agents がそれらのコマンドを選択して安全に使うためのガイドを提供します。

> ⓘ ノート
> Tier0 Skills は prompt によって自動的に起動され、agents に何をすべきか、どのコマンドを使うべきかを案内するように設計されています。

例

- Prompt： `Plant/Line1/Metric/Temperature` の現在温度を確認します。
- Skill
  - `tier0-uns`
  - 現在値は完全な topic path から読み取る必要があることを agent に伝え、適切な UNS read 操作を選択します。
- CLI： `tier0 uns read Plant/Line1/Metric/Temperature --json`

Skills

Skill	目的
`tier0`	Tier0 タスクを適切な skill にルーティングし、共通のセットアップ、認証、安全ルールを提供します。
`tier0-uns`	Browse、read、write、history、search、create、update、delete、restore などの UNS 操作を agents に案内します。
`tier0-flow`	SourceFlow と EventFlow の管理を agents に案内します。これには確認、エクスポー

Skill	目的
	ト、作成、更新、デプロイ、削除が含まれます。
`tier0-mqtt`	MQTT 認証情報の管理、publish、範囲を限定したリアルタイム subscribe を agents に案内します。
`tier0-files`	Upload、download、アクセス URL 生成、delete などの Tier0 ファイル操作を agents に案内します。

> **agent に skill repo を読ませる**
> 詳細については、agent に Tier0 Skill repo を読ませてください。

CLI 操作

操作対象インスタンスの設定

> ⓘ ノート
> Cloud はデフォルトで `https://tier0.dev` を使用するため、base URL の設定は不要です。

プライベートデプロイでは、ログイン前に base URL を設定する必要があります。設定後、以降のすべてのコマンドはそのインスタンスで実行されます。

```
tier0 config --base-url https://your-tier0.example.com
tier0 login # interactive browser login
```

認証

タブ: API KEY がある場合

1. 操作対象の Tier0 インスタンスから取得した API key を設定します。

```
tier0 config --api-key <api-key>
```

2 API key の identity と permissions を確認します。

```
tier0 auth whoami --json
```

3 `tier0` CLI で他の操作を実行します。

タブ: API KEY がない場合

1 ブラウザー認証リンクを取得します。

```
tier0 login --no-wait --json
```

2 返された `verification_url` にアクセスし、ブラウザーで認証を完了します。

3 返された `setup_code` でログイン結果を確認します。

```
tier0 login --setup-code <setup_code>
```

> ⚠ 重要
> API key の設定やブラウザーログインを開始する前に、`config` を実行して操作対象インスタンスを設定してください。そうしないと、CLI が誤ったインスタンスを操作する可能性があります。

操作のプレビュー

`--dry-run` を使用すると、対応する書き込み操作を Tier0 に送信せずに検証およびプレビューできます。

対応操作

- UNS: `write/create/update/delete/restore`
- Flow: `create/update/delete/deploy`
- API: `tier0 api`
- MQTT: `auth create/auth delete/publish`
- Files: `delete`

例

```
tier0 uns write --topic demo --value '{"value":1}' --dry-run
tier0 flow deploy --id 1 --flows-file flows.json --dry-run --json
tier0 api /openapi/v1/uns/write --body-file body.json --dry-run --json
tier0 mqtt auth create --name my-client --save my-client --dry-run --json
tier0 assets delete --file-path workspace/path/to/file.csv --dry-run --json
```

> ⓘ ノート
> `--json` オプションはリクエストを JSON 形式で出力します。

> ⚠ **request body には有効な JSON が必要です**
> 有効な JSON が提供されていない場合、CLI は不正な JSON を推測したり書き換えたりしません。

返されるプレビュー

> ⓘ ノート
> dry-run 出力では headers と API keys は表示されません。

```
{
  "ok": true,
  "dry_run": true,
  "data": {
```

```
    "api": [
      {"method": "POST", "url": "https://tier0.dev/openapi/v1/uns/write", "body":
{}}
    ]
  }
}
```

エラー処理

`--json` を使用すると、コマンド失敗時にエラーメッセージとコードを含む JSON object が返されます。

```
{"ok":false,"error":
{"type":"validation","subtype":"invalid_argument","param":"--qos","message":"-
-qos must be 0, 1, or 2"}}
```

Skills と CLI のアンインストール

- アンインストールコマンド

```
# Tier0 CLI を削除し、local configuration と skills は保持します。
npx @tier0/cli@latest uninstall

# Tier0 CLI を削除し、local configuration、credentials、cache を削除しますが、skills
は保持します。
npx @tier0/cli@latest uninstall --purge

# skills を削除します
npx @tier0/cli@latest uninstall --remove-skills

# 上記すべてを削除します
npx @tier0/cli@latest uninstall --purge --remove-skills
```

- Global NPM アンインストール

```
npm uninstall -g @tier0/cli
```

- 手動クリーンアップ

```
rm -rf ~/.tier0/bin/tier0 ~/.tier0/skills
npx -y --package=skills -- skills remove tier0 -y -g
```

CLI Reference

Tier0 CLI Reference には、すべての CLI コマンドが一覧化されています。

次へ

- 工場データを操作する — MQTT と API で工場データモデルを操作する方法。

ベストプラクティス

工場データをモデリングする

Demo UNS の設計：topic explosion の回避と payload structure のベストプラクティス。

Demo UNS の構造

> 最初に確認すること
> データをモデル化する前に、次の質問に答えてください。
>
> - UNS からどのデータを利用するか？
> - UNS にどのデータを戻すか？

物理データ

物理的な場所を持つ機械や設備の産業データは、通常 ISA-95 標準に基づいてモデル化します。

デモモデル

タブ: ISA-95 HIERARCHY

```
Enterprise (Level 4)
└── Site (Level 3/4)
    └── Area (Level 3)
        └── Work Center / Process Cell (Level 2/3)
            └── Equipment / Unit (Level 1/2)
                └── Control Module / Device (Level 0/1)
```

タブ: TREE STRUCTURE

```
DemoFactory
└── Site_01
    └── Production
        └── Line_01
```

```
            └── WorkOrderExecution
                ├── Metric
                │   ├── Target_Qty
                │   ├── Produced_Qty
                │   ├── Defect_Qty
                │   └── Completion_Rate
                └── State
                    ├── CurrentWorkOrder
                    └── WorkOrderStatus
```

タブ: JSON


```
{
  "namespace": [
    {
      "name": "DemoFactory",
      "alias": "DemoFactory_9e50bbcd833d",
      "type": "PATH",
      "children": [
        {
          "name": "Site_01",
          "alias": "DemoFactory__6d1fcad14e86",
          "type": "PATH",
          "children": [
            {
              "name": "Production",
              "alias": "__Site_01_Pro_91523d2ad304",
              "type": "PATH",
              "children": [
                {
                  "name": "Line_01",
                  "alias": "__Production__acd5841c1ada",
                  "type": "PATH",
                  "children": [
                    {
                      "name": "WorkOrderExecution",
                      "alias": "__Line_01_Wor_54904ff56e91",
                      "type": "PATH",
                      "children": [
                        {
                          "name": "Metric",
                          "alias": "metric_233fd84305c4",
                          "type": "PATH",
                          "children": [
                            {
                              "name": "Target_Qty",
                              "alias": "Target_Qty_8646b092e543",
```

```
                            "type": "TOPIC",
                            "topicType": "METRIC",
                            "description": "Target quantity for the current
Work Order.",
                            "fields": [
                              {
                                "name": "production_order_id",
                                "type": "STRING"
                              },
                              {
                                "name": "plan_id",
                                "type": "STRING"
                              },
                              {
                                "name": "plan_number",
                                "type": "STRING"
                              },
                              {
                                "name": "work_order_id",
                                "type": "STRING"
                              },
                              {
                                "name": "value",
                                "type": "DOUBLE"
                              },
                              {
                                "name": "unit",
                                "type": "STRING"
                              },
                              {
                                "name": "timestamp",
                                "type": "DATETIME"
                              }
                            ],
                            "enableHistory": "TRUE",
                            "mockData": "FALSE"
                          },
                          {
                            "name": "Produced_Qty",
                            "alias": "Produced_Qty_1f8cd3e11f77",
                            "type": "TOPIC",
                            "topicType": "METRIC",
                            "description": "Produced quantity for the
current Work Order.",
                            "fields": [
                              {
                                "name": "production_order_id",
                                "type": "STRING"
                              },
                              {
                                "name": "plan_id",
```

```
            "type": "STRING"
          },
          {
            "name": "plan_number",
            "type": "STRING"
          },
          {
            "name": "work_order_id",
            "type": "STRING"
          },
          {
            "name": "value",
            "type": "DOUBLE"
          },
          {
            "name": "unit",
            "type": "STRING"
          },
          {
            "name": "timestamp",
            "type": "DATETIME"
          }
        ],
        "enableHistory": "TRUE",
        "mockData": "FALSE"
      },
      {
        "name": "Defect_Qty",
        "alias": "Defect_Qty_b48fb3563ec1",
        "type": "TOPIC",
        "topicType": "METRIC",
        "description": "Defect quantity for the current
Work Order.",
        "fields": [
          {
            "name": "production_order_id",
            "type": "STRING"
          },
          {
            "name": "plan_id",
            "type": "STRING"
          },
          {
            "name": "plan_number",
            "type": "STRING"
          },
          {
            "name": "work_order_id",
            "type": "STRING"
          },
          {
```

```
            "name": "value",
            "type": "DOUBLE"
          },
          {
            "name": "unit",
            "type": "STRING"
          },
          {
            "name": "timestamp",
            "type": "DATETIME"
          }
        ],
        "enableHistory": "TRUE",
        "mockData": "FALSE"
      },
      {
        "name": "Completion_Rate",
        "alias": "Completion_R_7294e38e1530",
        "type": "TOPIC",
        "topicType": "METRIC",
        "description": "Completion percentage for the
current Work Order.",
        "fields": [
          {
            "name": "production_order_id",
            "type": "STRING"
          },
          {
            "name": "plan_id",
            "type": "STRING"
          },
          {
            "name": "plan_number",
            "type": "STRING"
          },
          {
            "name": "work_order_id",
            "type": "STRING"
          },
          {
            "name": "value",
            "type": "DOUBLE"
          },
          {
            "name": "unit",
            "type": "STRING"
          },
          {
            "name": "timestamp",
            "type": "DATETIME"
          }
```

```
                ],
                "enableHistory": "TRUE",
                "mockData": "FALSE"
              }
            ]
          },
          {
            "name": "State",
            "alias": "state_539a0f331fe0",
            "type": "PATH",
            "children": [
              {
                "name": "CurrentWorkOrder",
                "alias": "CurrentWorkO_a638320784dd",
                "type": "TOPIC",
                "topicType": "STATE",
                "description": "Current active Work Order on
Line_01.",
                "fields": [
                  {
                    "name": "production_order_id",
                    "type": "STRING"
                  },
                  {
                    "name": "plan_id",
                    "type": "STRING"
                  },
                  {
                    "name": "plan_number",
                    "type": "STRING"
                  },
                  {
                    "name": "work_order_id",
                    "type": "STRING"
                  },
                  {
                    "name": "target_line",
                    "type": "STRING"
                  },
                  {
                    "name": "workstation",
                    "type": "STRING"
                  },
                  {
                    "name": "product_code",
                    "type": "STRING"
                  },
                  {
                    "name": "target_qty",
                    "type": "DOUBLE"
                  },
```

```
                {
                  "name": "unit",
                  "type": "STRING"
                },
                {
                  "name": "started_at",
                  "type": "DATETIME"
                },
                {
                  "name": "updated_at",
                  "type": "DATETIME"
                }
              ],
              "enableHistory": "TRUE",
              "mockData": "FALSE"
            },
            {
              "name": "WorkOrderStatus",
              "alias": "WorkOrderSta_e24e3bf113d1",
              "type": "TOPIC",
              "topicType": "STATE",
              "description": "Execution status of the current
Work Order.",
              "fields": [
                {
                  "name": "production_order_id",
                  "type": "STRING"
                },
                {
                  "name": "plan_id",
                  "type": "STRING"
                },
                {
                  "name": "plan_number",
                  "type": "STRING"
                },
                {
                  "name": "work_order_id",
                  "type": "STRING"
                },
                {
                  "name": "status",
                  "type": "STRING"
                },
                {
                  "name": "reason",
                  "type": "STRING"
                },
                {
                  "name": "updated_at",
                  "type": "DATETIME"
```

```
                        }
                      ],
                      "enableHistory": "TRUE",
                      "mockData": "FALSE"
                    }
                  ]
                }
              ]
            }
          ]
        }
      ]
    }
  ]
}
```

業務データ

ERP の生産注文や作業指示計画など、業務計画やプロセスオブジェクトについては、業務コンテキストに沿ってデータをモデル化します。

> 🚀 実際の業務データについて
> 業務データをモデル化するときは、システム名だけでなく、業務オブジェクトと利用シーンも確認してください。

業務データ Modeling Guide

```
Factory / Business Domain / Business Object / Topic Type / Topic
```

- **Business Domain**：データが属する業務プロセスまたはシステムを確認します。

  ```
  ERP
  MES
  WMS
  ```

```
QMS
EAM
LIMS
Planning
Inventory
Maintenance
Quality
Logistics
...
```

- **Business Object**：業務ドメインで管理される中心オブジェクトです。

```
ProductionOrders
WorkOrderPlan
MaterialInventory
QualityInspection
MaintenanceOrders
PurchaseOrders
Shipments
...
```

- **Topic Type**：**Metric**、**Action**、**State** からデータタイプを選択します。

Topic Type	Used for	Examples
STATE	現在の状態または現在の事実	作業指示の状態、注文状態、設備状態
METRIC	継続的に変化する数値	温度、速度、生産量、OEE、エネルギー消費
ACTION	外部実行者へのコマンド	設備起動、設備停止、アラームリセット、作業指示の払い出し要求

デモモデル

タブ: TREE STRUCTURE

```
DemoFactory
```

```
└── ERP
    ├── ProductionOrders
    │   └── State
    │       ├── UpsertProductionOrder
    │       └── OrderList
    └── WorkOrderPlan
        ├── Metric
        │   └── SplitCount
        └── State
            ├── PlanStatus
            └── WorkOrderList
```

タブ: JSON


```
{
  "namespace": [
    {
      "name": "DemoFactory",
      "alias": "DemoFactory_f925c1ec5041",
      "type": "PATH",
      "children": [
        {
          "name": "ERP",
          "alias": "DemoFactory__e504805b1973",
          "type": "PATH",
          "children": [
            {
              "name": "ProductionOrders",
              "alias": "__ERP_Product_1e7984ac3a1d",
              "type": "PATH",
              "children": [
                {
                  "name": "State",
                  "alias": "state_c2616c8698fa",
                  "type": "PATH",
                  "children": [
                    {
                      "name": "UpsertProductionOrder",
                      "alias": "UpsertProduc_d5f166da35c6",
                      "type": "TOPIC",
                      "topicType": "STATE",
                      "description": "Event for creating or updating a
Production Order from WOM or ERP simulation.",
                      "fields": [
                        {
                          "name": "event_id",
                          "type": "STRING"
```

```
      },
      {
        "name": "event_type",
        "type": "STRING"
      },
      {
        "name": "source",
        "type": "STRING"
      },
      {
        "name": "production_order_id",
        "type": "STRING"
      },
      {
        "name": "order_number",
        "type": "STRING"
      },
      {
        "name": "product_code",
        "type": "STRING"
      },
      {
        "name": "planned_qty",
        "type": "DOUBLE"
      },
      {
        "name": "unit",
        "type": "STRING"
      },
      {
        "name": "status",
        "type": "STRING"
      },
      {
        "name": "due_date",
        "type": "DATETIME"
      },
      {
        "name": "updated_at",
        "type": "DATETIME"
      }
    ],
    "enableHistory": "TRUE",
    "mockData": "FALSE"
  },
  {
    "name": "OrderList",
    "alias": "OrderList_11fbca51a2bd",
    "type": "TOPIC",
    "topicType": "STATE",
```

```
                    "description": "Retained compatibility snapshot of
current Production Orders.",
                    "fields": [
                      {
                        "name": "orders",
                        "type": "STRING"
                      },
                      {
                        "name": "updated_at",
                        "type": "DATETIME"
                      }
                    ],
                    "enableHistory": "TRUE",
                    "mockData": "FALSE"
                  }
                ]
              }
            ]
          },
          {
            "name": "WorkOrderPlan",
            "alias": "__ERP_WorkOrd_19f04960daf5",
            "type": "PATH",
            "children": [
              {
                "name": "Metric",
                "alias": "metric_17258b809a4f",
                "type": "PATH",
                "children": [
                  {
                    "name": "SplitCount",
                    "alias": "SplitCount_032e86f5fbfc",
                    "type": "TOPIC",
                    "topicType": "METRIC",
                    "description": "Number of executable Work Orders
generated from a Work Order Plan.",
                    "fields": [
                      {
                        "name": "production_order_id",
                        "type": "STRING"
                      },
                      {
                        "name": "plan_id",
                        "type": "STRING"
                      },
                      {
                        "name": "plan_number",
                        "type": "STRING"
                      },
                      {
                        "name": "split_count",
```

```
                    "type": "INTEGER"
                  },
                  {
                    "name": "updated_at",
                    "type": "DATETIME"
                  }
                ],
                "enableHistory": "TRUE",
                "mockData": "FALSE"
              }
            ]
          },
          {
            "name": "State",
            "alias": "state_884cee548bf7",
            "type": "PATH",
            "children": [
              {
                "name": "PlanStatus",
                "alias": "PlanStatus_2e21c1327c49",
                "type": "TOPIC",
                "topicType": "STATE",
                "description": "Current lifecycle status of a Work Order
Plan.",
                "fields": [
                  {
                    "name": "production_order_id",
                    "type": "STRING"
                  },
                  {
                    "name": "plan_id",
                    "type": "STRING"
                  },
                  {
                    "name": "plan_number",
                    "type": "STRING"
                  },
                  {
                    "name": "status",
                    "type": "STRING"
                  },
                  {
                    "name": "updated_at",
                    "type": "DATETIME"
                  }
                ],
                "enableHistory": "TRUE",
                "mockData": "FALSE"
              },
              {
                "name": "WorkOrderList",
```

```
                    "alias": "WorkOrderLis_887b92f26fc2",
                    "type": "TOPIC",
                    "topicType": "STATE",
                    "description": "Split result list generated from a sent
Work Order Plan.",
                    "fields": [
                      {
                        "name": "production_order_id",
                        "type": "STRING"
                      },
                      {
                        "name": "plan_id",
                        "type": "STRING"
                      },
                      {
                        "name": "plan_number",
                        "type": "STRING"
                      },
                      {
                        "name": "split_count",
                        "type": "INTEGER"
                      },
                      {
                        "name": "lines",
                        "type": "STRING"
                      },
                      {
                        "name": "updated_at",
                        "type": "DATETIME"
                      }
                    ],
                    "enableHistory": "TRUE",
                    "mockData": "FALSE"
                  }
                ]
              }
            ]
          }
        ]
      }
    ]
}
```

Topic Explosion を避ける方法

Tier0 uses 3 fixed topic types: `Metric`, `Action`, and `State` to avoid topic explosion.

- 高頻度リアルタイムデータには `Metric` を使う

  高頻度の測定値やテレメトリは、各信号の詳細ごとに namespace を広げるのではなく `Metric` に配置します。

タブ: BAD

```
Factory_A
└── SMT_Line_1
    └── Machine_001
        ├── Temperature_Value
        ├── Temperature_Unit
        ├── Temperature_Status
        └── Speed_Value
```

タブ: BETTER

```
Factory_A
└── SMT_Line_1
    └── Metric
        └── Machine_001
```

Model fields:

```
{
  "temperature": 80,
  "temperature_unit": "Celsius",
  "temperature_status": "Normal",
  "speed": 300
}
```

- 低頻度ステータスデータには `State` を使う

  低頻度の状態、スナップショット、現在の業務状態は `State` に配置します。ジョブ、バッチ、注文などの変化するエンティティごとに新しい topic を作らないようにします。

タブ: BAD

```
Factory_A
└── SMT_Line_1
    └── State
        ├── Job_001
        ├── Job_002
        └── Job_003
```

タブ: BETTER

```
Factory_A
└── SMT_Line_1
    └── State
        └── Machine_001
```

Model fields:

```
{
  "current_job": "Job_003",
  "product": "Product_A"
}
```

- 低頻度アクションデータには `Action` を使う

  コマンド、リクエスト、オペレーター操作は `Action` に配置します。action の入力はパラメータごとに topic を作るのではなく、topic のフィールドとして保持します。

タブ: BAD

```
Factory_A
└── SMT_Line_1
    └── Machine_001
        ├── Start_Command
```

```
        ├── Target_Height
        └── Light_On
```

タブ: BETTER

```
Factory_A
└── SMT_Line_1
    └── Action
        └── StartJob
```

Model fields:

```
{
  "target_height": 12.5,
  "light_on": true,
  "requested_by": "operator"
}
```

Payload 構造

topic payload を設計するときは、消費されるデータと書き戻されるデータの両方を考慮します。

- **Action**

  Action payload には、データが Tier0 データベースに保存された時刻を記録する `_timestamp` が自動的に含まれます。

タブ: START_JOB

```
{
  "light_turn_on": true,
  "height_adjust": 12.5,
  "_timestamp": "2026-07-10T10:00:00Z"
}
```

タブ: STOP_JOB

```
{
  "water_pump_off": true,
  "window_close": true,
  "_timestamp": "2026-07-10T10:00:00Z"
}
```

- **Metric**

  Metric payload には、Tier0 データベースへの保存時刻を記録する `_timestamp` と、データ品質を表す `_quality` が自動的に含まれます。

タブ: ERP / SPLITCOUNT

```
{
  "production_order_id": "PO-20260710-001",
  "plan_id": "PLAN-001",
  "plan_number": "WOP-20260710-001",
  "split_count": 3,
  "updated_at": "2026-07-10T10:00:00Z",
  "_timestamp": "2026-07-10T10:00:00Z",
  "_quality": "Good"
}
```

タブ: SITE_01 / TARGET_QTY

```
{
  "production_order_id": "PO-20260710-001",
  "plan_id": "PLAN-001",
```

```
  "plan_number": "WOP-20260710-001",
  "work_order_id": "WO-001",
  "value": 1200,
  "unit": "pcs",
  "timestamp": "2026-07-10T10:00:00Z",
  "_timestamp": "2026-07-10T10:00:00Z",
  "_quality": "Good"
}
```

- **State**

  State payload には、データが Tier0 データベースに保存された時刻を記録する `_timestamp` が自動的に含まれます。

タブ: ERP / UPSERTPRODUCTIONORDER

```
{
  "event_id": "EVT-001",
  "event_type": "UPSERT",
  "source": "ERP",
  "production_order_id": "PO-20260710-001",
  "order_number": "PO-001",
  "product_code": "SKU-1001",
  "planned_qty": 1200,
  "unit": "pcs",
  "status": "Released",
  "due_date": "2026-07-12T00:00:00Z",
  "updated_at": "2026-07-10T10:00:00Z",
  "_timestamp": "2026-07-10T10:00:00Z"
}
```

タブ: SITE_01 / WORKORDERSTATUS

```
{
  "production_order_id": "PO-20260710-001",
  "plan_id": "PLAN-001",
  "plan_number": "WOP-20260710-001",
  "work_order_id": "WO-001",
  "status": "Running",
  "reason": "In production",
  "updated_at": "2026-07-10T10:00:00Z",
```

```
  "_timestamp": "2026-07-10T10:00:00Z"
}
```

ベストプラクティス

産業プロトコルの接続

Node-RED を使って OPC UA、Modbus、その他の典型的なプロトコルデバイスを UNS に接続し、実際の flow JSON 参照を提供します。

このセクションでは、OPC UA、Modbus、API などの一般的なデータタイプを例に、**Source Flow** を通じてデータソースを **UNS** に接続する流れを説明します。

OPC UA

データフローの設定

1. Tier0 にログインし、**Flow** に移動して新しい **Source Flow** を作成します。
2. flow に **inject**、**OPC UA Client**、**mqtt out** ノードを追加します。
3. 順番に接続し、各ノードをダブルクリックして詳細を設定します。

- **inject**
  1. 左下の **add** をクリックして property entry を追加します。
  2. properties を次の項目に設定します。

```
{
  "msg.payload": "[multiple data tags]",
  "msg.topic": "multiple"
}
```

- **OPC UA Client**

```
"Endpoint": "<OPC UA server address and port>", //e.g.
opc.tcp://127.0.0.1:4850
"Action": "SUBSCRIBE",
"Interval": "<data collecting interval>" //e.g. 10 seconds
```

- **mqtt out**

```
"Server": "<embedded UNS broker>", //same name as the flow
"Topic": "<UNS model>",
```

4 flow を保存して deploy します。

Flow Reference

OPC UA flow JSON を表示


```
[
    {
        "id": "fn_mapper_01",
        "type": "function",
        "z": "b002a07d1163c3de",
        "name": "ISA95 Mapping (Air Compressor)",
        "func": "// OPC UA NodeId → ISA-95 MQTT Topic Mapping\n\nconst mapping
= {\n    \"ns=2;i=2\": {\n        topic:
\"global_plant/Smart_Manufacturing_Group/Suzhou_Plant/Utility_Area/Compressed_
Air_System/Air_Compressor_Station/Air_Compressor_01/Metric/outlet_temperature\
",\n        field: \"temperature\"\n    },\n    \"ns=2;i=3\": {\n
topic:
\"global_plant/Smart_Manufacturing_Group/Suzhou_Plant/Utility_Area/Compressed_
Air_System/Air_Compressor_Station/Air_Compressor_01/Metric/discharge_pressure\
",\n        field: \"pressure\"\n    },\n    \"ns=2;i=4\": {\n        topic:
\"global_plant/Smart_Manufacturing_Group/Suzhou_Plant/Utility_Area/Compressed_
Air_System/Air_Compressor_Station/Air_Compressor_01/Metric/vibration_rms\",\n
field: \"vibration\"\n    }\n};\n\nlet value = msg.payload;\n\n// OPC UA
structure normalize\nif (value && typeof value === \"object\") {\n    if
(value.value?.value !== undefined) {\n        value = value.value.value;\n
} else if (value.value !== undefined) {\n        value = value.value;\n
}\n}\n\nconst m = mapping[msg.topic];\nif (!m) return null;\n\nmsg.topic =
m.topic;\n\nmsg.payload = {\n    [m.field]: Number(value)\n};\n\nreturn msg;",
        "outputs": 1,
        "timeout": "",
        "noerr": 0,
        "initialize": "",
        "finalize": "",
        "libs": [],
        "x": 340,
        "y": 320,
        "wires": [
            [
                "mqtt_out_01"
```

```
            ]
        ]
    },
    {
        "id": "mqtt_out_01",
        "type": "mqtt out",
        "z": "b002a07d1163c3de",
        "name": "MQTT Air Compressor Metrics",
        "topic": "",
        "qos": "0",
        "retain": "false",
        "respTopic": "",
        "contentType": "",
        "userProps": "",
        "correl": "",
        "expiry": "",
        "broker": "broker-djnwpg00y370",
        "x": 570,
        "y": 200,
        "wires": []
    },
    {
        "id": "1f242deefa9fdba9",
        "type": "OpcUa-Client",
        "z": "b002a07d1163c3de",
        "endpoint": "298ef5d50e9dc411",
        "action": "subscribe",
        "deadbandtype": "a",
        "deadbandvalue": 1,
        "time": 10,
        "timeUnit": "s",
        "certificate": "n",
        "localfile": "",
        "localkeyfile": "",
        "securitymode": "None",
        "securitypolicy": "None",
        "useTransport": false,
        "maxChunkCount": 1,
        "maxMessageSize": 8192,
        "receiveBufferSize": 8192,
        "sendBufferSize": 8192,
        "setstatusandtime": false,
        "keepsessionalive": false,
        "name": "",
        "x": 220,
        "y": 220,
        "wires": [
            [
                "fn_mapper_01"
            ],
            [],
```

```
            []
        ]
    },
    {
        "id": "7be5f021ed1f10ef",
        "type": "inject",
        "z": "b002a07d1163c3de",
        "name": "",
        "props": [
            {
                "p": "payload"
            },
            {
                "p": "topic",
                "vt": "str"
            }
        ],
        "repeat": "",
        "crontab": "",
        "once": false,
        "onceDelay": 0.1,
        "topic": "multiple",
        "payload": "[{\"nodeId\":\"ns=2;i=2\",\"datatype\":\"Float\"},
{\"nodeId\":\"ns=2;i=3\",\"datatype\":\"Float\"},
{\"nodeId\":\"ns=2;i=4\",\"datatype\":\"Float\"}]",
        "payloadType": "json",
        "x": 110,
        "y": 160,
        "wires": [
            [
                "1f242deefa9fdba9"
            ]
        ]
    },
    {
        "id": "broker-djnwpg00y370",
        "type": "mqtt-broker",
        "z": "b002a07d1163c3de",
        "name": "emqx:1883",
        "broker": "emqx",
        "port": "1883",
        "clientid": "355424069147520",
        "usetls": false,
        "protocolVersion": "4",
        "keepalive": "60",
        "cleansession": true,
        "birthTopic": "",
        "birthQos": "0",
        "birthPayload": "",
        "closeTopic": "",
        "closeQos": "0",
```

```
        "closePayload": "",
        "willTopic": "",
        "willQos": "0",
        "willPayload": ""
    },
    {
        "id": "298ef5d50e9dc411",
        "type": "OpcUa-Endpoint",
        "endpoint": "opc.tcp://172.31.151.237:4841",
        "secpol": "None",
        "secmode": "None",
        "none": true,
        "login": false,
        "usercert": false,
        "usercertificate": "",
        "userprivatekey": ""
    },
    {
        "id": "a003de8af8e62c75",
        "type": "global-config",
        "env": [],
        "modules": {
            "node-red-contrib-opcua": "0.2.339"
        }
    }
]
```


Modbus

データフローの設定

1. flow に **modbus-read** と **mqtt out** ノードを追加します。
2. 各ノードをダブルクリックして必要な情報を設定します。
    - **modbus-read**
        - **Settings**
            - **FC**: Modbus の function code です。FC 3 は holding registers を読み取ります。
            - **Address**: 読み取りを開始する register address です。通常は 0 から始まります。
            - **Quantity**: The number of consecutive registers to read.
            - **Poll Rate**: Modbus device を読み取る間隔です。たとえば 10 秒です。
            - **Server**: 接続する Modbus server、IP、port、protocol を設定します。

        - Server
            - **Host**: Modbus server の IP。
            - **Port**: Modbus server の port。
    - **mqtt out**

```
"Server": "<embedded UNS broker>", //same name as the flow
"Topic": "<UNS model>",
```

3 flow を保存して deploy します。

Flow Reference

<details>
<summary>Modbus flow JSON を表示</summary>

```
[
    {
        "id": "d3becb19e0d762f4",
        "type": "modbus-read",
        "z": "b002a07d1163c3de",
        "name": "status",
        "topic": "",
        "showStatusActivities": false,
        "logIOActivities": false,
        "showErrors": false,
        "showWarnings": true,
        "unitid": "1",
        "dataType": "HoldingRegister",
        "adr": "0",
        "quantity": "1",
        "rate": "10",
        "rateUnit": "s",
        "delayOnStart": false,
        "startDelayTime": "",
        "server": "c665b74952f6cf5a",
        "useIOFile": false,
        "ioFile": "",
        "useIOForPayload": false,
        "emptyMsgOnFail": false,
        "x": 130,
        "y": 500,
        "wires": [
            [
```

</details>

```
                "2987638967f632b4",
                "09b8538e09d0766a"
            ],
            []
        ]
    },
    {
        "id": "b068c119928c029a",
        "type": "mqtt out",
        "z": "b002a07d1163c3de",
        "name": "status",
        "topic":
"global_plant/Smart_Manufacturing_Group/Suzhou_Plant/Utility_Area/Compressed_A
ir_System/Air_Compressor_Station/Air_Compressor_01/State/compressor_status",
        "qos": "",
        "retain": "",
        "respTopic": "",
        "contentType": "",
        "userProps": "",
        "correl": "",
        "expiry": "",
        "broker": "broker-djnwpg00y370",
        "x": 510,
        "y": 540,
        "wires": []
    },
    {
        "id": "2987638967f632b4",
        "type": "debug",
        "z": "b002a07d1163c3de",
        "name": "debug 3",
        "active": true,
        "tosidebar": true,
        "console": false,
        "tostatus": false,
        "complete": "false",
        "statusVal": "",
        "statusType": "auto",
        "x": 260,
        "y": 600,
        "wires": []
    },
    {
        "id": "09b8538e09d0766a",
        "type": "function",
        "z": "b002a07d1163c3de",
        "name": "function 1",
        "func": "const value = msg.payload;\n\n// 1️⃣ verify\nif (value ===
undefined || value === null) {\n    node.warn(\"Empty Modbus payload\");\n
return null;\n}\n\n// 2️⃣ convert to number\nconst status_code =
Number(value);\n\n// 3️⃣ mapping to status\nconst statusMap = {\n    0:
```

```
\"maintenance\",\n    1: \"running\",\n    2: \"standby\",\n    3:
\"debug\",\n    4: \"fault\"\n};\n\n// 4️⃣ filtering\nif (!(status_code in
statusMap)) {\n    node.warn(\"Invalid status_code: \" + status_code);\n
return null;\n}\n\n// 5️⃣ contextualized output\nmsg.payload = {\n
status_code: status_code,\n    status_label: statusMap[status_code],\n
_valid: true,\n    _ts: new Date().toISOString()\n};\n\n// 6️⃣ optional  set
topic\nmsg.topic = \"v1/aircompressor01/state\";\n\nreturn msg;",
        "outputs": 1,
        "timeout": 0,
        "noerr": 0,
        "initialize": "",
        "finalize": "",
        "libs": [],
        "x": 320,
        "y": 500,
        "wires": [
            [
                "b068c119928c029a"
            ]
        ]
    },
    {
        "id": "c665b74952f6cf5a",
        "type": "modbus-client",
        "name": "modbus",
        "clienttype": "tcp",
        "bufferCommands": true,
        "stateLogEnabled": false,
        "queueLogEnabled": false,
        "failureLogEnabled": true,
        "tcpHost": "172.31.151.237",
        "tcpPort": "5020",
        "tcpType": "DEFAULT",
        "serialPort": "/dev/ttyUSB",
        "serialType": "RTU-BUFFERD",
        "serialBaudrate": 9600,
        "serialDatabits": 8,
        "serialStopbits": 1,
        "serialParity": "none",
        "serialConnectionDelay": 100,
        "serialAsciiResponseStartDelimiter": "0x3A",
        "unit_id": 1,
        "commandDelay": 1,
        "clientTimeout": 1000,
        "reconnectOnTimeout": true,
        "reconnectTimeout": 2000,
        "parallelUnitIdsAllowed": true,
        "showErrors": false,
        "showWarnings": true,
        "showLogs": true
    },
```

```
    {
        "id": "broker-djnwpg00y370",
        "type": "mqtt-broker",
        "z": "b002a07d1163c3de",
        "name": "emqx:1883",
        "broker": "emqx",
        "port": "1883",
        "clientid": "355424069147520",
        "usetls": false,
        "protocolVersion": "4",
        "keepalive": "60",
        "cleansession": true,
        "birthTopic": "",
        "birthQos": "0",
        "birthPayload": "",
        "closeTopic": "",
        "closeQos": "0",
        "closePayload": "",
        "willTopic": "",
        "willQos": "0",
        "willPayload": ""
    },
    {
        "id": "5cc203d0788c324d",
        "type": "global-config",
        "env": [],
        "modules": {
            "node-red-contrib-modbus": "5.43.0"
        }
    }
]
```

API

データフローの設定

1. flow に **inject**、**http request**、**mqtt out** ノードを追加します。
2. 各ノードをダブルクリックして必要な情報を設定します。
    - **http request**

```
"Method": "<API method>",
"URL": "<API URL>",
"Return": "a parsed JSON object",
"Headers": "<API headers>"
```

- mqtt out

```
"Server": "<embedded UNS broker>", //same name as the flow
"Topic": "<UNS model>",
```

3 flow を保存して deploy します。

Flow Reference

API flow JSON を表示

```
[
    {
        "id": "46273fcbb98afa11",
        "type": "inject",
        "z": "932355c6886aaf27",
        "name": "",
        "props": [
            {
                "p": "payload"
            },
            {
                "p": "topic",
                "vt": "str"
            }
        ],
        "repeat": "",
        "crontab": "",
        "once": false,
        "onceDelay": 0.1,
        "topic": "",
        "payload": "",
        "payloadType": "date",
        "x": 170,
        "y": 380,
        "wires": [
            [
                "3aca156076ad6dd1"
            ]
        ]
    },
    {
        "id": "3aca156076ad6dd1",
        "type": "http request",
```

```
        "z": "932355c6886aaf27",
        "name": "",
        "method": "GET",
        "ret": "obj",
        "paytoqs": "ignore",
        "url": "http://127.0.0.1:1880/api/wms/picking-zone-a/status",
        "tls": "",
        "persist": false,
        "proxy": "",
        "insecureHTTPParser": false,
        "authType": "",
        "senderr": false,
        "headers": [],
        "x": 380,
        "y": 380,
        "wires": [
            [
                "83fd65cf177c2ddf"
            ]
        ]
    },
    {
        "id": "83fd65cf177c2ddf",
        "type": "mqtt out",
        "z": "932355c6886aaf27",
        "name": "",
        "topic": "v1/Suzhou_Plant/Assembly-Area-1/Line-01/Station-
Screwing/Metric/torque",
        "qos": "",
        "retain": "",
        "respTopic": "",
        "contentType": "",
        "userProps": "",
        "correl": "",
        "expiry": "",
        "broker": "broker-djm245hxt9d9",
        "x": 780,
        "y": 440,
        "wires": []
    },
    {
        "id": "broker-djm245hxt9d9",
        "type": "mqtt-broker",
        "name": "emqx:1883",
        "broker": "emqx",
        "port": "1883",
        "clientid": "355039328257392",
        "usetls": false,
        "protocolVersion": "4",
        "keepalive": "60",
        "cleansession": true,
```

```
        "birthTopic": "",
        "birthQos": "0",
        "birthPayload": "",
        "closeTopic": "",
        "closeQos": "0",
        "closePayload": "",
        "willTopic": "",
        "willQos": "0",
        "willPayload": ""
    }
]
```

🚀 実用上のヒント

- **UNS** に publish する前に、必要に応じて **function** ノードでデータを整形できます。
- **debug** ノードを使って、その場でデータを確認できます。
- ノードの詳細は Node-RED flows を参照してください。

ベストプラクティス

セキュリティとレジリエンス

Tier0 の ID、データアクセス、デプロイの可用性、復旧に関する制御の概要。

Tier0 は、次の 2 つの側面からデータの安全性を確保します。

- セキュリティ：プラットフォーム、アプリケーション、データにアクセスできるユーザーを制御します。
- レジリエンス：障害発生後のサービス継続とデータ復旧に対応します。

エディションごとのセキュリティ範囲

エディション	セキュリティの範囲
Cloud	プラットフォームへのアクセス制御と、認証が必要な MQTT および API へのアクセスを提供します。
Enterprise	プラットフォームとデータへのアクセス制御に加え、プライベート環境向けのデプロイの可用性、バックアップ、復旧を提供します。
Edge（オープンソース）	ローカルでのデータ接続とモデリングに使用します。内蔵 MQTT ブローカーは認証なしのサブスクリプションを許可します。

各エディションの位置付けについては、最適な製品の選択を参照してください。

> ⓘ **Tier0 Cloud** のセキュリティに関する説明
> Cloud の本番サービスにおけるネットワーク境界、テナント分離、データ保護、セキュリティ運用については、**Tier0 Cloud** のネットワークとデータのセキュリティに関する説明を参照してください。この説明は Tier0 Cloud のみに適用されます。

セキュリティ

ID とアクセス制御

対象	Tier0 の仕組み	詳細
プラットフォームへのアクセス	• Cloud は Workspace のメンバー資格でアクセスを制御します。 • Enterprise はユーザーアカウントでアクセスを制御します。	ユーザー管理
Agent のアクセス	Agent は API キーで認証し、Tier0 CLI を使用してプラットフォームにアクセスして操作します。	CLI 認証
プロジェクトとアプリケーションへのアクセス	Tier0 は RBAC を使用してアプリケーションへのアクセスを制御します。	アクセス制御

ⓘ ノート
Workspace のメンバー資格によってアカウントが自動作成されることはありません。ユーザー自身がアカウントを登録してサインインする必要があります。アカウント登録には 2 段階の確認が必要です。

データアクセス制御

アクセス方法	Tier0 の仕組み	詳細
MQTT	• クライアントは Tier0 が生成した認証情報で認証します。 • TLS とセキュア WebSocket が転送中のデータを暗号化します。	MQTT 認証情報と接続

アクセス方法	Tier0 の仕組み	詳細
REST API	データへのアクセスには Tier0 が生成したキーが必要です。	API リファレンス

レジリエンス

デプロイの可用性

Enterprise は現在、センターノード向けに 2 つのデプロイ計画を提供しています。

計画	可用性の境界	詳細
標準デプロイ	• データベースのプライマリ／スタンバイのフェイルオーバーに対応します。 • Enterprise アプリケーションと EMQX は単一障害点として残ります。	標準デプロイ計画
高可用デプロイ	• Enterprise のアクティブ／スタンバイノードを追加し、データベース、キャッシュ、オブジェクトストレージ、管理対象のランタイムファイルのフェイルオーバー動作を定義します。 • 無停止やデータ損失ゼロを保証するものではありません。	高可用デプロイ計画

ⓘ ノート

可用性の制限の詳細は、**SLA と高可用性の境界**を参照してください。

バックアップと復旧

Enterprise はデータベースのバックアップと復旧を提供します。復旧後はデプロイ状態を確認し、基本機能を検証して、プラットフォームにサインインできることを確認してください。

- バックアップについては、バックアップと復元を参照してください。
- 復旧検証の要件については、**SLA と高可用性の境界**の訓練要件を参照してください。

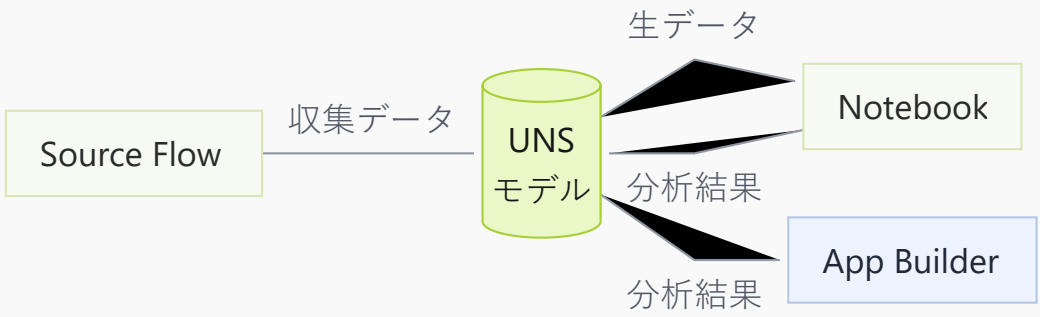
ⓘ 応答時間
通常の応答時間は 3 営業日です。

ベストプラクティス

分析アプリの構築

Aramco Bowtie ケース（匿名化データ）を使い、Notebook で分析アプリを構築します。

Tier0 は Python による高度なデータ分析に Marimo Notebook を使用します。このガイドでは Bowtie アプリケーションを例として使用します。



サンプル背景

製油所の腐食は、複数の運転要因に影響される複雑なプロセスです。この例では、プロセスデータを組み合わせて腐食発生確率を推定し、予防保全を支援するリアルタイム腐食リスク評価を示します。

生データを取得する

1 Tier0 で **UNS** に移動し、データモデルをインポートします。

UNS モデル JSON を表示

```
{
  "notes":
"type:PATH|TOPIC,topicType:STATE|ACTION|METRIC,schema.type:INTEGER|STRING
|FLOAT|DOUBLE|BOOLEAN|LONG|DATETIME",
  "namespace": [
    {
```

```
      "name": "Refinery",
      "type": "PATH",
      "children": [
        {
          "name": "CDU_Plant",
          "type": "PATH",
          "children": [
            {
              "name": "State",
              "type": "PATH",
              "topicType": "STATE",
              "children": [
                {
                  "name": "Corrosion_Risk",
                  "type": "TOPIC",
                  "topicType": "STATE",
                  "description": "Current corrosion risk state inferred
by Bayesian Network",
                  "enableHistory": "TRUE",
                  "schema": [
                    {
                      "name": "risk_state",
                      "type": "STRING"
                    },
                    {
                      "name": "previous_state",
                      "type": "STRING"
                    },
                    {
                      "name": "model_confidence",
                      "type": "FLOAT"
                    },
                    {
                      "name": "timestamp",
                      "type": "DATETIME"
                    }
                  ]
                }
              ]
            },
            {
              "name": "Metric",
              "type": "PATH",
              "topicType": "METRIC",
              "children": [
                {
                  "name": "Corrosion_Monitoring",
                  "type": "TOPIC",
                  "topicType": "METRIC",
                  "description": "Real-time process measurements for CDU
atmospheric overhead corrosion monitoring",
```

```
  "enableHistory": "TRUE",
  "mockData": "FALSE",
  "schema": [
    {
      "name": "d103_chloride",
      "type": "FLOAT"
    },
    {
      "name": "d103_ph",
      "type": "FLOAT"
    },
    {
      "name": "wash_water_flow",
      "type": "FLOAT"
    },
    {
      "name": "desalter_salt_ptb",
      "type": "FLOAT"
    },
    {
      "name": "desalter_bsw",
      "type": "FLOAT"
    },
    {
      "name": "wash_water_rate",
      "type": "FLOAT"
    },
    {
      "name": "rrd_ph",
      "type": "FLOAT"
    },
    {
      "name": "rrd_chloride",
      "type": "FLOAT"
    },
    {
      "name": "rrd_total_iron",
      "type": "FLOAT"
    },
    {
      "name": "sap_evidence",
      "type": "STRING"
    },
    {
      "name": "injection_quill_fail",
      "type": "BOOLEAN"
    },
    {
      "name": "timestamp",
      "type": "DATETIME"
    }
```

```
                ]
              },
              {
                "name": "Corrosion_Analysis",
                "type": "TOPIC",
                "topicType": "METRIC",
                "description": "Posterior probabilities generated by
the Bayesian Network",
                "enableHistory": "TRUE",
                "mockData": "FALSE",
                "schema": [
                  {
                    "name": "p_normal",
                    "type": "DOUBLE"
                  },
                  {
                    "name": "p_developing",
                    "type": "DOUBLE"
                  },
                  {
                    "name": "p_confirmed",
                    "type": "DOUBLE"
                  },
                  {
                    "name": "total_corrosion_risk",
                    "type": "DOUBLE"
                  },
                  {
                    "name": "lopc_probability",
                    "type": "DOUBLE"
                  },
                  {
                    "name": "shutdown_probability",
                    "type": "DOUBLE"
                  },
                  {
                    "name": "escalation_probability",
                    "type": "DOUBLE"
                  },
                  {
                    "name": "timestamp",
                    "type": "DATETIME"
                  }
                ]
              }
            ]
          }
        ]
      }
    }
```

```
  ]
}
```

2 **Flows** に移動し、**Source Flow** を作成して生データを接続し、**UNS** に publish します。

Source Flow JSON を表示

```
[
    {
        "id": "6173469ed5064ccf",
        "type": "tab",
        "label": "corrosion",
        "disabled": false,
        "info": ""
    },
    {
        "id": "ad20b016baf7247f",
        "type": "OpcUa-Client",
        "z": "6173469ed5064ccf",
        "endpoint": "66bd118c33eb12f8",
        "action": "subscribe",
        "deadbandtype": "a",
        "deadbandvalue": 1,
        "time": 10,
        "timeUnit": "s",
        "certificate": "n",
        "localfile": "",
        "localkeyfile": "",
        "securitymode": "None",
        "securitypolicy": "None",
        "useTransport": false,
        "maxChunkCount": 1,
        "maxMessageSize": 8192,
        "receiveBufferSize": 8192,
        "sendBufferSize": 8192,
        "setstatusandtime": false,
        "keepsessionalive": false,
        "name": "",
        "x": 280,
        "y": 440,
        "wires": [
            [
                "ac9195bb4863dbdd",
                "5a3d61b2ae40dcb6"
            ],
            [],
            []
```

```
            ]
        },
        {
            "id": "b6746ed066ba0e1a",
            "type": "inject",
            "z": "6173469ed5064ccf",
            "name": "",
            "props": [
                {
                    "p": "payload"
                },
                {
                    "p": "topic",
                    "vt": "str"
                }
            ],
            "repeat": "",
            "crontab": "",
            "once": false,
            "onceDelay": 0.1,
            "topic": "multiple",
            "payload": "[   {     \"name\": \"D103_CHLORIDE\",
\"nodeId\": \"ns=2;i=3\",     \"datatype\": \"Float\"   },   {
\"name\": \"D103_PH\",     \"nodeId\": \"ns=2;i=4\",     \"datatype\":
\"Float\"   },   {     \"name\": \"WASH_WATER_FLOW\",     \"nodeId\":
\"ns=2;i=5\",     \"datatype\": \"Float\"   },   {     \"name\":
\"DESALTER_SALT_PTB\",     \"nodeId\": \"ns=2;i=6\",     \"datatype\":
\"Float\"   },   {     \"name\": \"DESALTER_BSW\",     \"nodeId\":
\"ns=2;i=7\",     \"datatype\": \"Float\"   },   {     \"name\":
\"WASH_WATER_RATE\",     \"nodeId\": \"ns=2;i=8\",     \"datatype\":
\"Float\"   },   {     \"name\": \"RRD_PH\",     \"nodeId\":
\"ns=2;i=9\",     \"datatype\": \"Float\"   },   {     \"name\":
\"RRD_CHLORIDE\",     \"nodeId\": \"ns=2;i=10\",     \"datatype\":
\"Float\"   },   {     \"name\": \"RRD_TOTAL_IRON\",     \"nodeId\":
\"ns=2;i=11\",     \"datatype\": \"Float\"   },   {     \"name\":
\"SAP_EVIDENCE\",     \"nodeId\": \"ns=2;i=12\",     \"datatype\":
\"UInt16\"   },   {     \"name\": \"INJECTION_QUILL_FAIL\",
\"nodeId\": \"ns=2;i=13\",     \"datatype\": \"Boolean\"   } ]",
            "payloadType": "json",
            "x": 90,
            "y": 440,
            "wires": [
                [
                    "ad20b016baf7247f"
                ]
            ]
        },
        {
            "id": "ac9195bb4863dbdd",
            "type": "debug",
            "z": "6173469ed5064ccf",
```

```
            "name": "debug 2",
            "active": true,
            "tosidebar": true,
            "console": false,
            "tostatus": false,
            "complete": "false",
            "statusVal": "",
            "statusType": "auto",
            "x": 490,
            "y": 480,
            "wires": []
        },
        {
            "id": "5a3d61b2ae40dcb6",
            "type": "function",
            "z": "6173469ed5064ccf",
            "name": "function 1",
            "func": "const TOPIC =
\"Refinery/CDU_Plant/Metric/Corrosion_Monitoring\";\n\nconst nodeMap =
{\n    \"ns=2;i=3\": { field: \"d103_chloride\", type: \"float\" },\n
\"ns=2;i=4\": { field: \"d103_ph\", type: \"float\" },\n    \"ns=2;i=5\":
{ field: \"wash_water_flow\", type: \"float\" },\n    \"ns=2;i=6\": {
field: \"desalter_salt_ptb\", type: \"float\" },\n    \"ns=2;i=7\": {
field: \"desalter_bsw\", type: \"float\" },\n    \"ns=2;i=8\": { field:
\"wash_water_rate\", type: \"float\" },\n    \"ns=2;i=9\": { field:
\"rrd_ph\", type: \"float\" },\n    \"ns=2;i=10\": { field:
\"rrd_chloride\", type: \"float\" },\n    \"ns=2;i=11\": { field:
\"rrd_total_iron\", type: \"float\" },\n    \"ns=2;i=12\": { field:
\"sap_evidence\", type: \"sap\" },\n    \"ns=2;i=13\": { field:
\"injection_quill_fail\", type: \"boolean\" }\n};\n\nconst sapMap = {\n
0: \"none\",\n    1: \"work_order\",\n    2:
\"inspection_finding\"\n};\n\nfunction normalizeNodeId(v) {\n    if (!v)
return \"\";\n    if (typeof v === \"string\") return v;\n    if
(v.toString) return v.toString();\n    return \"\";\n}\n\nfunction
extractValue(payload) {\n    // OPC UA DataValue: payload.value.value\n
if (payload && payload.value && payload.value.value !== undefined) {\n
return payload.value.value;\n    }\n\n    // Some OPC UA nodes output
Variant directly.\n    if (payload && payload.value !== undefined) {\n
return payload.value;\n    }\n\n    return payload;\n}\n\nfunction
extractNodeId(msg) {\n    return normalizeNodeId(\n        msg.nodeId
||\n        msg.topic ||\n        msg.payload?.nodeId ||\n
msg.payload?.nodeid\n    );\n}\n\nconst nodeId =
extractNodeId(msg);\nconst spec = nodeMap[nodeId];\n\nif (!spec) {\n
node.warn(`Unknown OPC UA nodeId: ${nodeId}`);\n    return
null;\n}\n\nlet value = extractValue(msg.payload);\n\nif (spec.type ===
\"float\") {\n    value = Number(value);\n} else if (spec.type ===
\"boolean\") {\n    value = Boolean(value);\n} else if (spec.type ===
\"sap\") {\n    value = sapMap[Number(value)] || \"unknown\";\n}\n\nlet
latest = context.get(\"latest\") || {};\nlatest[spec.field] =
value;\nlatest.timestamp = new
Date().toISOString();\ncontext.set(\"latest\", latest);\n\n// Publish
```

```
only after all 11 fields have been received at least once.\nconst
requiredFields = Object.values(nodeMap).map(x => x.field);\nconst ready =
requiredFields.every(field => latest[field] !== undefined);\n\nif
(!ready) {\n    node.status({\n        fill: \"yellow\",\n        shape:
\"ring\",\n        text: `waiting ${Object.keys(latest).length - 1}/11`\n
});\n    return null;\n}\n\nmsg.topic = TOPIC;\nmsg.payload = {\n
d103_chloride: latest.d103_chloride,\n    d103_ph: latest.d103_ph,\n
wash_water_flow: latest.wash_water_flow,\n    desalter_salt_ptb:
latest.desalter_salt_ptb,\n    desalter_bsw: latest.desalter_bsw,\n
wash_water_rate: latest.wash_water_rate,\n    rrd_ph: latest.rrd_ph,\n
rrd_chloride: latest.rrd_chloride,\n    rrd_total_iron:
latest.rrd_total_iron,\n    sap_evidence: latest.sap_evidence,\n
injection_quill_fail: latest.injection_quill_fail,\n    timestamp:
latest.timestamp\n};\n\nnode.status({\n    fill: \"green\",\n    shape:
\"dot\",\n    text: \"published corrosion monitoring\"\n});\n\nreturn
msg;",
          "outputs": 1,
          "timeout": 0,
          "noerr": 0,
          "initialize": "",
          "finalize": "",
          "libs": [],
          "x": 370,
          "y": 580,
          "wires": [
              [
                  "aa15be33a84fb812",
                  "4fdd68dda3ee5c9e"
              ]
          ]
      },
      {
          "id": "aa15be33a84fb812",
          "type": "mqtt out",
          "z": "6173469ed5064ccf",
          "name": "",
          "topic": "",
          "qos": "",
          "retain": "",
          "respTopic": "",
          "contentType": "",
          "userProps": "",
          "correl": "",
          "expiry": "",
          "broker": "broker-djx75cwgiywq",
          "x": 550,
          "y": 580,
          "wires": []
      },
      {
          "id": "4fdd68dda3ee5c9e",
```

```
        "type": "debug",
        "z": "6173469ed5064ccf",
        "name": "debug 3",
        "active": true,
        "tosidebar": true,
        "console": false,
        "tostatus": false,
        "complete": "false",
        "statusVal": "",
        "statusType": "auto",
        "x": 450,
        "y": 680,
        "wires": []
    },
    {
        "id": "66bd118c33eb12f8",
        "type": "OpcUa-Endpoint",
        "endpoint": "opc.tcp://172.31.151.237:4850",
        "secpol": "None",
        "secmode": "None",
        "none": true,
        "login": false,
        "usercert": false,
        "usercertificate": "",
        "userprivatekey": ""
    },
    {
        "id": "broker-djx75cwgiywq",
        "type": "mqtt-broker",
        "z": "6173469ed5064ccf",
        "name": "corrosion",
        "broker": "emqx",
        "port": "1883",
        "clientid": "357356352615696",
        "usetls": false,
        "protocolVersion": "4",
        "keepalive": "60",
        "cleansession": true,
        "birthTopic": "",
        "birthQos": "0",
        "birthPayload": "",
        "closeTopic": "",
        "closeQos": "0",
        "closePayload": "",
        "willTopic": "",
        "willQos": "0",
        "willPayload": ""
    },
    {
        "id": "c5201f1ad482b182",
        "type": "global-config",
```

```
        "env": [],
        "modules": {
            "node-red-contrib-opcua": "0.2.339"
        }
    }
]
```

Notebook で分析アプリを構築する

1. Tier0 で **Notebook** に移動し、新しい notebook を作成します。
2. notebook を開き、**UNS** のデータを分析するために次の cell を追加します。
    - セル 1 - 必要な依存関係をインポートし、MQTT broker、topics、実行時設定を構成します。

コードを表示

```
@app.cell
def _():
    """Imports and runtime configuration."""
    import json
    import math
    import os
    import threading
    import time
    from datetime import datetime, timezone

    import marimo as mo
    import pandas as pd
    import paho.mqtt.client as mqtt

    MQTT_BROKER_HOST = os.getenv("TIER0_MQTT_HOST", "enterprise-new-
dev.tier0.dev")
    MQTT_BROKER_PORT = int(os.getenv("TIER0_MQTT_PORT", "1883"))
    MQTT_USERNAME = os.getenv("TIER0_MQTT_USERNAME",
"357890055321856")
    MQTT_PASSWORD = os.getenv("TIER0_MQTT_PASSWORD",
"caa6b34242002a3dff4948dd73ce8a")
    MQTT_CLIENT_ID = os.getenv(
        "TIER0_MQTT_CLIENT_ID",
        "357890055321856_bn",
    )

    SOURCE_TOPIC = "Refinery/CDU_Plant/Metric/Corrosion_Monitoring"
```

```
    ANALYSIS_TOPIC = "Refinery/CDU_Plant/Metric/Corrosion_Analysis"
    RISK_TOPIC = "Refinery/CDU_Plant/State/Corrosion_Risk"

    SOURCE_QOS = int(os.getenv("TIER0_SOURCE_QOS", "1"))
    OUTPUT_QOS = int(os.getenv("TIER0_OUTPUT_QOS", "1"))
    AUTOSTART = os.getenv("TIER0_BN_AUTOSTART", "true").lower() in {
        "1", "true", "yes", "on"
    }
    ROOT_CONFIDENCE = float(os.getenv("TIER0_BN_ROOT_CONFIDENCE",
"0.85"))

    def utc_now_iso() -> str:
        return (
            datetime.now(timezone.utc)
            .isoformat(timespec="milliseconds")
            .replace("+00:00", "Z")
        )

    mo.md(
        """
        # CDU Corrosion Bayesian Analysis

        This notebook subscribes to a complete corrosion-monitoring
snapshot,
        performs one Bayesian inference for each new source message,
and publishes
        the analysis results back to the UNS.
        """
    )
    return (
        ANALYSIS_TOPIC,
        AUTOSTART,
        MQTT_BROKER_HOST,
        MQTT_BROKER_PORT,
        MQTT_CLIENT_ID,
        MQTT_PASSWORD,
        MQTT_USERNAME,
        OUTPUT_QOS,
        RISK_TOPIC,
        ROOT_CONFIDENCE,
        SOURCE_QOS,
        SOURCE_TOPIC,
        datetime,
        json,
        math,
        mo,
        mqtt,
        os,
        pd,
        threading,
        time,
```

```
        timezone,
        utc_now_iso,
    )
```

- セル 2 - Bayesian Network の topology、状態空間、CPT generator、確率ルールを定義します。

コードを表示

```
@app.cell
def _(math, pd):
    """Static Bayesian-network definition and CPT generators."""
    import itertools

    from pgmpy.factors.discrete import TabularCPD
    from pgmpy.inference import VariableElimination
    from pgmpy.models import DiscreteBayesianNetwork

    STATE_SPACES = {
        "STATE_3": ["LOW", "NORMAL", "HIGH"],
        "STATE_YN": ["NO", "YES"],
        "PB2_3STATE": ["FAIL", "IL_OK", "SL_OK"],
        "TOP_EVENT_STATE": ["NORMAL", "DEVELOPING", "CONFIRMED"],
    }

    NODE_STATE_SPACE = {
        "T1_HighChlorideLoad": "STATE_3",
        "T2_ChemProtectionFail": "STATE_3",
        "T3_WashWaterLow": "STATE_3",
        "PB1_Desalter_OK": "STATE_YN",
        "PB2_WashQty_State": "PB2_3STATE",
        "PB3_Neutralization_OK": "STATE_YN",
        "MB1_ESD_OK": "STATE_YN",
        "MB2_TSV_OK": "STATE_YN",
        "TE_CorrosionState": "TOP_EVENT_STATE",
        "EV_TOTIRON_High": "STATE_YN",
        "EV_SAP_Positive": "STATE_YN",
        "EV_QuillFailure": "STATE_YN",
        "C1_LOPC": "STATE_YN",
        "C2_UnplannedShutdown": "STATE_YN",
        "C3_Escalation": "STATE_YN",
    }

    EDGES = [
        ("T1_HighChlorideLoad", "TE_CorrosionState"),
        ("T2_ChemProtectionFail", "TE_CorrosionState"),
        ("T3_WashWaterLow", "TE_CorrosionState"),
```

```
        ("PB1_Desalter_OK", "TE_CorrosionState"),
        ("PB2_WashQty_State", "TE_CorrosionState"),
        ("PB3_Neutralization_OK", "TE_CorrosionState"),
        ("TE_CorrosionState", "EV_TOTIRON_High"),
        ("TE_CorrosionState", "EV_SAP_Positive"),
        ("TE_CorrosionState", "EV_QuillFailure"),
        ("TE_CorrosionState", "C1_LOPC"),
        ("MB1_ESD_OK", "C1_LOPC"),
        ("MB2_TSV_OK", "C1_LOPC"),
        ("TE_CorrosionState", "C2_UnplannedShutdown"),
        ("MB1_ESD_OK", "C2_UnplannedShutdown"),
        ("C1_LOPC", "C3_Escalation"),
        ("MB1_ESD_OK", "C3_Escalation"),
        ("MB2_TSV_OK", "C3_Escalation"),
    ]

    def states_for(node_id: str) -> list[str]:
        return STATE_SPACES[NODE_STATE_SPACE[node_id]]

    def softmax(values: list[float]) -> list[float]:
        maximum = max(values)
        exps = [math.exp(value - maximum) for value in values]
        total = sum(exps)
        return [value / total for value in exps]

    THREAT_SCORE = {"LOW": 0.0, "NORMAL": 1.0, "HIGH": 2.0}
    PB1_OK_BONUS = {"NO": 0.8, "YES": -0.6}
    PB3_OK_BONUS = {"NO": 1.0, "YES": -0.7}
    PB2_STATE_BONUS = {"FAIL": 1.2, "IL_OK": 0.3, "SL_OK": -0.6}

    def top_event_distribution(t1, t2, t3, pb1, pb2, pb3) ->
dict[str, float]:
        score = (
            THREAT_SCORE[t1]
            + THREAT_SCORE[t2]
            + THREAT_SCORE[t3]
            + PB1_OK_BONUS[pb1]
            + PB2_STATE_BONUS[pb2]
            + PB3_OK_BONUS[pb3]
        )
        probabilities = softmax(
            [
                2.2 - score,
                -0.2 + 0.6 * score,
                -1.2 + 0.8 * score,
            ]
        )
        return dict(zip(STATE_SPACES["TOP_EVENT_STATE"],
probabilities))

    TE_PARENTS = [
```

```
        "T1_HighChlorideLoad",
        "T2_ChemProtectionFail",
        "T3_WashWaterLow",
        "PB1_Desalter_OK",
        "PB2_WashQty_State",
        "PB3_Neutralization_OK",
    ]

    TE_PARENT_SPACES = [states_for(parent) for parent in TE_PARENTS]
    te_rows = []
    for combination in itertools.product(*TE_PARENT_SPACES):
        distribution = top_event_distribution(*combination)
        te_rows.append(
            [
                *combination,
                distribution["NORMAL"],
                distribution["DEVELOPING"],
                distribution["CONFIRMED"],
            ]
        )
    TE_CPT_DF = pd.DataFrame(
        te_rows,
        columns=TE_PARENTS
        + ["P_NORMAL", "P_DEVELOPING", "P_CONFIRMED"],
    )

    def evidence_yes_probability(te_state: str, kind: str) -> float:
        mappings = {
            "TOTIRON": {"NORMAL": 0.15, "DEVELOPING": 0.65,
"CONFIRMED": 0.85},
            "SAP": {"NORMAL": 0.08, "DEVELOPING": 0.35, "CONFIRMED":
0.75},
            "QUILL": {"NORMAL": 0.03, "DEVELOPING": 0.15,
"CONFIRMED": 0.55},
        }
        return mappings[kind][te_state]

    def lopc_yes_probability(te: str, mb1: str, mb2: str) -> float:
        base = {"NORMAL": 0.02, "DEVELOPING": 0.08, "CONFIRMED":
0.25}[te]
        return min(0.98, base + (0.10 if mb1 == "NO" else 0) + (0.18
if mb2 == "NO" else 0))

    def shutdown_yes_probability(te: str, mb1: str) -> float:
        base = {"NORMAL": 0.05, "DEVELOPING": 0.35, "CONFIRMED":
0.60}[te]
        return min(0.98, base + (0.10 if mb1 == "NO" else 0))

    def escalation_yes_probability(lopc: str, mb1: str, mb2: str) ->
float:
        if lopc == "NO":
```

```
            return 0.01
        return min(0.98, 0.10 + (0.18 if mb1 == "NO" else 0) + (0.35
if mb2 == "NO" else 0))

    def peaked_prior(node_id: str, center_state: str, confidence:
float) -> list[float]:
        states = states_for(node_id)
        confidence = min(max(float(confidence), 0.0), 1.0)
        if center_state not in states:
            return [1.0 / len(states)] * len(states)
        if len(states) == 1:
            return [1.0]
        remainder = (1.0 - confidence) / (len(states) - 1)
        probabilities = [remainder] * len(states)
        probabilities[states.index(center_state)] = confidence
        return probabilities

    def binary_cpd(child: str, parent: str, yes_probability_fn):
        parent_states = states_for(parent)
        yes_values = [float(yes_probability_fn(state)) for state in
parent_states]
        return TabularCPD(
            variable=child,
            variable_card=2,
            values=[[1.0 - value for value in yes_values],
yes_values],
            evidence=[parent],
            evidence_card=[len(parent_states)],
            state_names={child: states_for(child), parent:
parent_states},
        )

    def binary_multi_cpd(child: str, parents: list[str],
yes_probability_fn):
        parent_spaces = [states_for(parent) for parent in parents]
        combinations = list(itertools.product(*parent_spaces))
        yes_values = [float(yes_probability_fn(*combination)) for
combination in combinations]
        return TabularCPD(
            variable=child,
            variable_card=2,
            values=[[1.0 - value for value in yes_values],
yes_values],
            evidence=parents,
            evidence_card=[len(space) for space in parent_spaces],
            state_names={
                child: states_for(child),
                **{parent: states_for(parent) for parent in
parents},
            },
        )
```

```
    return (
        DiscreteBayesianNetwork,
        EDGES,
        NODE_STATE_SPACE,
        STATE_SPACES,
        TE_CPT_DF,
        TE_PARENTS,
        TabularCPD,
        VariableElimination,
        binary_cpd,
        binary_multi_cpd,
        escalation_yes_probability,
        evidence_yes_probability,
        itertools,
        lopc_yes_probability,
        peaked_prior,
        shutdown_yes_probability,
        states_for,
    )
```

- セル 3 - 受信した MQTT payload を検証し、連続的な process value を Bayesian Network の状態に変換します。

コードを表示

```
@app.cell
def _(ROOT_CONFIDENCE, utc_now_iso):
    """Source validation, discretization, and one-snapshot
analysis."""
    import hashlib

    REQUIRED_SOURCE_FIELDS = [
        "d103_chloride",
        "d103_ph",
        "wash_water_flow",
        "desalter_salt_ptb",
        "desalter_bsw",
        "wash_water_rate",
        "rrd_ph",
        "rrd_chloride",
        "rrd_total_iron",
        "sap_evidence",
        "injection_quill_fail",
        "timestamp",
    ]
```

```
    def parse_boolean(value) -> bool:
        if isinstance(value, bool):
            return value
        if isinstance(value, (int, float)):
            return bool(value)
        if isinstance(value, str):
            normalized = value.strip().lower()
            if normalized in {"true", "1", "yes", "y", "on"}:
                return True
            if normalized in {"false", "0", "no", "n", "off"}:
                return False
        raise ValueError(f"invalid boolean value: {value!r}")

    def normalize_sap(value) -> str:
        normalized = str(value).strip().lower()
        if normalized in {"0", "none", "normal", "", "nan"}:
            return "None"
        if normalized in {"1", "work_order", "work order", "minor",
"warning"}:
            return "Minor corrosion observed"
        if normalized in {
            "2",
            "inspection_finding",
            "inspection finding",
            "confirmed",
            "abnormal",
        }:
            return "Confirmed corrosion / thickness loss"
        raise ValueError(f"unsupported sap_evidence value:
{value!r}")

    def normalize_source_payload(payload: dict) -> dict:
        if not isinstance(payload, dict):
            raise TypeError("source MQTT payload must be a JSON
object")

        lower = {str(key).lower(): value for key, value in
payload.items()}
        missing = [field for field in REQUIRED_SOURCE_FIELDS if
field not in lower]
        if missing:
            raise ValueError(f"source payload is missing fields:
{missing}")

        normalized = {
            "d103_chloride": float(lower["d103_chloride"]),
            "d103_ph": float(lower["d103_ph"]),
            "wash_water_flow": float(lower["wash_water_flow"]),
            "desalter_salt_ptb": float(lower["desalter_salt_ptb"]),
            "desalter_bsw": float(lower["desalter_bsw"]),
            "wash_water_rate": float(lower["wash_water_rate"]),
```

```
            "rrd_ph": float(lower["rrd_ph"]),
            "rrd_chloride": float(lower["rrd_chloride"]),
            "rrd_total_iron": float(lower["rrd_total_iron"]),
            "sap_evidence": normalize_sap(lower["sap_evidence"]),
            "injection_quill_fail":
parse_boolean(lower["injection_quill_fail"]),
            "timestamp": str(lower["timestamp"]).strip(),
        }

        numeric_ranges = {
            "d103_chloride": (0.0, 1000.0),
            "d103_ph": (0.0, 14.0),
            "wash_water_flow": (0.0, 10000.0),
            "desalter_salt_ptb": (0.0, 1000.0),
            "desalter_bsw": (0.0, 100.0),
            "wash_water_rate": (0.0, 100.0),
            "rrd_ph": (0.0, 14.0),
            "rrd_chloride": (0.0, 10000.0),
            "rrd_total_iron": (0.0, 10000.0),
        }
        for field, (minimum, maximum) in numeric_ranges.items():
            value = normalized[field]
            if not minimum <= value <= maximum:
                raise ValueError(f"{field} out of accepted range:
{value}")

        if not normalized["timestamp"]:
            raise ValueError("timestamp cannot be empty")
        return normalized

    def source_message_id(payload: dict) -> str:
        # Timestamp is the primary event key. Hash protects against
two different
        # snapshots accidentally sharing the same timestamp.
        ordered = "|".join(str(payload[field]) for field in
REQUIRED_SOURCE_FIELDS)
        return hashlib.sha256(ordered.encode("utf-8")).hexdigest()

    def discretize_source(payload: dict) -> tuple[dict, dict]:
        t1 = (
            "LOW"
            if payload["d103_chloride"] < 5.0
            else "NORMAL"
            if payload["d103_chloride"] <= 20.0
            else "HIGH"
        )

        chemical_failure_score = int(payload["d103_ph"] < 5.5) +
int(
            payload["d103_chloride"] > 10.0
        )
```

```
        t2 = (
            "HIGH"
            if chemical_failure_score == 2
            else "NORMAL"
            if chemical_failure_score == 1
            else "LOW"
        )

        t3 = (
            "HIGH"
            if payload["wash_water_flow"] < 6.0
            else "NORMAL"
            if payload["wash_water_flow"] <= 10.0
            else "LOW"
        )

        pb1 = (
            "YES"
            if payload["desalter_salt_ptb"] <= 1.0
            and payload["desalter_bsw"] <= 0.2
            else "NO"
        )

        pb2 = (
            "FAIL"
            if payload["wash_water_rate"] < 4.0
            else "IL_OK"
            if payload["wash_water_rate"] < 6.0
            else "SL_OK"
        )

        pb3 = (
            "YES"
            if payload["rrd_ph"] >= 5.5 and payload["rrd_chloride"]
<= 10.0
            else "NO"
        )

        root_centers = {
            "T1_HighChlorideLoad": t1,
            "T2_ChemProtectionFail": t2,
            "T3_WashWaterLow": t3,
            "PB1_Desalter_OK": pb1,
            "PB2_WashQty_State": pb2,
            "PB3_Neutralization_OK": pb3,
            # The source model currently has no MB fields. Keep both
mitigations
            # available by default, matching the previous notebook
UI defaults.
            "MB1_ESD_OK": "YES",
            "MB2_TSV_OK": "YES",
```

```
        }

        evidence = {
            "EV_TOTIRON_High": "YES" if payload["rrd_total_iron"] >=
10.0 else "NO",
            "EV_SAP_Positive": "YES" if payload["sap_evidence"] !=
"None" else "NO",
            "EV_QuillFailure": "YES" if
payload["injection_quill_fail"] else "NO",
        }
        return root_centers, evidence

    return (
        REQUIRED_SOURCE_FIELDS,
        discretize_source,
        normalize_source_payload,
        parse_boolean,
        source_message_id,
    )
```

- セル 4 - 現在の snapshot に対して Bayesian Network を構築し、推論を実行して output payload を生成します。

コードを表示

```
@app.cell
def _(
    DiscreteBayesianNetwork,
    EDGES,
    ROOT_CONFIDENCE,
    STATE_SPACES,
    TE_CPT_DF,
    TE_PARENTS,
    TabularCPD,
    VariableElimination,
    binary_cpd,
    binary_multi_cpd,
    discretize_source,
    escalation_yes_probability,
    evidence_yes_probability,
    itertools,
    lopc_yes_probability,
    peaked_prior,
    shutdown_yes_probability,
    states_for,
    utc_now_iso,
):
```

```
    """Build a fresh BN for the current snapshot and execute
inference."""

    def factor_probability(factor, variable: str, state: str) ->
float:
        states = factor.state_names[variable]
        values = list(map(float, factor.values))
        return float(values[states.index(state)])

    def build_model(root_centers: dict):
        model = DiscreteBayesianNetwork(EDGES)

        root_cpds = []
        for node_id, center_state in root_centers.items():
            states = states_for(node_id)
            probabilities = peaked_prior(node_id, center_state,
ROOT_CONFIDENCE)
            root_cpds.append(
                TabularCPD(
                    variable=node_id,
                    variable_card=len(states),
                    values=[[probability] for probability in
probabilities],
                    state_names={node_id: states},
                )
            )

        te_values = [
            TE_CPT_DF["P_NORMAL"].astype(float).tolist(),
            TE_CPT_DF["P_DEVELOPING"].astype(float).tolist(),
            TE_CPT_DF["P_CONFIRMED"].astype(float).tolist(),
        ]
        te_cpd = TabularCPD(
            variable="TE_CorrosionState",
            variable_card=3,
            values=te_values,
            evidence=TE_PARENTS,
            evidence_card=[len(states_for(parent)) for parent in
TE_PARENTS],
            state_names={
                "TE_CorrosionState":
STATE_SPACES["TOP_EVENT_STATE"],
                **{parent: states_for(parent) for parent in
TE_PARENTS},
            },
        )

        evidence_cpds = [
            binary_cpd(
                "EV_TOTIRON_High",
                "TE_CorrosionState",
```

```
                lambda state: evidence_yes_probability(state,
"TOTIRON"),
            ),
            binary_cpd(
                "EV_SAP_Positive",
                "TE_CorrosionState",
                lambda state: evidence_yes_probability(state,
"SAP"),
            ),
            binary_cpd(
                "EV_QuillFailure",
                "TE_CorrosionState",
                lambda state: evidence_yes_probability(state,
"QUILL"),
            ),
        ]

        consequence_cpds = [
            binary_multi_cpd(
                "C1_LOPC",
                ["TE_CorrosionState", "MB1_ESD_OK", "MB2_TSV_OK"],
                lopc_yes_probability,
            ),
            binary_multi_cpd(
                "C2_UnplannedShutdown",
                ["TE_CorrosionState", "MB1_ESD_OK"],
                shutdown_yes_probability,
            ),
            binary_multi_cpd(
                "C3_Escalation",
                ["C1_LOPC", "MB1_ESD_OK", "MB2_TSV_OK"],
                escalation_yes_probability,
            ),
        ]

        model.add_cpds(*root_cpds, te_cpd, *evidence_cpds,
*consequence_cpds)
        if not model.check_model():
            raise RuntimeError("Bayesian network model validation
failed")
        return model

    def analyze_snapshot(payload: dict, previous_risk_state: str |
None = None) -> dict:
        root_centers, evidence = discretize_source(payload)
        model = build_model(root_centers)
        inference = VariableElimination(model)

        q_te = inference.query(["TE_CorrosionState"],
evidence=evidence, show_progress=False)
```

```
        q_c1 = inference.query(["C1_LOPC"], evidence=evidence,
show_progress=False)
        q_c2 = inference.query(["C2_UnplannedShutdown"],
evidence=evidence, show_progress=False)
        q_c3 = inference.query(["C3_Escalation"], evidence=evidence,
show_progress=False)

        p_normal = factor_probability(q_te, "TE_CorrosionState",
"NORMAL")
        p_developing = factor_probability(q_te, "TE_CorrosionState",
"DEVELOPING")
        p_confirmed = factor_probability(q_te, "TE_CorrosionState",
"CONFIRMED")
        probabilities = {
            "NORMAL": p_normal,
            "DEVELOPING": p_developing,
            "CONFIRMED": p_confirmed,
        }
        risk_state = max(probabilities, key=probabilities.get)
        analysis_timestamp = utc_now_iso()

        analysis_payload = {
            "p_normal": p_normal,
            "p_developing": p_developing,
            "p_confirmed": p_confirmed,
            "total_corrosion_risk": p_developing + p_confirmed,
            "lopc_probability": factor_probability(q_c1, "C1_LOPC",
"YES"),
            "shutdown_probability": factor_probability(
                q_c2, "C2_UnplannedShutdown", "YES"
            ),
            "escalation_probability": factor_probability(q_c3,
"C3_Escalation", "YES"),
            "timestamp": analysis_timestamp,
        }

        risk_payload = {
            "risk_state": risk_state,
            "previous_state": previous_risk_state or risk_state,
            "model_confidence": max(probabilities.values()),
            "timestamp": analysis_timestamp,
        }

        return {
            "source_timestamp": payload["timestamp"],
            "root_states": root_centers,
            "evidence": evidence,
            "analysis_payload": analysis_payload,
            "risk_payload": risk_payload,
            "risk_state": risk_state,
        }
```

```
    return analyze_snapshot, build_model, factor_probability
```

- セル 5 - source topic を購読し、受信した各 snapshot を処理して Bayesian 分析を実行し、結果を publish します。

コードを表示

```
@app.cell
def _(
    ANALYSIS_TOPIC,
    AUTOSTART,
    MQTT_BROKER_HOST,
    MQTT_BROKER_PORT,
    MQTT_CLIENT_ID,
    MQTT_PASSWORD,
    MQTT_USERNAME,
    OUTPUT_QOS,
    RISK_TOPIC,
    SOURCE_QOS,
    SOURCE_TOPIC,
    analyze_snapshot,
    json,
    mqtt,
    normalize_source_payload,
    source_message_id,
    threading,
    utc_now_iso,
):
    """Long-running MQTT subscriber and publisher service."""

    class CorrosionBNService:
        def __init__(self):
            self.lock = threading.RLock()
            self.client = None
            self.running = False
            self.connected = False
            self.last_message_id = None
            self.last_source_timestamp = None
            self.last_analysis_timestamp = None
            self.previous_risk_state = None
            self.processed_count = 0
            self.duplicate_count = 0
            self.error_count = 0
            self.last_error = None
            self.last_result = None
            self.connection_rc = None
```

```
        def _create_client(self):
            # Use MQTT 3.1.1 and callback API v1 for broad
compatibility with
            # EMQX and different paho-mqtt releases bundled in
Notebook images.
            kwargs = {
                "client_id": MQTT_CLIENT_ID,
                "clean_session": True,
                "protocol": mqtt.MQTTv311,
                "transport": "tcp",
            }
            try:
                client = mqtt.Client(

callback_api_version=mqtt.CallbackAPIVersion.VERSION1,
                    **kwargs,
                )
            except (AttributeError, TypeError):
                client = mqtt.Client(**kwargs)

            client.username_pw_set(MQTT_USERNAME, MQTT_PASSWORD)
            client.on_connect = self._on_connect
            client.on_disconnect = self._on_disconnect
            client.on_message = self._on_message
            client.reconnect_delay_set(min_delay=1, max_delay=30)
            client.enable_logger()
            return client

        def _on_connect(self, client, userdata, flags, rc,
properties=None):
            code = int(rc)
            with self.lock:
                self.connected = code == 0
                self.connection_rc = code
                self.last_error = None if code == 0 else f"MQTT
connection rejected, rc={code}"
            if code == 0:
                sub_rc, mid = client.subscribe(SOURCE_TOPIC,
qos=SOURCE_QOS)
                if sub_rc != mqtt.MQTT_ERR_SUCCESS:
                    with self.lock:
                        self.last_error = f"MQTT subscribe failed,
rc={sub_rc}"
                    return
                print(f"[BN SERVICE] connected and subscribed to
{SOURCE_TOPIC}")
            else:
                print(f"[BN SERVICE ERROR] connection rejected rc=
{code}")
```

```
        def _on_disconnect(self, client, userdata, rc,
properties=None):
            with self.lock:
                self.connected = False
                self.connection_rc = int(rc) if rc is not None else
None
                if self.running and rc:
                    self.last_error = f"MQTT disconnected
unexpectedly, rc={rc}"
            print(f"[BN SERVICE] disconnected: {rc}")

        def _publish_json(self, topic: str, payload: dict):
            info = self.client.publish(
                topic,
                payload=json.dumps(payload, separators=(",", ":"),
ensure_ascii=False),
                qos=OUTPUT_QOS,
                retain=False,
            )
            info.wait_for_publish(timeout=10)
            if info.rc != mqtt.MQTT_ERR_SUCCESS:
                raise RuntimeError(f"MQTT publish failed for
{topic}: rc={info.rc}")

        def _on_message(self, client, userdata, msg):
            try:
                raw_payload = json.loads(msg.payload.decode("utf-
8"))
                payload = normalize_source_payload(raw_payload)
                message_id = source_message_id(payload)

                with self.lock:
                    if message_id == self.last_message_id:
                        self.duplicate_count += 1
                        return
                    previous_risk_state = self.previous_risk_state

                result = analyze_snapshot(payload,
previous_risk_state)

                # Publish only after both payloads have been
generated successfully.
                self._publish_json(ANALYSIS_TOPIC,
result["analysis_payload"])
                self._publish_json(RISK_TOPIC,
result["risk_payload"])

                with self.lock:
                    self.last_message_id = message_id
                    self.last_source_timestamp =
payload["timestamp"]
```

```
                    self.last_analysis_timestamp =
result["analysis_payload"]["timestamp"]
                    self.previous_risk_state = result["risk_state"]
                    self.processed_count += 1
                    self.last_result = result
                    self.last_error = None

                print(
                    "[BN SERVICE] processed",
                    payload["timestamp"],
                    "->",
                    result["risk_state"],
                )
            except Exception as exc:
                with self.lock:
                    self.error_count += 1
                    self.last_error = f"{type(exc).__name__}: {exc}"
                print("[BN SERVICE ERROR]", self.last_error)

        def start(self):
            with self.lock:
                if self.running:
                    return False
                if not MQTT_PASSWORD:
                    raise RuntimeError(
                        "TIER0_MQTT_PASSWORD is empty. Set it before
starting the service."
                    )
                self.client = self._create_client()
                self.running = True
            try:
                # Synchronous connect surfaces DNS, socket, and
authentication
                # failures immediately instead of leaving
running=True forever.
                rc = self.client.connect(
                    MQTT_BROKER_HOST,
                    MQTT_BROKER_PORT,
                    keepalive=30,
                )
                if rc != mqtt.MQTT_ERR_SUCCESS:
                    raise RuntimeError(f"MQTT connect() failed, rc=
{rc}")
                self.client.loop_start()
                return True
            except Exception as exc:
                with self.lock:
                    self.running = False
                    self.connected = False
                    self.last_error = f"{type(exc).__name__}: {exc}"
                    self.client = None
```

```
                raise

        def stop(self):
            with self.lock:
                if not self.running:
                    return False
                client = self.client
                self.running = False
                self.connected = False
                self.client = None
            try:
                client.disconnect()
            finally:
                client.loop_stop()
            return True

        def restart(self):
            self.stop()
            return self.start()

        def status(self) -> dict:
            with self.lock:
                return {
                    "running": self.running,
                    "connected": self.connected,
                    "connection_rc": self.connection_rc,
                    "client_id": MQTT_CLIENT_ID,
                    "broker": f"{MQTT_BROKER_HOST}:
{MQTT_BROKER_PORT}",
                    "source_topic": SOURCE_TOPIC,
                    "analysis_topic": ANALYSIS_TOPIC,
                    "risk_topic": RISK_TOPIC,
                    "processed_count": self.processed_count,
                    "duplicate_count": self.duplicate_count,
                    "error_count": self.error_count,
                    "last_source_timestamp":
self.last_source_timestamp,
                    "last_analysis_timestamp":
self.last_analysis_timestamp,
                    "previous_risk_state": self.previous_risk_state,
                    "last_error": self.last_error,
                }

    # Reuse a previous service instance when the cell is re-
executed, preventing
    # duplicate subscribers inside the same Python process.
    existing = globals().get("_CORROSION_BN_SERVICE")
    if existing is not None:
        try:
            existing.stop()
        except Exception:
```

```
            pass

    CORROSION_BN_SERVICE = CorrosionBNService()
    globals()["_CORROSION_BN_SERVICE"] = CORROSION_BN_SERVICE

    AUTOSTART_ERROR = None
    if AUTOSTART:
        try:
            CORROSION_BN_SERVICE.start()
        except Exception as exc:
            AUTOSTART_ERROR = f"{type(exc).__name__}: {exc}"

    return AUTOSTART_ERROR, CORROSION_BN_SERVICE, CorrosionBNService
```

- セル 6 - MQTT 接続ステータスとサービス統計を表示します。

コードを表示

```
@app.cell
def _(AUTOSTART_ERROR, CORROSION_BN_SERVICE, mo, pd):
    """Service status and usage instructions."""
    status = CORROSION_BN_SERVICE.status()

    blocks = [
        mo.md("## MQTT analysis service"),
        pd.DataFrame([status]),
    ]
    if status.get("last_error"):
        blocks.append(mo.md(f"### Connection / processing
error\n`{status['last_error']}`"))
    elif status.get("running") and not status.get("connected"):
        blocks.append(mo.md("### Connecting\nThe MQTT client has
started but has not completed the broker connection yet."))

    if AUTOSTART_ERROR:
        blocks.append(
            mo.md(
                f"""
                ### Service not started

                `{AUTOSTART_ERROR}`

                Set the MQTT password in the Notebook environment
and run this cell again:

                ~~~python
                import os
```

```
                # MQTT credentials are already configured in this
notebook.
                ~~~
                """
            )
        )
    else:
        blocks.append(
            mo.md(
                """
                The service starts automatically and processes one
complete source
                snapshot for every new MQTT message. No manual
analysis trigger or SQL
                polling is required.

                Available controls in Python:

                ~~~python
                CORROSION_BN_SERVICE.status()
                CORROSION_BN_SERVICE.stop()
                CORROSION_BN_SERVICE.start()
                CORROSION_BN_SERVICE.restart()
                ~~~
                """
            )
        )

    mo.vstack(blocks)
    return status,
```

- セル 7 - 最新の Bayesian 分析結果を表示します。

コードを表示

```
@app.cell
def _(CORROSION_BN_SERVICE, mo):
    """Display the most recent analysis result when one is
available."""
    latest = CORROSION_BN_SERVICE.last_result
    if latest is None:
        mo.md("### Latest result\nNo source message has been
processed yet.")
    else:
        mo.vstack(
            [
                mo.md("### Latest source snapshot result"),
```

```
                mo.md(f"- Source timestamp:
`{latest['source_timestamp']}`"),
                mo.md(f"- Risk state: `{latest['risk_state']}`"),
                mo.md("#### Root states"),
                latest["root_states"],
                mo.md("#### Evidence"),
                latest["evidence"],
                mo.md("#### Corrosion analysis payload"),
                latest["analysis_payload"],
                mo.md("#### Corrosion risk payload"),
                latest["risk_payload"],
            ]
        )
    return


if __name__ == "__main__":
    app.run()
```

3. すべての cell を実行し、**UNS** に移動して結果を確認します。

Bow-tie App を構築する

1. Tier0 で **Builder** に移動します。
2. ダイアログにアプリケーション要件を入力し、構築を開始します。

> ⚠ 必ず伝えること
> agent には UNS のデータを使うことを伝え、具体的な topic path も指定してください。指定しないと、agent が正しいデータを表示できない場合があります。

Builder prompt を表示

```
    CDU 常圧塔頂腐食向けの、モダンな産業用 Bow-Tie Analysis アプリケーションを構築してください。

    次の UNS topic のみをデータソースとして使用してください。

    - Refinery/CDU_Plant/Metric/Corrosion_Monitoring
    - Refinery/CDU_Plant/Metric/Corrosion_Analysis
    - Refinery/CDU_Plant/State/Corrosion_Risk
```

新しい UNS メッセージが届いたら、アプリケーションは自動的に更新される必要があります。

このアプリケーションは、オペレーターが現在の腐食リスク、関連する脅威、バリア状態、起こり得る結果を理解できるようにしてください。

次を含む単一ページの dashboard を作成してください。

1. メインセクションとして Bow-Tie 図を配置する：
- 左側に脅威を表示
- 脅威とトップイベントの間に予防バリアを表示
- 腐食を中央のトップイベントとして表示
- トップイベントの後に緩和バリアを表示
- 右側に結果を表示

2. 目立つリスクサマリーを表示する：
- 現在のリスク状態
- モデル信頼度
- normal、developing、confirmed の確率
- 総腐食リスク

3. Corrosion_Monitoring から主要値を表示するコンパクトな live process data panel を作成する。chloride、pH、wash water flow、desalter performance、wash water rate、total iron、SAP evidence、injection quill status を含めてください。

4. 結果指標：
- 一次封じ込めの喪失
- 計画外停止
- エスカレーションリスク

5. 視覚ステータスルール：
- 緑は normal または healthy condition
- 琥珀色/黄色は developing risk または warning condition
- 赤は confirmed risk、failed barrier、または severe consequence
- グレーは data unavailable
- 青は interaction と informational highlight のみに使用

6. インタラクション：
- 脅威、バリア、結果を選択したら、現在の状態、関連する process value、短い説明を表示してください。
- 新しい UNS データが到着したら、インターフェースを自動更新してください。
- 最新データの timestamp と、組み込み SDK からの connection/subscription status を表示してください。

デザイン要件：

アプリケーション全体で、既存の visual design language と styling を維持してください。

モダンな industrial operations dashboard スタイルを使用してください。

```
    - pure black ではなく dark blue-gray color palette を使用する。
    - subtle gradient を持つ layered panels を使用する。
    - 16-20px 程度の大きめの角丸を持つ card-based layout を使用する。
    - heavy shadow ではなく thin borders で分離する。
    - soft depth と控えめな glow effects を使用する。
    - 十分な padding、一貫した alignment、きれいな spacing を保つ。
    - body text には IBM Plex Sans を使用する。
    - headings には Space Grotesk を使用する。
    - 明確な visual hierarchy を持つ、コンパクトで情報密度の高い card を使用する。
    - letter spacing を広げた uppercase section labels を使用する。
    - business analytics ではなく、professional refinery / SCADA interface にする。
    - skeuomorphic elements ではなく、flat illustrations と SVG graphics を使用する。
    - smooth で subtle な transitions と hover effects を使用する。

    全体の見た目は clean、modern、technical、operational にしてください。

    避けるもの：
    - pure black backgrounds
    - glassmorphism
    - cyberpunk styling
    - neon lighting
    - overly decorative effects
    - business analytics dashboard styling

    clear hierarchy、compact cards、readable labels、responsive layout を備えた、clean で professional な refinery operations style を使用してください。

    対応する UNS model で定義された payload fields を正確に使用してください。追加フィールド、計算値、mock data を作らないでください。
```

3. Once the application is complete after certain rounds of refining, deploy it.

4. Go to **Launchpad**, open the application and check.

ベストプラクティス

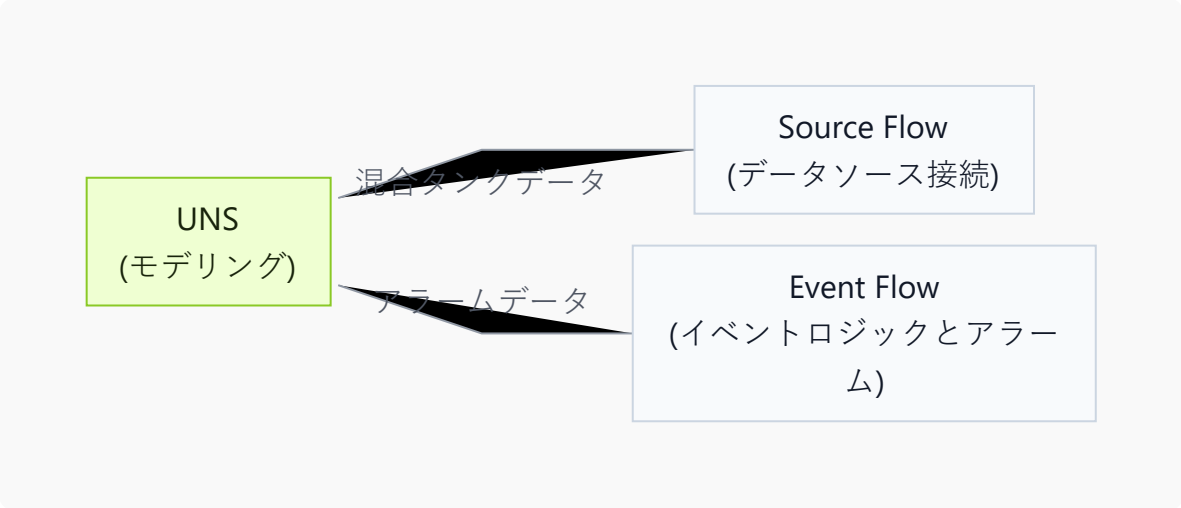
Agent でデータワークフローを構築する

UNS Agent を使い、データモデリング、データ接続、データイベントを含むワークフローを構築します。

データモデル、接続、イベントを複数のモジュールで順番に作る代わりに、UNS Agent が会話を通じてワークフローを処理します。

ワークフローの背景

混合タンクは材料を混ぜながら加熱します。データモデルを構築し、温度、水位、ヒーター状態を収集して、タンクの過熱を判定しアラームを発生させます。



ワークフローの構築方法

1. **UNS** で `full_access` を指定して UNS Agent との会話を開始します。
2. UNS モデルを構築するためのプロンプトを入力します。

```
混合タンクの温度、水位、ヒーター状態を表すデータモデルを作成します。温度と水位は 1 つの metric topic に、ヒーター状態は 1 つの state topic に配置してください。
```

3 モデルが完成したことを確認したら、対応するデータをモデルに接続するプロンプトを入力します。

```
この 2 つの topic にデータを送信する Source Flow を作成し、妥当なデータをシミュレートしてください。
```

> 実際のデータソースがある場合
> データソース情報を Agent に伝えて接続させます。

4 次のロジックでイベントを作成するプロンプトを入力します。

- 低水位警告：水位が 20% 未満でヒーターがオンの場合に発生します。
- 空焚きアラーム：水位が 15% 未満、温度が 90°C を超え、ヒーターがオンの場合に発生します。

```
次のアラームロジックの Event Flow を作成してください：
  - Warning：level < 20 かつ heater_status = true のときに発生。
  - Critical：level < 15、temperature > 90、heater_status = true のときに発生。
  - level > 25 または heater_status = false のとき、アクティブなアラームをクリア。
既存の UNS モデル配下にアラーム結果を受け取る新しい topic を作成してください。出力にはアラームレベル、メッセージ、アクティブ状態、タイムスタンプを含めます。
```

5 **UNS** で結果を確認します。

> ノート
> 各ステップの後も Agent と会話を続けて結果を調整できます。

ベストプラクティス

Digital Twin でデータを表示する

UNS データを 3D モデルとシーンでリアルタイムに表示します。

Tier0 の digital twin 機能により、リアルな 3D シーンとモデル上で UNS データをリアルタイムに表示できます。

工場シーンを可視化する

1. **Model** にモデルをアップロードした後、UNS データでマッピングするノードを選択し、リアルタイム動作に使います。
2. 異なる UNS データを持つインスタンスをモデルに追加し、複数の機械をシミュレートします。
3. **Scene** で、**Instance Library** からシーン内に存在するインスタンスを追加します。
4. 上部ツールを使ってモデルを移動、拡大縮小、回転し、シーンに合わせて視点と座標位置を調整します。
5. **Object** パネルでモデル位置とバインドされた UNS データを確認します。

リファレンス

API リファレンス

Tier0 の API 統合の要点と、機械可読な OpenAPI リファレンスです。

はじめに

- ベース URL には `https://<your-tier0-host>/openapi/v1` を使用します。
- `/assets/files/public` 以外の OpenAPI エンドポイントには `x-api-key` を送信します。`public` としてマークされたファイルは API キーなしでダウンロードできます。
- まず `POST /auth/whoami` を呼び出し、API キーの ID と有効な権限を確認します。

> ⓘ **OpenAPI リファレンス**
> リクエストスキーマ、レスポンススキーマ、および実行可能なリクエストは、インタラクティブな **API リファレンス** で確認できます。

結果の処理

HTTP `200` が返っても、UNS の一括操作は一部失敗することがあります。成功と判断する前に、各結果を確認してください。

Flow のカスタム HTTP In ルート

Source Flow と **Event Flow** 内で開発する API の入力とレスポンスは、デプロイ済み Flow によって定義されます。

- **Source Flow** は `/flow/source/` を使用し、デフォルトで公開されます。機密性の高い動作を公開する前に、入口または Flow レベルのアクセス制御を追加してください。
- **Event Flow** は `/flow/event/` を使用し、`flow.read` 権限を持つ `x-api-key` が必要です。

> ⓘ **Source Flow または Event Flow のカスタム API の例**
> API URL が `/integrations/echo` で、`HTTP In` ノードで API を開発する場合:
>
> - `HTTP In` のルートは `/integrations/echo` です。残りは Tier0 が補完します。

- 外部からの呼び出し URL は `https://tier0.example.com/flow/source/integrations/echo` または `https://tier0.example.com/flow/event/integrations/echo` です。

トラブルシューティング

- `401` / `403`: API キー、スコープ、所有者の権限を確認します。
- `404`: HTTP メソッド、ルートプレフィックス、ノード URL、デプロイ状況を確認します。
- `No response`: すべての Flow パスが `HTTP Response` ノードに到達し、元のリクエストとレスポンスのプロパティを破棄していないことを確認します。

リファレンス

CLI コマンドリファレンス

CLI と Agent Skill のコマンドリファレンス。

コマンド、フラグ、例を検索...

グローバルフラグ これらのフラグは任意の `tier0` コマンドで使用できます。

`--json` 生の JSON を出力 `--debug` HTTP リクエスト/レスポンス詳細を出力

基本操作

`tier0 doctor`

ローカル Tier0 CLI の接続と認証を診断

使用方法

```
tier0 doctor
```

注記

- ローカル設定、OpenAPI 接続、API key の有無、API key の識別情報を確認します。

例

```
tier0 doctor
```

`tier0 config`

設定を表示または更新

使用方法

```
tier0 config
```

フラグ

`--base-url`
プラットフォームの base URL を設定

`--api-key`
API key を設定（tier0 login の代替）

`--lang`
非推奨の互換フラグ。CLI 出力は英語のみです
非推奨

例

```
tier0 config
```

```
tier0 config --base-url https://tier0.dev
```

```
tier0 config --api-key sk-per-xxxxxx
```

`tier0 login`

Device Flow で認証

使用方法

```
tier0 login
```

フラグ

`--no-wait`

認証 URL を出力し、ポーリングせずに終了

`--setup-code`

新規作成せず既存の setup code をポーリング

`--base-url`

プラットフォームの base URL を上書き

注記

- `--no-wait` を指定しない場合、CLI は最大 10 分間、5 秒ごとにポーリングします。
- `--no-wait` `--json` を指定すると、CLI は `verification_url`、`setup_code`、`expires_in` を返します。

例

```
tier0 login
```

```
tier0 login --no-wait --json
```

```
tier0 login --setup-code <setup_code>
```

`tier0 api`

API endpoint を直接呼び出す

使用方法

```
tier0 api <endpoint>
```

フラグ

`--body`

リクエストボディの JSON 文字列

`--body-file`

ファイルからリクエストボディを読み込む

`--method`

HTTP メソッド（GET|POST|PUT|DELETE）
デフォルト: `POST`

`--dry-run`

実行せずにリクエストをプレビュー

注記

- `--body` と `--body-file` は同時に指定できません。

例

```
tier0 api /openapi/v1/uns/browse --body '{"path":"/"}'
```

```
tier0 api /openapi/v1/uns/read --body '{"topics":["demo"]}'
```

```
tier0 api /openapi/v1/uns/write --body-file body.json
```

`tier0 upgrade`

CLI を最新バージョンにアップグレード

使用方法

```
tier0 upgrade
```

フラグ

`--dry-run`
インストールせず更新を確認

`-v, --version`
指定バージョンへアップグレード（デフォルト: latest）

例

```
tier0 upgrade
```

```
tier0 upgrade --dry-run
```

`tier0 uninstall`

tier0 CLI をアンインストール

使用方法

```
tier0 uninstall
```

フラグ

`--purge`
設定ファイル（認証情報）も削除

`--remove-skills`
検出された AI agents から Tier0 Skill も削除

`--keep-skills`
agent skills を保持（非推奨。現在はこれがデフォルトです）
非推奨

注記

- Agent Skills と設定はデフォルトで保持されます。
- CLI のアンインストール時に、ローカルの内蔵 skills ディレクトリは削除されます。

例

```
tier0 uninstall
```

```
tier0 uninstall --purge
```

```
tier0 uninstall --remove-skills
```

```
tier0 uninstall --purge --remove-skills
```

`tier0 version`

バージョンを表示

使用方法

```
tier0 version
```

例

```
tier0 version
```

スキル

`tier0 skills install`

CLI に内蔵された Skill をインストールまたは修復

使用方法

```
tier0 skills install
```

フラグ

`--force`
有効な Skill を内蔵ベースラインで置き換える

`--no-sync`
検出された AI agents に同期せず、ローカルのみにインストール

例

```
tier0 skills install
```

```
tier0 skills install --force
```

`tier0 skills list`

インストール済み skills を一覧表示

エイリアス: **ls**

使用方法

```
tier0 skills list
```

例

```
tier0 skills list
```

`tier0 skills update`

skills を最新バージョンに更新

エイリアス: upgrade

使用方法

```
tier0 skills update
```

フラグ

`--dry-run`
インストールせず更新を確認

例

```
tier0 skills update
```

```
tier0 skills update --dry-run
```

`tier0 skills version`

skills のバージョン情報を表示

エイリアス: ver

使用方法

```
tier0 skills version
```

例

```
tier0 skills version
```

`tier0 skills status`

有効な Skill と内蔵 Skill の状態を表示

使用方法

```
tier0 skills status
```

例

```
tier0 skills status
```

`tier0 skills sync`

有効な Skill を検出された AI agents に同期

使用方法

```
tier0 skills sync
```

例

```
tier0 skills sync
```

認証

`tier0 auth whoami`

現在の API key の識別情報を表示

使用方法

```
tier0 auth whoami
```

注記

- ユーザー、workspace、key 名、key タイプ、ロール、権限を表示します。

例

```
tier0 auth whoami
```

```
tier0 auth whoami --json
```

UNS

`tier0 uns browse`

UNS namespace ツリーを参照

使用方法

```
tier0 uns browse
```

フラグ

`-p, --path`
UNS ツリーで参照するパス
デフォルト: `/`

`-d, --max-depth`
最大再帰深度（0 = 無制限）
デフォルト: `1`

`--include-metadata`
ノードメタデータを含める

`--include-leaf-value`
リーフノードの値を含める

例

```
tier0 uns browse
```

```
tier0 uns browse --path /devices
```

```
tier0 uns browse --path / --max-depth 2
```

`tier0 uns search`

UNS topics を検索

使用方法

```
tier0 uns search
```

フラグ

`-k, --keyword`
名前キーワードで検索

`--path-prefix`
パス接頭辞でフィルタ
デフォルト: `/`

`--topic-type`
topic type でフィルタ

`--page`
ページ番号
デフォルト: `1`

`-l, --size`
ページサイズ（最大件数）
デフォルト: `20`

`--include-metadata`
ノードメタデータを含める

`--include-leaf-value`
リーフノードの値を含める

例

```
tier0 uns search --keyword temp
```

```
tier0 uns search --path-prefix /devices --size 50
```

```
tier0 uns search --keyword temp --include-metadata
```

`tier0 uns read`

UNS topics の現在値を読み取る

使用方法

```
tier0 uns read [topic...]
```

```
tier0 uns read --topic <topic>
```

フラグ

`-t, --topic`
読み取る topic 名（繰り返し可。位置引数も使用できます）
繰り返し可

`--include-metadata`
topic メタデータ（topicType、fields、description）を含める

`--include-leaf-value`
リーフノードの値を含める

注記

- 位置引数または `--topic` で少なくとも 1 つの topic が必要です。

例

```
tier0 uns read demo
```

```
tier0 uns read --topic demo
```

```

```

```
tier0 uns read temp humidity
```

```
tier0 uns read --topic sensor1 --include-metadata
```

`tier0 uns write`

UNS topic に値を書き込む

使用方法

```
tier0 uns write
```

フラグ

`-t, --topic`
書き込み先 topic 名（必須）
必須

`-v, --value`
JSON 値文字列（--file と同時指定不可）

`-f, --file`
ファイルから値を読み込む（--value と同時指定不可）

`--qos`
MQTT QoS レベル（0/1/2）
デフォルト: `0`

`--retain`
MQTT retain フラグを設定

`--dry-run`
実行せずにリクエストをプレビュー

注記

- `--value` と `--file` は同時に指定できません。
- 値は topic fields に一致する有効な JSON object である必要があります。

例

```
tier0 uns write --topic demo --value '{"temp":25}'
```

```
tier0 uns write --topic demo --file payload.json
```

```
tier0 uns write --topic sensor1 --value '{"on":true}' --qos 1 --retain
```

`tier0 uns history`

topics の履歴データを照会

使用方法

```
tier0 uns history
```

フラグ

`-t, --topic`

topic 名（繰り返し可、必須）
必須　繰り返し可

`--start`

開始時刻: 相対指定（-1h/-30m/-7d）、ISO 8601、または 'now'（必須）
必須

`--end`

終了時刻: 相対指定、ISO 8601、または 'now'（デフォルト: now）
デフォルト: `now`

`--page`

ページ番号
デフォルト: `1`

`-l, --size`

ページサイズ（最大データポイント数）

デフォルト: `100`

`--interval`

集計間隔（例: 1m、1h、1d）

`--fn`

集計関数（avg/max/min/sum/count）

`--field`

集計フィールド名

例

```
tier0 uns history -t demo --start -1h
```

```
tier0 uns history -t demo --start -24h --end now --fn avg --interval 1h
```

```
tier0 uns history -t demo --start 2026-01-01T00:00:00Z --end 2026-01-02T00:00:00Z
```

`tier0 uns create`

UNS namespace ノードを作成

使用方法

```
tier0 uns create
```

フラグ

`-t, --topic`

topic パスまたはリーフ名（--file を使用しない場合は必須）

`--parent`
親パス接頭辞（任意。--topic と組み合わせます）

`--type`
ノードタイプ: 'path'（フォルダー）または 'topic'（データポイント）

`-d, --display-name`
表示名

`--description`
説明

`--alias`
エイリアス

`-f, --file`
JSON ファイルから namespace 定義を読み込む（{"namespace":[]} または配列のみ）

`--topic-type`
非推奨: topic type は現在、パス（リーフ直前の Metric/Action/State フォルダー）から推定されます
非推奨

`--fields`
schema fields の JSON 配列

`--fields-file`
ファイルから schema fields の JSON 配列を読み込む

`--dry-run`
実行せずにリクエストをプレビュー

注記

- `--file` を使用しない場合、`--topic` と `--type` が必要です。
- `--fields` と `--fields-file` は同時に指定できません。
- topic ノードでは、リーフ直前のパス要素が Metric、Action、または State である必要があります。

例

```
tier0 uns create --topic Plant/Line1/Metric/Temperature --type topic
```

```
tier0 uns create --parent Factory1/Line1/Station1 --topic
Metric/ProductionCount --type topic
```

```
tier0 uns create --topic Plant/Line1 --type path --display-name 'Line 1'
```

```
tier0 uns create --file namespace.json
```

`tier0 uns update`

UNS topic のメタデータを更新

使用方法

```
tier0 uns update
```

フラグ

`-p, --path`
更新する topic パス（必須）
必須

`-n, --name`
新しい名前

`--alias`
新しいエイリアス

`--description`
新しい説明

`--clear-description`
空の shell 引数に依存せず説明をクリアする

`-d, --display-name`

新しい表示名

`--extend-properties`

拡張プロパティ JSON object

`--fields`

schema fields の JSON 配列

`--fields-file`

ファイルから schema fields の JSON 配列を読み込む

`--update-mask`

更新するフィールド（繰り返し可。例: name、description、fields）
繰り返し可

`--dry-run`

実行せずにリクエストをプレビュー

注記

- `--description` と `--clear-description` は同時に指定できません。
- `--fields` と `--fields-file` は同時に指定できません。
- 更新するフィールドを少なくとも 1 つ指定する必要があります。

例

```
tier0 uns update --path Plant/Line1/Metric/Temperature --display-name
'Line 1 Temp'
```

```
tier0 uns update --path Plant/Line1 --description 'Production line 1' --
update-mask description
```

```
tier0 uns update --path Plant/Line1/Metric/Temperature --fields
'[{"name":"temp","type":"float","unit":"C"}]' --update-mask fields
```

`tier0 uns delete`

UNS ノードを削除

使用方法

```
tier0 uns delete
```

フラグ

`-p, --path`
削除するノードパス（繰り返し可、必須）
必須　繰り返し可

`--hard`
ハード削除（元に戻せません）

`-y, --yes`
高リスク操作を確認（必須）
必須

`--dry-run`
実行せずにリクエストをプレビュー

注記

- デフォルトはソフト削除で、復元できます。
- `--hard` は元に戻せません。
- `--dry-run` を使用しない限り、高リスク操作には `--yes` が必要です。

例

```
tier0 uns delete --path factory/line1/sensor/temp --yes
```

```
tier0 uns delete --path factory/line1/sensor/temp --path
factory/line1/sensor/humi --yes
```

```
tier0 uns delete --path factory/line1/sensor/temp --hard --yes
```

`tier0 uns restore`

履歴から topic を復元

使用方法

```
tier0 uns restore
```

フラグ

`-p, --path`
復元する topic パス（必須）
必須

`-y, --yes`
高リスク操作を確認（必須）
必須

`--dry-run`
実行せずにリクエストをプレビュー

注記

- `--dry-run` を使用しない限り、高リスク操作には `--yes` が必要です。

例

```
tier0 uns restore --path Plant/Line1/Metric/Temperature --yes
```

Node-RED フロー

`tier0 flow list`

flows を一覧表示

エイリアス: ls

使用方法

```
tier0 flow list
```

フラグ

`-k, --keyword`
名前キーワードでフィルタ

`-t, --type`
タイプでフィルタ（SourceFlow/EventFlow）

`--source`
SourceFlow のみ表示

`--event`
EventFlow のみ表示

注記

- `--source` と `--event` は同時に指定できません。
- `--type` は `--source` または `--event` と同時に指定できません。

例

```
tier0 flow list
```

```
tier0 flow list --source --json
```

`tier0 flow get`

flow の詳細を取得

使用方法

```
tier0 flow get --id <id>
```

```
tier0 flow get <id>
```

フラグ

`--id`
Flow ID
デフォルト: `0`

注記

- `--id` または位置引数で正の Flow ID を指定する必要があります。

例

```
tier0 flow get --id 1 --json
```

```
tier0 flow get 1 --json
```

`tier0 flow create`

新しい flow を作成

使用方法

```
tier0 flow create
```

フラグ

`-n, --name`
Flow 名（必須）
必須

`-t, --type`
Flow タイプ: SourceFlow | EventFlow（必須）
必須

`--source`
タイプを SourceFlow に設定

`--event`
タイプを EventFlow に設定

`--desc`
説明

`--template`
初期テンプレート JSON 文字列

`--template-file`
ファイルから初期テンプレートを読み込む

`--dry-run`
実行せずにリクエストをプレビュー

注記

- Flow タイプの選択には `--type`、`--source`、`--event` のいずれか 1 つだけを使用してください。
- `--template` と `--template-file` は同時に指定できません。

例

```
tier0 flow create --name "modbus-collector" --source --desc "Modbus TCP
collector"
```

```
tier0 flow create --name "alert-handler" --event --desc "Temperature
alarm processor"
```

`tier0 flow update`

flow メタデータを更新

使用方法

```
tier0 flow update --id <id>
```

```
tier0 flow update <id>
```

フラグ

`--id`
Flow ID（必須）
デフォルト: `0`

`-n, --name`
新しい名前

`--desc`
新しい説明

`--template`
新しいテンプレート JSON 文字列

`--template-file`
ファイルから新しいテンプレートを読み込む

`--favorite`
お気に入りに追加

`--unfavorite`
お気に入りから削除

`--dry-run`
実行せずにリクエストをプレビュー

注記

- `--id` または位置引数で正の Flow ID を指定する必要があります。
- `--template` と `--template-file` は同時に指定できません。

- `--favorite` と `--unfavorite` は同時に指定できません。
- 更新するフィールドを少なくとも 1 つ指定する必要があります。

例

```
tier0 flow update --id 1 --name "line1-collector"
```

```
tier0 flow update --id 1 --favorite
```

`tier0 flow data`

Node-RED canvas JSON を取得

使用方法

```
tier0 flow data --id <id>
```

```
tier0 flow data <id>
```

フラグ

`--id`
Flow ID
デフォルト: `0`

`-o, --out`
出力をファイルに保存

注記

- `--id` または位置引数で正の Flow ID を指定する必要があります。

例

```
tier0 flow data --id 1 --out flows.json
```

`tier0 flow deploy`

Node-RED canvas JSON をデプロイ

使用方法

```
tier0 flow deploy --id <id>
```

```
tier0 flow deploy <id>
```

フラグ

`--id`
Flow ID（必須）
デフォルト: `0`

`--flows-json`
Node-RED canvas JSON 文字列

`-f, --flows-file`
ファイルから Node-RED canvas JSON を読み込む（推奨）

`-y, --yes`
高リスク操作を確認（必須）
必須

`--dry-run`
実行せずにリクエストをプレビュー

注記

- `--id` または位置引数で正の Flow ID を指定する必要があります。
- `--flows-json` と `--flows-file` は同時に指定できません。
- デプロイすると、この Flow 内の既存の Node-RED ノードがすべて置き換えられます。

- `--dry-run` を使用しない限り、高リスク操作には `--yes` が必要です。

例

```
tier0 flow deploy --id 1 -f flows.json --dry-run --json
```

```
tier0 flow deploy --id 1 -f flows.json --yes
```

`tier0 flow delete`

flow を削除

エイリアス: del, rm

使用方法

```
tier0 flow delete --id <id>
```

```
tier0 flow delete <id>[,<id>...]
```

フラグ

`--id`
削除する Flow ID（繰り返し可）
繰り返し可

`-y, --yes`
高リスク操作を確認（必須）
必須

`--dry-run`
実行せずにリクエストをプレビュー

注記

- 位置引数にはカンマ区切りの Flow ID を含められます。
- 削除すると Node-RED コンテナーが停止し、元に戻せません。
- `--dry-run` を使用しない限り、高リスク操作には `--yes` が必要です。

例

```
tier0 flow delete --id 1 --yes
```

`tier0 flow nodes`

利用可能な Node-RED ノードタイプを一覧表示

使用方法

```
tier0 flow nodes [source|event]
```

```
tier0 flow nodes --type <SourceFlow|EventFlow>
```

```
tier0 flow nodes --source
```

```
tier0 flow nodes --event
```

フラグ

`-t, --type`
Flow タイプ（SourceFlow/EventFlow）

`--source`
SourceFlow ノードを表示

`--event`

EventFlow ノードを表示

注記

- 位置引数の Flow タイプは最大 1 つまで指定できます。
- `--source` と `--event` は同時に指定できません。
- 位置引数/タイプ入力を `--source` または `--event` と併用しないでください。

例

```
tier0 flow nodes source --json
```

```
tier0 flow nodes --source --json
```

```
tier0 flow nodes --event --json
```

ファイル

`tier0 assets upload`

Tier0 object storage にファイルをアップロード

使用方法

```
tier0 assets upload <local-file>
```

フラグ

`--business`

業務シーン

デフォルト: `attachment`

`--use-by`

使用範囲: user|workspace|platform
デフォルト: `workspace`

`--visibility`

可視性: public|private
デフォルト: `private`

`--app-instance-id`

AI アプリインスタンス ID

`--session-id`

AI セッション ID

`--multipart-size`

マルチパートのパートサイズ（例: 10MB、最小 5MB）
デフォルト: `CLI-defined default`

`--concurrency`

マルチパートアップロードの同時実行数（並列パート数）
デフォルト: `4`

`--resume`

中断されたマルチパートアップロードを再開

`--abort`

中断されたマルチパートアップロードを中止し、状態をクリーンアップ

注記

- ローカルファイル引数を 1 つだけ指定する必要があります。
- 100MB を超えるファイルは自動的にマルチパートアップロードを使用します。
- CLI は厳密な最大ファイルサイズを強制しません。サーバー側でプラン/クォータ制限が適用されます。

例

```
tier0 assets upload ./report.csv
```

```
tier0 assets upload ./report.csv --visibility public --business
attachment
```

```
tier0 assets upload ./report.csv --use-by workspace --visibility private
```

```
tier0 assets upload ./large-backup.tar.gz
```

```
tier0 assets upload ./large-backup.tar.gz --resume
```

```
tier0 assets upload ./large-backup.tar.gz --abort
```

```
tier0 assets upload ./large-backup.tar.gz --multipart-size 20MB --
concurrency 8
```

`tier0 assets download`

Tier0 object storage からファイルをダウンロード

使用方法

```
tier0 assets download
```

フラグ

`--file-path`
アップロードで返されたファイルパス
必須

`-o, --output`

出力ファイルパス（デフォルト: stdout）

`--content-disposition`

カスタム Content-Disposition

例

```
tier0 assets download --file-path workspace/.../report.csv -o
./report.csv
```

```
tier0 assets download --file-path workspace/.../report.csv
```

`tier0 assets url`

Tier0 object storage のファイルアクセス URL を取得

使用方法

```
tier0 assets url
```

フラグ

`--file-path`

アップロードで返されたファイルパス

必須

`--expired-sec`

署名付き URL の有効期限（秒）

デフォルト: `3600`

`--content-disposition`

カスタム Content-Disposition

例

```
tier0 assets url --file-path workspace/.../report.csv
```

```
tier0 assets url --file-path workspace/.../report.csv --expired-sec 300
```

`tier0 assets delete`

Tier0 object storage のファイルを削除

使用方法

```
tier0 assets delete
```

フラグ

`--file-path`
アップロードで返されたファイルパス
必須

`-y, --yes`
高リスク操作を確認（必須）
必須

`--dry-run`
実行せずにリクエストをプレビュー

注記

- 削除は元に戻せません。
- `--dry-run` を使用しない限り、高リスク操作には `--yes` が必要です。

例

```
tier0 assets delete --file-path workspace/.../report.csv --dry-run --
json
```

```
tier0 assets delete --file-path workspace/.../report.csv --yes
```

リファレンス

組み込み Builder Skills

Tier0 Builder に組み込まれた AI Skills のリファレンス。

Tier0 App Builder には現在、7 個の active な組み込み AI Skills があります。これらの Skills は、agent が要件を産業アプリケーションに変換し、Tier0 platform scaffold に合わせ、UI の問題を修復し、preview 環境を安定させるために使われます。

`app-srs`

意味：新しい産業向け enterprise app の requirements analysis entry point です。WMS、MES、ERP、CRM、QMS、EAM、承認、在庫、サプライヤー、顧客、スケジューリングなどの enterprise workflow に使用します。

例：

```
$app-srs を使用して、自動車部品工場向けの品質異常管理システムを設計してください。
```

自然言語でも起動できます：

```
入荷、棚入れ、ピッキング、出荷、棚卸しを含む倉庫管理アプリを生成してください。
```

実行内容：

1. 業務ロール、workflow、状態、例外を分析します。
2. 要件が十分に明確か確認します。
3. 完全な SRS を生成します。
4. デフォルトで `specs/spec.md` に保存します。
5. ユーザーが要件のみを求めない限り、結果を `app-builder` に渡します。

`app-builder`

意味：アプリケーションを実際に構築する core skill です。

例：

```
$app-builder を使用して specs/spec.md から最初のバージョンを構築してください。
```

内部作業タイプ：agent は prompt と現在の project state に基づいて使用する work type を決定します。

- **First Release**：SRS から最初のバージョンを構築します。
- **Delta Feature**：既存アプリに大きな module または workflow を追加します。
- **Light Edit**：小さな field、copy、style、validation の変更を、完全な build flow なしで処理します。

確認項目：

- 見える入口。
- 実行可能なユーザーパス。
- データまたは状態の変化。
- ロール権限。
- build と preview の検証。

> ヒント
> 初回 build で `specs/spec.md` がない場合、agent は先に `app-srs` で作成します。

`align-platform-app`

意味：既存アプリを Tier0 platform scaffold に合わせるための migration skill です。

```
$align-platform-app を使用して、この古い Vite React project を Tier0 platform
scaffold に移行してください。
```

適用できる **source**：

- Vite、CRA、Next.js、React Router、static HTML、full-stack Node/React apps。
- 旧 Tier0 apps。
- LocalStorage、IndexedDB、Zustand persist、in-memory mocks、simulated data に依存する apps。

実行内容：

- Tier0 TanStack Start scaffold に合わせます。
- 永続的な業務データを Drizzle/PostgreSQL に移行します。
- routes、APIs、services、database setup を合わせます。
- app 独自の login、logout、role switching flow を削除します。
- Gateway が管理する identity と roles に接続します。
- `/api/health`、`/api/manifest`、`artifact.toml` などの platform contracts を保持します。
- preview と production startup を検証します。

`migrate-node-red-app`

意味：Node-RED application を Tier0 platform scaffold に合わせるための migration skill です。通常は `align-platform-app` と一緒に使います。

例：

```
$align-platform-app と $migrate-node-red-app を使用して、この Node-RED backend
を Tier0 TanStack Start app に移行してください。
```

実行内容：

- flows、subflows、function nodes、change nodes、switch nodes を棚卸しします。
- HTTP、MQTT、database、scheduled jobs、external API calls を分析します。
- migration mapping table を作成します。
- CRUD、workflow、scheduling、integration、一時的な simulation behavior を分離します。
- business logic を services に移します。

- interfaces を API routes に移します。
- state を PostgreSQL に移します。
- long-running work と integrations を workers、queues、または external services に移します。
- browser、CRUD、workflow、integration cutover を個別に検証します。

> ⚠ 注意
> 通常の preview や packaging の問題は単独では扱いません。

`ui-repair`

意味：既存アプリ向けの repair skill です。初回 app generation ではなく、UI quality issues に使います。

例：

```
$ui-repair を使用して、小さい画面での注文編集 dialog の overflow と form misalignment を修正してください。
```

適した問題：

- form や required marker のずれ。
- dialog または drawer の overflow。
- 予期しない horizontal scrolling。
- select menu または dropdown の clipping。
- table または chart layout の崩れ。
- visual hierarchy が弱い flat page。
- UI language の混在。
- PDA、kiosk、dashboard、industrial interface layout の問題。

> ⚠ 注意
> この Skill は最小の owning component を特定し、該当 surface を修復します。app 全体を作り直すものではありません。複雑な responsive work には `responsive-craft` を使って

ください。

`responsive-craft`

意味：responsive layout の audit、新規 responsive build、multi-breakpoint preview のための skill です。

呼び出しモード：

- **audit**：既存の responsive layout を audit して修復します。
- **build**：responsive page をゼロから構築します。
- **preview**：multi-breakpoint live preview を開始します。

例：

```
$responsive-craft audit を使用して、この dashboard の mobile layout を修正してください。
```

```
$responsive-craft build を使用して、phone、tablet、desktop に対応する inventory workspace を作成してください。
```

```
$responsive-craft preview を使用して、375、768、1024、1440 px の breakpoints を表示してください。
```

実行スタイル：

- **Adaptive**：すばやく進め、design fork が出た時点で確認します。
- **Guided**：実装前に各 component の responsive behavior spec を書きます。

`preview-runtime-stability`

意味：app development 中の preview failure を診断する runtime troubleshooting skill です。business feature は構築しません。

例：

```
$preview-runtime-stability を使用して "Preview loading timed out" を診断してください。
```

適した問題：

- `node_modules` の欠落または不完全な install。
- npm `ERESOLVE`。
- Vite または TanStack Start の startup failure。
- port `5173` conflict。
- `/api/health` failure。
- SDK SSR module error。
- `artifact.toml` preview port mismatch。
- HMR 中の frequent full-page reload。
- stale Node または Vite process が終了していない。

標準フロー：

1. dependency installation を確認します。
2. port `5173` を確認します。
3. health endpoint を probe します。
4. preview が healthy なら再利用します。
5. unhealthy なら old preview を cleanup します。
6. restart して port、health endpoint、HMR behavior を検証します。

Skill 選択ガイド

- 要件が不明確、または durable specification が必要な場合は、まず `app-srs` を使います。
- implementation scope が明確な場合は `app-builder` を使います。
- migration work には `align-platform-app` と `migrate-node-red-app` を使います。

- 既存 app の visual、responsive、runtime 問題には `ui-repair`、`responsive-craft`、`preview-runtime-stability` を使います。

リファレンス

ユーザー管理

Tier0 はメンバー資格、ライセンス、カスタムユーザーアカウントを通じて、お客様によるプラットフォームへのアクセスとユーザー権限の管理を可能にします。

Tier0 では、エディションごとに異なる方法でユーザー権限とアクセスを管理します。

- Cloud では、Workspace へのメンバー招待によって、誰が Workspace にアクセスできるかを管理します。
- Enterprise では、管理者がユーザーアカウントとロールを割り当て、アクセスできる機能を決定します。

アクセス制御

タブ: CLOUD

- Workspace のオーナーまたは既存のメンバーは、電話番号またはメールアドレスを使って新しいメンバーを招待できます。
- 新しいメンバーは、招待コードまたは招待リンクを受け取った後、アカウントを登録して Workspace に参加します。
- 招待時に新しいメンバーのシート種別（Operator または Builder）を設定し、利用できる機能の範囲を決めます。
    - **Builder**：すべての機能モジュールにアクセスできます。
    - **Operator**：アプリケーションセンターにのみアクセスできます。
- アカウント登録には 2 段階の確認が必要です。

> ⚠ 注意
> Workspace のメンバー資格を得ても、プラットフォームへのアクセスが自動的に付与されるわけではありません。プラットフォームを利用するには、招待を受けた電話番号またはメールアドレスでアカウントを登録する必要があります。

> ⓘ ノート
> アカウント登録後、新しい Workspace を申請することもできます。

タブ: ENTERPRISE

- 管理者はデフォルトのアカウントで初めてサインインした後、パスワードを変更し、**設定**でユーザーアカウントを追加してロールを割り当てます。
- 管理者は、アカウントの割り当てによってプラットフォームを利用できるユーザーを決定し、ロールの割り当てによって各アカウントがアクセスできる機能を決定します。
    - **Admin**：すべての機能モジュールにアクセスできます。
    - **Operator**：アクセス可能なすべてのアプリケーションが一覧表示される **Launchpad** にのみアクセスできます。

> ⓘ ノート
> 管理者が作成したアカウントでサインインし、割り当てられたロールの権限内で機能を利用します。

アカウント管理

タブ: CLOUD

操作	実行できるユーザー	説明
シート種別の変更	作成者 / Builder	• 作成者はすべてのメンバーのシート種別を変更できます。 • Builder は Operator のシート種別のみ変更できます。
Workspace の削除	作成者	Workspace を削除すると、すべてのデータが直ちに消去され、復元できません。
Workspace の移譲	作成者	—

操作	実行できるユーザー	説明
メンバーの削除	作成者 / Builder	• 作成者はすべてのメンバーを削除できます。 • Builder は Operator のみ削除できます。
アカウントの削除	すべてのメンバー	• 作成者は先に Workspace を削除するか、ほかのメンバーに移譲する必要があります。 • メンバーのアカウントは直ちにすべての Workspace から退出します。 • アカウントデータは直ちに消去され、復元できません。

タブ: ENTERPRISE

- Admin はパスワードのリセット、ユーザーロールの編集、アカウントの無効化、アカウントの削除を行えます。
- 初期デフォルトアカウントのロールは編集できず、このアカウントを無効化または削除することもできません。
- アカウントを削除すると、すべてのデータが直ちに消去され、復元できません。

リファレンス

Tier0 Cloud のネットワークとデータのセキュリティに関する説明

Tier0 Cloud の本番 SaaS サービスにおけるネットワーク境界、テナント分離、データ保護、セキュリティ運用について説明します。

適用範囲： AWS シンガポールリージョンにデプロイされた Tier0 Cloud の本番 SaaS サービス。
文書日付： 2026 年 10 月 22 日

Tier0 Cloud は、ネットワークアクセス制御、テナント分離、ID とアクセスの管理、データ暗号化、バックアップと復旧によって、ネットワーク攻撃や不正アクセスのリスクを低減します。本書では、プラットフォームのセキュリティ対策と、セキュリティインシデントが他のテナント、プラットフォーム内部のサービス、お客様のネットワークへ波及することを抑える制御の境界について説明します。

> ⓘ **Edge エディション**
> この説明は Tier0 Cloud のみに適用されます。Edge のセットアップについては、インストールを参照してください。

1 ネットワーク境界とアクセス制御

Tier0 Cloud の公開 Web および API へのアクセスは TLS で暗号化され、統一されたエッジ入口とアプリケーションゲートウェイを経由します。主要サービスの入口ではオリジンへのアクセスを検証し、検証要件を満たさない直接アクセスを拒否することで、エッジ入口を迂回したオリジンへのアクセスリスクを低減します。

本番環境は、クラウドアカウント、ネットワーク、クラスター、データベースの各層で開発環境およびステージング環境から分離され、アクセス認証情報も個別に管理されます。本番環境への管理アクセスには、認証済みの専用ネットワーク経路を使用します。

2 テナント分離とインシデントの波及防止

2.1 ワークロードの分離

テナントのワークロードは個別の Kubernetes 名前空間で実行され、専用ノードプールとスケジューリング制約によって、プラットフォームのサービスおよび管理コンポーネントから分離

されます。この仕組みは、テナントのアプリケーションとプラットフォームのコンポーネントが直接接触する範囲を制限し、あるアプリケーションでセキュリティインシデントが発生した場合の他のサービスへの影響を低減します。

2.2 ネットワークの分離

テナントの名前空間に対する受信・送信通信には、デフォルトで拒否し、明示的なルールでのみ許可するネットワークポリシーを適用します。プラットフォームがポリシーを自動的に設定・同期し、クラスターのネットワークコンポーネントが適用します。

受信通信は、同一テナントのワークロードと、必要な業務機能を担うプラットフォームサービスからの通信に限定されます。送信通信は、クラスター内の DNS 名前解決、同一テナントのサービス、指定されたプラットフォームゲートウェイ、メッセージングサービス、割り当てられたデータベースエンドポイントに対して、ルールに従って許可されます。パブリックインターネットへのアクセスもネットワークポリシーの制約を受けます。明示的に許可された業務上の依存先を除き、プライベートネットワークアドレスへのアクセスを制限し、クラウドインスタンスのメタデータアドレスへの直接アクセスを遮断します。

これらの対策は、許可されていないテナント間接続や、テナントのワークロードからプラットフォーム内部ネットワークへのラテラルムーブメントを制限します。あるテナントのアプリケーションでセキュリティインシデントが発生しても、それだけで他のテナントやプラットフォームの管理コンポーネントへのアクセス権限が付与されることはありません。許可済みの業務接続にも、対応するネットワークルールと ID・アクセス制御が適用されます。

2.3 データと ID の分離

テナントのアプリケーションデータは、データベース層で論理的に分離されます。個別のデータベース単位が割り当てられる場合は、ネットワークアクセスルールによって接続元も制限されます。

プラットフォームはサービスの役割に応じてアクセス権限を設定し、各ワークロードがアクセスできるリソースを必要な機能の範囲に限定します。ネットワーク分離と ID に基づく認可を組み合わせることで、影響を受けたワークロードがアクセスできるリソースの範囲を制限します。

2.4 お客様のネットワークへの接続境界

本節のテナント分離対策は、Tier0 Cloud のプラットフォーム内部に適用されます。お客様がコネクター、ゲートウェイ、その他の連携方式で内部システムを接続する場合、お客様のネットワークへのアクセス境界は、実際の接続方式、ルーティング、ファイアウォールルール、使用する認証情報に付与された権限にも依存します。

接続時には、業務に必要な宛先アドレス、ポート、アカウント権限だけを許可し、接続コンポーネントと他の内部システムをネットワーク上で分離してください。プラットフォーム内部の分離は、お客様側のネットワークアクセス制御を代替するものではありません。双方が具体的な接続構成に基づいてアクセス範囲を決定・検証し、許可済みの接続を経由して他の内部システムへ影響が波及するリスクを抑える必要があります。

3 データ保護

プラットフォームは、データの転送、保存、アクセスに対して次の保護対策を実施します。

- 公開 Web および API の通信と、マネージドデータベースおよびキャッシュへのアクセスに TLS 暗号化を使用します。
- マネージドデータベース、キャッシュ、オブジェクトストレージでは保存時の暗号化を有効にします。
- オブジェクトストレージではパブリックアクセスをブロックし、暗号化要件、HTTPS 接続要件、リソース権限によってデータアクセスを制限します。
- テナントデータへのアクセスには、データベース分離と ID・アクセス制御の両方を適用します。

4 バックアップと復旧

Tier0 Cloud は、本番データベースと、バックアップ対象に含まれるサービス単位のデータベースをバックアップします。バックアップデータはアクセスが制限された専用のオブジェクトストレージに保存され、対応する保持ポリシーに従って管理されます。

バックアップジョブには専用の ID を使用し、指定された保存先で必要な読み取り・書き込み操作だけを許可します。バックアップオブジェクトの削除権限は付与しません。保持期限が切れたバックアップはストレージのライフサイクルルールで削除し、バックアップ用の認証情報が漏えいした場合に復旧用データが削除されるリスクを低減します。

バックアップ完了後は整合性を確認し、分離された環境でのリストア検証によってバックアップが利用可能であることを確認します。アプリケーション設定はバージョン管理し、ビルド成

果物は非公開のコンテナーイメージレジストリに保存することで、サービスの復旧とバージョンのロールバックを支援します。

5 管理アクセスと本番環境の変更

本番環境への管理アクセスには、個人ごとに割り当てられた ID を使用し、最小権限の原則に従って権限を付与します。機密性の高い操作を行うコンソールの ID には、多要素認証を必須とします。

日常点検には、ワークロードの状態、ログ、イベントを参照できる制限付きの読み取り専用 ID を使用します。この ID では、機密性の高い認証情報の読み取り、リソースの変更、リモートコマンドの実行はできません。本番環境の変更には、個別に認可された ID と手順を使用します。

インフラストラクチャ設定、デプロイマニフェスト、運用スクリプトはバージョン管理します。本番環境の変更では、目的、影響範囲、検証手順、ロールバック計画を明確にし、実行者の ID、設定差分、検証結果を記録して、追跡と復旧を可能にします。

6 監視とセキュリティインシデント対応

プラットフォームは、本番環境の稼働メトリクス、ログ、健全性情報を集約し、異常アラートを運用対応チャネルへ送信します。監視対象には、サービスの可用性、エラー率、データベースの健全性、証明書の有効期限、バックアップジョブの状態、クラスターの稼働状態が含まれます。

セキュリティインシデントが発生した場合は、影響範囲の封じ込めを優先し、関連するログ、設定、発生から対応までの時系列記録を保全します。必要な隔離と調査を完了した後にサービスを復旧し、その後、振り返りと是正措置の進捗管理を行います。インシデント対応中の本番環境の変更も、所定の手順に従って認可・記録します。

7 セキュリティに関するお問い合わせ

セキュリティ評価、お客様によるデューデリジェンス、脆弱性の疑いについては、Tier0 の担当者またはセキュリティチームにお問い合わせください。非公開のアーキテクチャや制御の詳細は、適切な秘密保持の取り決めの下で提供できる場合があります。初回のお問い合わせには、アクセス認証情報、秘密鍵、お客様の機密データを含めないでください。

リファレンス

ISA-95 設備階層

ISA-95 で物理資産と制御システムをモデル化するための参照階層。

ISA-95 の設備階層は、組織内の物理資産と制御システムがどのように構造化されるかを示します。

次の階層に分かれます：

- **Enterprise**：最上位の階層で、組織全体を表します。全体の業務プロセスや意思決定主体を含み、通常は ERP システムに支えられます。
- **Site**：特定の地理的な拠点または施設を表します。各 site は独自の管理システムとローカルな生産制御を持つことができます。
- **Area**：site 内の運用区分で、包装、組立、倉庫などの機能エリアを表すことが多いです。
- **Process Cell**：area 内の製造の基本単位です。特定の製品を生産したり特定の機能を実行したりするために必要な設備を含みます。
- **Unit**：process cell 内の独立した部分で、混合、加熱、組立などの特定機能を担当します。
- **Equipment Module**：unit 内で特定タスクを実行するモジュール型設備です。たとえば混合 unit では、加熱要素が equipment module になります。
- **Control Module**：最下位の階層で、センサー、ポンプ、アクチュエーター、PID 制御ループなど、物理プロセスを直接制御する個別デバイスやコントローラーを表します。

リファレンス

標準デプロイ計画

Tier0 Enterprise の 4 台の仮想マシンによる標準デプロイの参考構成。

目的

簡易構成では Docker Swarm を使用せず、単一の `enterprise` アプリケーションノードを配置します。仮想マシンは合計 `4` 台です。

- `1` 台の enterprise ノードで `tier0-enterprise + EMQX` を実行します。
- `2` 台のデータベースノードで PostgreSQL + TimescaleDB のプライマリ／スタンバイクラスターを構成します。
- `1` 台の Witness ノードがデータベースの監視、クォーラム判定、フェイルオーバーに参加します。

簡易構成はデプロイの複雑さを低減します。一方、アプリケーション層と EMQX は単一障害点となります。`DB VIP + Witness` による高可用性を維持するのはデータベース層のみです。

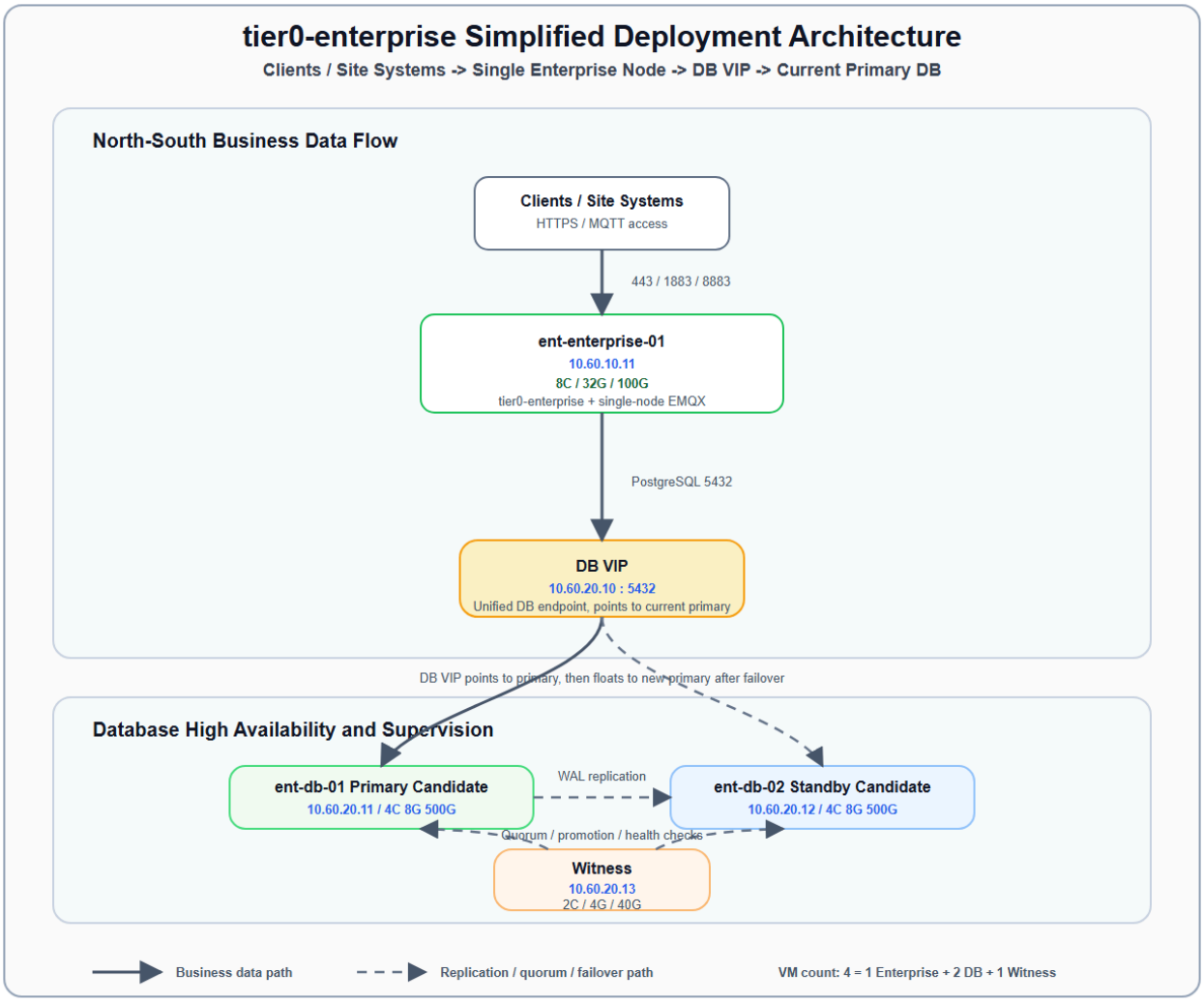
IP アドレス計画

以下の IP アドレスは計画用のプレースホルダーです。実装前に現場の実際のネットワークに合わせて置き換えてください。

種別	名前	計画 IP	説明
アプリケーションノード	`ent-enterprise-01`	`10.60.10.11`	`tier0-enterprise + EMQX` を実行する単一の enterprise ノード
論理エンドポイント	`DB VIP`	`10.60.20.10`	現在のプライマリのみを指す共通データベースエンドポイント
データベースノード	`ent-db-01`	`10.60.20.11`	PostgreSQL + TimescaleDB のプ

種別	名前	計画 IP	説明
			ライマリ候補
データベースノード	`ent-db-02`	`10.60.20.12`	PostgreSQL + TimescaleDB のスタンバイ候補
クォーラムノード	`ent-db-witness-01`	`10.60.20.13`	`repmgr witness` ／データベースのクォーラム監視

全体トポロジー



主要な経路：

- 業務経路：クライアント／現場システム → `ent-enterprise-01` → `DB VIP` → 現在のプライマリデータベース。

- `tier0-enterprise` と `EMQX` は、どちらも `ent-enterprise-01` 上で実行します。
- データベースへのアクセスには必ず `DB VIP` を使用し、アプリケーションから `ent-db-01/02` への直接アクセスは禁止します。
- `Witness` はデータベースのクォーラム判定、昇格、健全性確認のみに参加し、業務経路には含まれません。

データベースの監視とフェイルオーバー

フェーズ	経路	説明
通常のアクセス	`ent-enterprise-01 -> DB VIP -> current primary`	アプリケーションは常に `10.60.20.10:5432` に接続します
通常のレプリケーション	`current primary -> current standby`	PostgreSQL WAL ストリーミングレプリケーション
通常の監視	`DB-01/DB-02 -> Witness`	`repmgrd` が状態を報告し、Witness がクォーラム判定に参加します
プライマリ障害	`standby -> Witness`	スタンバイがクォーラム確認を要求し、スプリットブレインを防止します
スタンバイの昇格	`standby promote`	Witness の確認後にスタンバイを昇格させます
VIP の移動	`DB VIP -> new primary`	`10.60.20.10` を新しいプライマリに割り当てます
アプリケーションの復旧	`ent-enterprise-01 -> DB VIP -> new primary`	接続プールの再接続後にアプリケーションが復旧します

ノード構成

ノード	役割	vCPU	メモリー	ストレージ	システムディスクの推奨構成	データディスクの推奨構成	主要コンポーネント
`ent-enterprise-01`	単一の enterprise ノード	`8`	`32G`	`100G`	単一の `/` システムディスク、100G	個別のデータディスクなし	tier0-enterprise、EMQX、任意で Nginx/HAProxy
`ent-db-01`	PostgreSQL/TSDB のプライマリ候補	`4`	`8G`	`500G`	`/` 80G	`/pgdata` 360G、`/pgwal` 40G、`/backup-staging` 20G	PostgreSQL、TimescaleDB、repmgr、keepalived
`ent-db-02`	PostgreSQL/TSDB のスタンバイ候補	`4`	`8G`	`500G`	`/` 80G	`/pgdata` 360G、`/pgwal` 40G、`/backup-staging` 20G	PostgreSQL、TimescaleDB、repmgr、keepalived

ノード	役割	vCPU	メモリー	ストレージ	システムディスクの推奨構成	データディスクの推奨構成	主要コンポーネント
`ent-db-witness-01`	Witness／クォーラム監視	`2`	`4G`	`40G`	`/` 40G	個別の割り当てなし	repmgr witness、監視エージェント、監査ログ

構成の集計

分類	台数	ノードごとの仕様	vCPU 小計	メモリー小計	ストレージ小計
enterprise アプリケーションノード	`1`	`8C / 32G / 100G`	`8C`	`32G`	`100G`
PostgreSQL + TimescaleDB ノード	`2`	`4C / 8G / 500G`	`8C`	`16G`	`1000G`
Witness ノード	`1`	`2C / 4G / 40G`	`2C`	`4G`	`40G`
合計	`4`	-	`18C`	`52G`	`1140G`

合計値：

- 仮想マシン：`4` 台
- vCPU：`18`
- メモリー：`52G`

- ストレージ：`1.14T`

接続マトリックス

運用アクセス

対象	方法	ポート	説明
すべての仮想マシン	`ssh -i ~/.ssh/chen-rsa ops@<IP>`	`22/TCP`	共通の鍵認証による SSH を実装後に使用します。それまでは現場のアカウントを使用します
`ent-enterprise-01`	`https://10.60.10.11`	`443/TCP`	業務用エンドポイント
`DB VIP`	`psql -h 10.60.20.10 -p 5432`	`5432/TCP`	enterprise ノードと運用の許可リストのみ許可します

アプリケーションと EMQX のポート

接続元	接続先	ポート	用途
ユーザー／現場システム	`ent-enterprise-01`	`80/TCP`	HTTP 入口または HTTPS リダイレクト、任意
ユーザー／現場システム	`ent-enterprise-01`	`443/TCP`	HTTPS の業務用入口
デバイス／現場システム	`ent-enterprise-01`	`1883/TCP`	EMQX MQTT 平文通信、任意
デバイス／現場システム	`ent-enterprise-01`	`8883/TCP`	EMQX MQTT TLS、推奨

接続元	接続先	ポート	用途
デバイス／現場システム	`ent-enterprise-01`	`8083/TCP`	MQTT WebSocket、任意
デバイス／現場システム	`ent-enterprise-01`	`8084/TCP`	MQTT WebSocket TLS、任意
運用ネットワーク	`ent-enterprise-01`	`18083/TCP`	EMQX Dashboard、運用の許可リストのみ
`ent-enterprise-01`	`DB VIP`	`5432/TCP`	PostgreSQL の業務アクセス

データベースと監視の通信

接続元	接続先	ポート	用途
`ent-db-01`	`ent-db-02`	`5432/TCP`	PostgreSQL WAL ストリーミングレプリケーション
`ent-db-02`	`ent-db-01`	`5432/TCP`	復旧後の逆方向レプリケーション
`ent-db-01/02`	`ent-db-witness-01`	`5432/TCP`	repmgr メタデータ／Witness 通信
`ent-db-01`	`ent-db-02`	`VRRP`	DB VIP の移動。プロトコル `112` を許可します

デプロイの推奨事項

enterprise アプリケーションノード

- Docker Swarm はデプロイしません。
- `tier0-enterprise` と `EMQX` を同一の `ent-enterprise-01` ノードにデプロイします。

- `ent-enterprise-01` はアプリケーション層の単一障害点です。障害時は手動で復旧するか、バックアップから再構築します。
- このノードには `100G` のシステムディスクが 1 台しかないため、業務データの長期保存には適しません。
- `tier0-enterprise` の業務データは PostgreSQL/TimescaleDB に保存する必要があります。
- アップロード、インポート／エクスポート、レポートのファイルはローカルに保存してバックアップできます。後で標準構成へ移行する場合は、これらのファイルを NFS/NAS またはオブジェクトストレージへ移行してください。

単一ノードの EMQX

- 簡易構成の EMQX は、単一ノードのステートフルサービスです。
- データディレクトリを `ent-enterprise-01` 上に固定します。例：`/data/emqx`。
- 設定、証明書、プラグイン、ルールエンジンの設定をバックアップします。
- メッセージ層の高可用性は提供しません。これが要件を満たさない場合は、3 ノードの EMQX クラスターを持つ標準構成を使用してください。

PostgreSQL + TimescaleDB

- PostgreSQL：`16` または現場で検証済みのバージョン。
- TimescaleDB：選択した PostgreSQL のメジャーバージョンと互換性のある安定版。
- レプリケーション：物理ストリーミングレプリケーション。
- フェイルオーバー：`repmgrd + witness`。
- データベースの入口：`DB VIP`。
- 接続プール：`ent-enterprise-01` 上で PgBouncer を実行し、データベースへの接続負荷を低減できます。

Witness

- Witness は `repmgr witness` のメタデータと監視ロジックのみを保持します。
- Witness は業務 SQL を処理しません。

- Witness の障害で現在のプライマリは停止しませんが、自動フェイルオーバーの信頼性は低下します。

ポートとファイアウォールの基準

ポート／プロトコル	許可範囲	用途
`22/TCP`	運用ネットワークから 4 台の仮想マシン	SSH
`80/TCP`	ユーザーネットワークから `ent-enterprise-01`	HTTP リダイレクトまたは内部の平文通信入口
`443/TCP`	ユーザーネットワークから `ent-enterprise-01`	HTTPS の業務用入口
`1883/TCP`	デバイス／現場システムから `ent-enterprise-01`	EMQX MQTT 平文通信、任意
`8883/TCP`	デバイス／現場システムから `ent-enterprise-01`	EMQX MQTT TLS、推奨
`8083/TCP`	デバイス／現場システムから `ent-enterprise-01`	EMQX WebSocket、任意
`8084/TCP`	デバイス／現場システムから `ent-enterprise-01`	EMQX WebSocket TLS、任意
`18083/TCP`	運用ネットワークから `ent-enterprise-01`	EMQX Dashboard、許可リストのみ
`5432/TCP`	enterprise から `DB VIP`、DB/Witness 内部	PostgreSQL／repmgr
`112/VRRP`	データベースノード間	DB VIP の移動
`9100/TCP`	監視システムから 4 台の仮想マシン	node exporter、任意

高可用性の機能と障害の境界

障害シナリオ	想定される動作	業務への影響
`ent-enterprise-01` の障害	アプリケーションと EMQX が停止し、手動復旧またはバックアップからの再構築が必要です	アプリケーション層が全面停止します
現在の DB プライマリの障害	Witness のクォーラム確認後にスタンバイが昇格し、DB VIP が移動します	数秒から数分の書き込み中断。アプリケーションの再接続が必要です
DB スタンバイの障害	プライマリは稼働を継続し、レプリケーションのアラートが発生します	直接の業務停止はありませんが、DB のホットスタンバイを失います
Witness の障害	現在のプライマリは稼働を継続しますが、自動フェイルオーバーの信頼性が低下します	直接の業務停止はありません。Witness を優先して復旧します
DB VIP の移動失敗	新しいプライマリが存在しても、共通の入口に到達できない場合があります	業務が停止します。VIP の手動割り当てが必要です

本番稼働前のチェックリスト

確認項目	合格基準
4 台の仮想マシンのホスト名が固定されている	`hostnamectl` が計画と一致します
共通の SSH 接続が利用できる	運用端末から鍵認証で 4 台すべてにアクセスできます
時刻同期	4 台間の時刻差が `1s` 未満です
ファイアウォールが許可されている	第 9 節のポートが定められた範囲で許可されています

確認項目	合格基準
enterprise サービスが正常である	`tier0-enterprise` のヘルスチェックに合格します
EMQX の状態が検証されている	EMQX ノード、待ち受けポート、永続化ディレクトリが正常です
enterprise のディスク使用率が安全な範囲にある	`/` の使用率が `70%` 未満で、ログとキャッシュのクリーンアップが設定されています
DB のプライマリ／スタンバイが正常である	`repmgr cluster show` でプライマリ、スタンバイ、Witness が正常です
DB VIP が利用できる	enterprise ノードから `10.60.20.10:5432` に接続できます
DB プライマリのフェイルオーバー訓練	スタンバイの昇格と DB VIP の移動ができます
バックアップのリストア訓練	テスト用データベースへバックアップをリストアできます

実装の順序

1. 4 台の仮想マシンを作成し、ホスト名、IP、時刻同期、SSH を設定します。
2. OS の基本設定、ファイアウォール、マウント、カーネルパラメーターを初期化します。
3. `ent-enterprise-01` に `tier0-enterprise` をデプロイします。
4. `ent-enterprise-01` に単一ノードの EMQX をデプロイします。
5. PostgreSQL + TimescaleDB のプライマリ／スタンバイをデプロイします。
6. repmgr witness、repmgrd、DB VIP 移動スクリプトをデプロイします。
7. 接続、EMQX の復旧、データベースのフェイルオーバー、バックアップのリストア、日常点検を検証します。
8. 運用 Runbook、認証情報の保管、日常点検タスクを整備します。

リスクと未確定事項

- 簡易構成にはアプリケーション層の単一障害点があります。`ent-enterprise-01` が停止すると、業務サービス全体が利用できなくなります。
- EMQX は単一ノードであり、メッセージ層の高可用性を提供しません。
- `ent-enterprise-01` のシステムディスクは `100G` の 1 台のみです。ログ、キャッシュ、添付ファイル、一時ファイルの使用量を管理する必要があります。
- 後でアプリケーション層または EMQX の高可用性が必要になった場合は、標準構成へ移行します。
- データベースの RPO/RTO は未確定です。このプライマリ／スタンバイ構成はデータ損失ゼロを保証しません。
- DB VIP 移動スクリプトは現場での検証が必要です。特に、仮想化／ネットワーク基盤が VRRP を許可していることを確認してください。

リファレンス

高可用性デプロイ計画

Tier0 Enterprise の高可用性環境向けリファレンスデプロイ計画。

計画の目的

この V2 計画は、オフラインまたはプライベート環境に論理的な Tier0 Enterprise Fleet Center をデプロイするためのものです。6 台のマシンが、業務、ミドルウェア、データベース、ストレージ、Agent の役割を分担します。ユーザーと Enterprise 内部サービスは常に固定 VIP アドレスに接続します。1 台のマシンまたは 1 つのプロキシに障害が発生した場合、Keepalived、HAProxy、repmgr、RustFS、Fleet Agent がそれぞれのレイヤーで引き継ぎます。

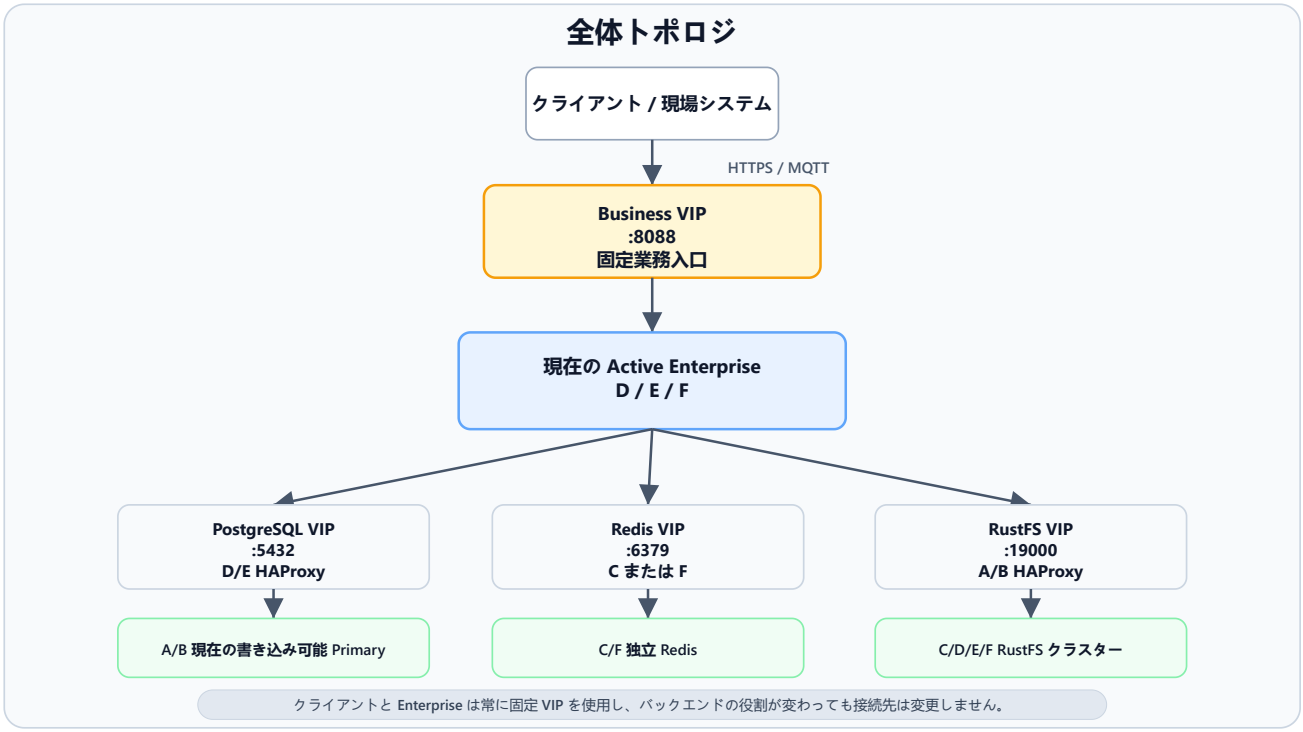
この計画は、同一ネットワークセグメント内の単一マシン障害からの復旧を対象にしています。これは Active/Standby の災害復旧設計であり、複数データセンター、合意ベースの選出、またはデータ損失ゼロを保証する厳密な高可用性クラスターではありません。

設計目標は次のとおりです。

- 現在の業務ノードに障害が発生した後、Fleet Center の入口を健全な Standby ノードへ移動できる。
- PostgreSQL、Redis、RustFS は固定 VIP アドレスを使用し、バックエンドの役割が変わっても Enterprise の `.env` を変更しない。
- App、Flow、Marimo、License などのローカル業務ファイルを Active から Standby へ同期できる。
- 1 つの Cluster License は 1 つの論理インストールに対応し、Enterprise が別ノードに切り替わっても再アクティベーションしない。
- 独立した deployer が 6 台の初期化、ロール割り当て、デプロイ、状態監視、フェイルオーバー検証を行う。
- VIP フェイルオーバー、ファイル同期、独立 Redis インスタンス、データベースレプリケーション、オブジェクトストレージ冗長性を明確に分離する。

この計画は Fleet Center のみに適用されます。Branch Enterprise ノードは通常の単一マシンデプロイとしてインストールされ、この 6 台 Active/Standby 設計には参加しません。

全体トポロジー



主要なリクエスト経路は次のとおりです。

```
クライアント / 現場システム
    -> Business VIP :8088
    -> 現在の Active Enterprise (D / E / F)
    |-- PostgreSQL VIP :5432 -> D/E HAProxy -> A/B の現在の書き込み可能 Primary
    |-- Redis VIP      :6379 -> C または F の独立 Redis
    `-- RustFS VIP     :19000 -> A/B HAProxy -> C/D/E/F RustFS クラスター
```

4 つの VIP は業務設定内の固定入口です。Enterprise がノードを切り替えたり、ミドルウェアのバックエンドロールが変わったりしても、クライアントと Enterprise は接続アドレスを変更しません。

計画 IP アドレス

次のアドレスは例です。実装前に、現場の同一 IPv4 サブネット内の実アドレスに置き換えてください。4 つの VIP は DHCP、静的ホスト、または他の VRRP インスタンスで使用されていてはいけません。

名前	例 IP	説明
deployer-controller	`10.60.10.20`	独立 deployer。6 台の業務マシンには含めない
ent-ha-01 / A	`10.60.10.11`	PostgreSQL Primary と関連ロール
ent-ha-02 / B	`10.60.10.12`	PostgreSQL Standby と関連ロール
ent-ha-03 / C	`10.60.10.13`	Witness、Redis-1、RustFS-1
ent-ha-04 / D	`10.60.10.14`	初期 Active Enterprise と関連ロール
ent-ha-05 / E	`10.60.10.15`	Enterprise Standby と関連ロール
ent-ha-06 / F	`10.60.10.16`	Enterprise Standby と関連ロール
Business VIP	`10.60.10.101`	Enterprise 統一入口、デフォルト `8088`
PostgreSQL VIP	`10.60.10.102`	データベース統一入口、デフォルト `5432`
Redis VIP	`10.60.10.103`	Redis 統一入口、デフォルト `6379`
RustFS VIP	`10.60.10.104`	S3 統一入口、デフォルト `19000`

実装前に、業務 NIC 名、IPv4 プレフィックス、ゲートウェイ、DNS、NTP、ホスト名、SSH ユーザー、ディスクマウントポイント、ファイアウォール許可リストを確認してください。
VIP、ノード IP、NIC をスクリプトで推測して本番に直接使用してはいけません。

6 台の役割



マシン	デフォルトでデプロイするコンポーネント	主な責務
A	PostgreSQL Primary、RustFS HAProxy、Keepalived	初期データベース Primary、1 番目の RustFS プロキシ候補
B	PostgreSQL Standby、RustFS HAProxy、Keepalived	ストリーミング Standby、2 番目の RustFS プロキシ候補
C	PostgreSQL Witness、Redis-1、RustFS-1	データベース Witness/状態記録、Redis と RustFS のデータノード
D	PG HAProxy、Keepalived、RustFS-2、Enterprise、Fleet Agent、rsync	初期業務 Active、1 番目の PostgreSQL プロキシ候補
E	PG HAProxy、Keepalived、RustFS-3、	業務 Standby、2 番目の PostgreSQL プロキシ候補

マシン	デフォルトでデプロイするコンポーネント	主な責務
	Enterprise、Fleet Agent、rsync	
F	Redis-2、RustFS-4、Enterprise、Fleet Agent、rsync	業務 Standby、2 番目の Redis インスタンス

この割り当ては単一マシン障害への耐性と、6 台全体のリソースバランスを目的としています。deployer はホスト名/IP アドレスの自然順でマシンを A-F に対応付け、デフォルトロールを選択しますが、最終割り当ては現場のディスク構成と障害ドメインに照らして確認する必要があります。

マシン構成チェックリスト

Enterprise 業務レイヤーは Single Active/Standby です。任意の時点で、Business VIP を保持するノードだけが業務リクエストを処理します。業務レイヤーの拡張は、1 台の業務ノードの CPU、メモリ、ディスクを増強する垂直拡張で行います。Enterprise ノード数を掛け算して同時処理能力を見積もらないでください。

マシン	ノード種別	vCPU	メモリ	基本ストレージ	ディスク推奨
A	PostgreSQL Primary ノード	8C	16G	1T	`/` は約 80G、残りは PostgreSQL データ、WAL、バックアップ staging 用
B	PostgreSQL Standby ノード	8C	16G	1T	`/` は約 80G、残りは PostgreSQL データ、WAL、バッ

マシン	ノード種別	vCPU	メモリ	基本ストレージ	ディスク推奨
					クアップ staging 用
C	Witness / ミドルウェアノード	4C	8G	500G	システム、Redis、RustFS データ用の固定マウントポイントを割り当てる
D	初期 Active Enterprise ノード	8C	32G	500G	システム、コンテナ、Enterprise 同期ファイル、RustFS データ用の固定マウントポイントを割り当てる
E	Standby Enterprise ノード	8C	32G	500G	D と同じ
F	Standby Enterprise ノード	8C	32G	500G	D と同じ

カテゴリー	台数	1 台あたり仕様	vCPU 小計	メモリ小計	ストレージ小計
PostgreSQL データベースノード	2	`8C / 16G / 1T`	16C	32G	2T

カテゴリー	台数	1 台あたり仕様	vCPU 小計	メモリ小計	ストレージ小計
Witness / ミドルウェアノード	1	`4C / 8G / 500G`	4C	8G	500G
Enterprise 業務ノード	3	`8C / 32G / 500G`	24C	96G	1.5T
合計	**6**	-	**44C**	**136G**	**4T**

構成サマリー

Enterprise 統一接続設定

D、E、F は同じ Enterprise 構成を保持します。

依存先	構成方針
PostgreSQL	A/B ノード IP ではなく PostgreSQL VIP に接続する
Redis	C/F ノード IP ではなく Redis VIP に接続する
File storage	`FILESTORE_DRIVER=s3` を使用し、S3 endpoint を RustFS VIP に向ける
Fleet identity	3 ノードが 1 つの論理 `installationId` を共有し、各ノードは独自の `memberId` を持つ
Fleet Agent	各 Enterprise ノードで 1 つのプロセスレベル Fleet Agent を実行する
License	Business VIP 経由で 1 回だけアクティベートし、Bundle を HA fileset に含める

`installationId` と `memberId` はインストールプロセスで生成、永続化されます。運用者が手動で作成したり、マシン間で `memberId` をコピーしたりしないでください。

VIP と候補ノード

VIP	候補ノード	ヘルス判定	引き継ぎ結果
Business VIP	D, E, F	Fleet Agent readiness、業務コンテナ状態、同期 generation	Owner だけが Enterprise HA 業務グループを起動する
PostgreSQL VIP	D, E	HAProxy/Keepalived 状態。HAProxy は書き込み可能 Primary のみにルーティングする	Enterprise は同じデータベースアドレスを使い続ける
Redis VIP	C, F	Redis `PING` とローカルサービス状態	もう一方の独立インスタンスへ切り替わる。キャッシュ/セッションは失われる可能性がある
RustFS VIP	A, B	HAProxy と RustFS backend のヘルス	S3 アドレスを変えずにプロキシ入口だけが移動する

コンポーネントデプロイ設計

Enterprise、Fleet Agent、Business VIP

- D、E、F は同じ完全な Enterprise オフライン ZIP をインストールする。
- Fleet Agent はホストレベルのプロセスサービスであり、業務コンテナではない。
- Keepalived は Business VIP を管理し、Fleet Agent readiness endpoint を呼び出してローカルノードが VIP を保持できるか判断する。
- Business VIP Owner だけが業務グループを実行し、他の 2 台は Standby にとどまる。
- VIP を失ったノードは業務グループを停止しなければならない。復旧したノードはまず Standby として再参加する。

- EMQX は Single Active 業務グループの一部として実行する。Agent 制御外で再起動したり、3 台間で別の選出機構を作ったりしない。

PostgreSQL Primary/Standby、Witness、統一入口

- A は Primary、B は Streaming Standby、C は repmgr Witness として開始する。
- Witness はクラスター状態を記録し、障害判断に参加する。完全な業務データは保存せず、読み書きも処理しない。
- Primary に障害が発生した場合、repmgr は健全な Standby を新しい Primary に昇格させる。
- D/E HAProxy は `5432` トラフィックを現在の書き込み可能 Primary のみにルーティングする。
- PostgreSQL VIP は D/E プロキシノード間を移動する。プロキシフェイルオーバーとデータベース Primary/Standby フェイルオーバーは分離されている。
- 受け入れでは「書き込み可能 Primary が 1 つだけであること」と「Standby が streaming を再開すること」の両方を証明する必要がある。

Redis 独立インスタンス

- C と F はそれぞれ 1 つの standalone Redis インスタンスを実行する。
- 2 つの Redis は Sentinel、Cluster、Primary/Replica レプリケーションを構成しない。
- Redis VIP は接続入口だけを切り替える。現在のインスタンスに障害が発生すると、VIP はもう一方の健全なインスタンスへ移動する。
- Redis には失われてもよい、または再構築できるデータだけを保存する。切り替え後の空キャッシュや再ログインは設計境界内である。
- 将来 Redis が破棄できない状態を保存する場合は、独立インスタンス前提を維持せず、レプリケーション/仲裁設計へアップグレードする。

RustFS 4 ノードクラスター

- C、D、E、F は 4 ノードの分散 RustFS オブジェクトストレージクラスターを構成する。
- A と B の HAProxy は健全な RustFS backend へプロキシする。
- RustFS VIP は A と B の間を移動する。Enterprise は常にこの VIP 経由で S3 にアクセスする。

- 単一 RustFS ノード障害はクラスター機構で処理し、単一プロキシ障害は VIP フェイルオーバーで処理する。
- オブジェクトデータを rsync、ホットコピー、または RustFS マウントディレクトリの直接編集で同期しない。
- RustFS のノード数、ディスク数、erasure coding の可用性境界は、デプロイされたバージョンのクラスター検査結果に従う。

App、Flow、Marimo、License ファイル同期

Fleet Agent は、PostgreSQL/RustFS に直接置けないが、新しい Active ノードに存在する必要があるローカル業務ファイルだけを同期します。

- App runtime ディレクトリと `node_modules`
- Flow、Marimo、Notebook、および関連するローカル runtime ファイル
- License Bundle
- 現在の実装で明示的に登録されたその他の HA fileset

同期は、deployer が管理する業務ノード間の SSH trust 上で system `rsync` を使用します。各リリースは完全な generation/manifest を作成します。Standby ノードは generation が完全であることを確認してからのみ引き継げます。PostgreSQL データディレクトリ、Redis データディレクトリ、RustFS データディレクトリはこの同期範囲に含まれません。

接続方法とネットワーク基準

アクセス関係

Source	Target	Purpose
ユーザー / 現場システム	Business VIP	Web/API 業務アクセス
D/E/F Enterprise	PostgreSQL VIP	業務データベースの読み書き
D/E/F Enterprise	Redis VIP	キャッシュとセッション
D/E/F Enterprise	RustFS VIP	S3 オブジェクトの読み書き

Source	Target	Purpose
Deployer controller	A-F	SSH、デプロイ、状態収集、運用
D/E/F	D/E/F	rsync/SSH ファイル同期
A/B/C	A/B/C	PostgreSQL streaming replication、repmgr、Witness 通信
RustFS members/proxies	C/D/E/F	RustFS クラスターと S3 backend 通信
同じ VIP を共有するノード	対応する候補ノード	VRRP advertisement と VIP 移動

ポートとプロトコル

ポートは、必要最小限の source-to-target 関係で許可してください。すべての管理ポートをユーザーネットワークへ直接公開しないでください。

Port / protocol	推奨許可範囲	用途
`22/TCP`	Deployer -> A-F; D/E/F 相互同期	SSH、デプロイ、rsync
`8088/TCP`	ユーザーネットワーク -> Business VIP	Enterprise 業務入口
`5432/TCP`	D/E/F -> PostgreSQL VIP; PG/proxy 内部アクセス	PostgreSQL、streaming replication、proxy
`6379/TCP`	D/E/F -> Redis VIP; health-check nodes -> C/F	Redis アクセスとヘルスチェック
`19000/TCP`	D/E/F -> RustFS VIP	Enterprise 固定 S3 入口
`9000/TCP`	A/B -> C/D/E/F; RustFS members	RustFS S3 backend/cluster 通信。バージョンごとに確認

Port / protocol	推奨許可範囲	用途
`9001/TCP`	管理対象運用ネットワーク -> RustFS admin entry	任意の RustFS admin endpoint。ユーザーネットワークへ公開しない
`19731/TCP`	Localhost / Keepalived / 管理対象運用ネットワーク	Fleet Agent HTTP health/control interface
`18080/TCP`	運用ネットワーク -> deployer controller	Deployer HTTPS UI
`112/VRRP`	各 VIP の候補ノード	Keepalived VIP 移動。TCP/UDP ではなく IP プロトコル番号
`1883/8883`	現場デバイスネットワーク -> 業務入口。製品設定に従う	MQTT/MQTTS

フェイルオーバーと機能境界

障害シナリオ	期待される動作	境界
D Active 業務ノード障害	Business VIP が E または F に移動し、新しい Owner が readiness 通過後に業務グループを起動する	Standby 上で最新 HA fileset generation が完了している必要がある
PostgreSQL A Primary 障害	B が Primary に昇格し、PostgreSQL VIP は D/E proxy 経由で書き込み可能 Primary にルーティングし続ける	データ損失は streaming replication の状態に依存する
現在の Redis ノード障害	Redis VIP がもう一方の独立 Redis インスタンスへ移動する	キャッシュ/セッションは失われる可能性がある

障害シナリオ	期待される動作	境界
RustFS proxy A または B 障害	RustFS VIP がもう一方の proxy へ移動する	RustFS データは 4 ノードクラスターの健全性に依存する
単一 RustFS データノード障害	冗長性条件が満たされていれば RustFS クラスターはサービスを継続する	実際の境界はディスク数と erasure coding 構成に依存する

ネットワーク分断、複数ノード同時障害、誤った VRRP priority、ディスク満杯、時刻ずれ、deployer 外の手動変更は通常の自動復旧保証の範囲外であり、運用手順で扱う必要があります。

バックアップ、監視、点検

- PostgreSQL は独立した論理または物理バックアップとリストア検証が必要です。
- RustFS は容量、ディスク、クラスターのヘルス監視が必要です。
- Redis は将来のレプリケーション設計が導入されるまではキャッシュ/セッションとして扱う必要があります。
- D/E/F の Fleet Agent と Keepalived 状態を監視する必要があります。
- rsync generation lag を監視する必要があります。古い generation の Standby は業務トラフィックを引き継いではいけません。
- 定期フェイルオーバードリルには、Business VIP、PostgreSQL 昇格、Redis VIP 切り替え、RustFS proxy 切り替え、バックアップからの復元を含める必要があります。

本番前チェック

- deployer controller から 6 台すべてへ SSH で到達できる。
- ホスト名、IP アドレス、NIC 名、NTP、DNS、ファイアウォールルールが固定され、記録されている。
- 4 つの VIP が未使用で、候補ノード間を移動できる。
- ディスクマウントポイントが固定され、再起動後も維持される。
- Enterprise、PostgreSQL、Redis、RustFS、Keepalived、HAProxy、repmgr、Fleet Agent、rsync の構成が deployer により生成され、レビューされている。

- Cluster License が Business VIP 経由で 1 回だけアクティベートされている。
- 本番トラフィック投入前にバックアップとリストア手順をリハーサルしている。

実装手順

1. deployer controller と 6 台のマシンを準備する。
2. IP、VIP、NIC、ディスク、DNS、NTP、SSH、ファイアウォール情報を確認する。
3. Enterprise オフラインパッケージと構成資料をアップロードする。
4. PostgreSQL primary/standby と Witness をデプロイする。
5. Redis 独立インスタンスと Redis VIP をデプロイする。
6. RustFS 4 ノードクラスターと RustFS VIP をデプロイする。
7. D/E/F に Enterprise、Fleet Agent、rsync fileset、Business VIP をデプロイする。
8. Business VIP 経由で License をアクティベートする。
9. 機能検証とフェイルオーバードリルを実行する。
10. 最終トポロジー、パスワード/キー、バックアップポリシー、監視対象、復旧手順を記録する。

受け入れ基準

機能受け入れ

- ユーザーは Business VIP 経由で Enterprise にアクセスできる。
- Enterprise は PostgreSQL VIP 経由でのみ PostgreSQL を読み書きする。
- Enterprise は Redis VIP 経由でのみ Redis を使用する。
- Enterprise は RustFS VIP 経由でのみ RustFS を使用する。
- フェイルオーバー後、App、Flow、Marimo、Notebook、License ファイルが Active ノードに存在する。
- Branch Enterprise は引き続き独立してインストールでき、この 6 台 HA グループには参加しない。

フェイルオーバードリル受け入れ

- Active 業務ノードを停止する: Business VIP が健全な Standby に移動し、業務グループはそこだけで起動する。

- PostgreSQL Primary を停止する: Standby が昇格し、PostgreSQL VIP は書き込み可能 Primary を指し続ける。
- Redis VIP owner を停止する: Redis VIP がもう一方のインスタンスへ移動し、キャッシュ/セッション損失を許容する。
- RustFS proxy owner を停止する: RustFS VIP がもう一方の proxy へ移動する。
- RustFS データノードを 1 台停止する: オブジェクトアクセスは実際の RustFS 冗長性境界内で維持される。
- 障害ノードを復旧する: Standby または健全な member として復帰し、新しいデータやファイルを上書きしない。

リスクと現場確認項目

- ネットワークが VRRP protocol `112` を許可するか。
- 4 つの VIP アドレスが本当に未使用か。
- ディスクマウント構成と容量が実際のファイル保持と RustFS 冗長性要件を満たすか。
- Redis が破棄可能なデータだけを保存しているか。
- PostgreSQL replication lag とバックアップポリシーが業務 RPO を満たすか。
- 運用担当者が VIP フェイルオーバー、ファイル同期、データベースレプリケーション、オブジェクトストレージ冗長性の違いを理解しているか。

結論

現在の計画は、6 台のマシン、4 つの固定 VIP、Single Active Enterprise を中心にしています。業務入口、データベース入口、キャッシュ入口、オブジェクトストレージ入口を分離します。同一ネットワークセグメント内で一般的な単一マシン障害をカバーでき、フェイルオーバー後に接続構成を変更したり、License を再アクティベートしたり、App ファイルを手動コピーしたりする作業を大幅に減らします。

同時に、3 つの境界を受け入れる必要があります。Redis はデータをレプリケートせず、業務 fileset は非同期で同期され、ネットワーク分断には合意仲裁がありません。したがって、この計画は **Fleet Center 6 台 Single Active Active/Standby 災害復旧計画** として定義するのが適切です。ゼロ中断、ゼロデータ損失、またはデータセンター間の厳密な高可用性を外部に約束してはいけません。

リファレンス

SLA と高可用性の境界

標準 Tier0 Enterprise デプロイの SLA 推奨、高可用性境界、受け入れ条件、訓練、強化項目です。

このドキュメントは、3 台のアプリケーション / Swarm ノード、2 台の PostgreSQL HA ノード、1 台の witness ノードで構成される 6 VM 完全オフラインデプロイに適用されます。

アーキテクチャ基準

レイヤー	現在の設計	高可用性の境界
アプリケーションランタイム	Enterprise New compose は固定の `tier0_app_active_node` 上で実行されます。	`backend`、`redis`、`EMQX`、`SourceFlow`、`EventFlow`、`Marimo`、`OPC UA Server` はまだ自動マルチレプリカサービスではありません。
HTTP 入口	3 台のアプリケーションノードで keepalived APP VIP + HAProxy を実行します。	VIP は浮動できます。HAProxy はアクティブノードの `8088/TCP` に転送し、`/healthz` を確認します。
MQTT / OPC UA	EMQX は `1883/8883/8083/8084`、OPC UA は `4840` を使用します。	これらはアクティブノードへ直接接続します。TCP VIP プロキシには含まれないため、プロトコル入口の HA は保証されません。
データベース	PostgreSQL / TimescaleDB ストリーミングレプリケーション + repmgr + witness + DB VIP。	primary 障害時に standby を自動昇格できます。非同期レプリケーションは厳密なゼロ RPO を保証できません。

レイヤー	現在の設計	高可用性の境界
バックアップと復元	`backup-db.sh` / `restore-db.sh`。	復旧性を提供しますが、オンライン HA と同じではありません。

推奨 SLA

サービス対象	推奨表現	制約
プラットフォーム HTTP 入口	月間可用性 `99.5%` から開始します。	VIP フェイルオーバーはアクティブアプリケーションノードの自動移行と同じではありません。
データベースアクセス	運用目標：`RTO <= 2 minutes`。	RPO はレプリケーション遅延に依存します。同じアーキテクチャでの過去フェイルオーバーテストは約 59-64 秒でした。
MQTT / OPC UA	現在の設計では HA SLA を約束しません。	先に HAProxy TCP proxy、専用 VIP、またはサービスクラスタリングを追加する必要があります。
バックアップと復元	毎日バックアップ。大きな変更前は必須。	復元時間はデータ量と現地 I/O に依存します。

`99.9%` 以上を主張する前に、アプリケーションの自動移行またはマルチレプリカデプロイ、プロトコル入口の HA、監視アラート、復旧訓練を完了してください。計画中の機能を納品済み機能として記述しないでください。

受け入れ条件

各デプロイまたはアップグレードでは、少なくとも以下を満たす必要があります。

- `verify-materials.sh` が成功し、デリバリーパッケージの SHA256 チェックが通る。
- 3 台の Swarm ノードが `Ready` である。

- `APP VIP /healthz` と `/readyz` が `200` を返し、ホームページと `/uns` にアクセスできる。
- アクティブノード上の 7 つの compose サービスが実行中である。
- `DB VIP:5432` にアクセスでき、primary、standby、witness の topology が正常である。
- デフォルトアカウントで実際にログインできる。ポート、コンテナ、HTTP 200 応答だけで IAM 初期化成功を判断しない。
- HTTP、EMQX ポート、WebSocket/WSS、OPC UA TCP の smoke check が通る。

点検とアラート

毎日以下を実行します。

```
./bin/verify.sh --site config/site.conf
./bin/smoke-business-emqx.sh --site config/site.conf
```

少なくとも以下のアラートを設定します。

- APP VIP `/healthz` または `/readyz` の失敗。
- DB VIP 到達不可、primary 不在、split-brain リスク、レプリケーション中断、またはしきい値超過の遅延。
- keepalived、HAProxy、PostgreSQL、repmgrd の異常状態。
- `backend`、`redis`、`EMQX` などの重要コンテナの終了または頻繁な再起動。
- データディスク使用率が `80% / 90%` を超過。
- 最新バックアップが失敗、または生成されていない。

訓練要件

訓練	頻度	検証基準
APP VIP failover	デリバリー前および四半期ごと。	現在の VIP ノードで keepalived を停止した後、`/healthz` と `/readyz` が復旧する。
DB primary 障害	デリバリー前および半年ごと。	standby が昇格し、DB VIP が復旧し、旧 primary が standby として再構築される。
データベース復元	毎月のサンプル確認、および大きな変更前。	バックアップが復元可能であること。復元後に `verify`、スモークチェック、ログイン検証が通過すること。
資材検証	すべてのパッケージビルド。	source material、package SHA、展開検証が通る。

インシデント対応原則

1. まず影響範囲を確認します。HTTP、MQTT、OPC UA、database、単一ノード影響を確認します。
2. VIP、HAProxy、アクティブアプリケーション、database primary の復旧を優先します。
3. ログ、データディレクトリ、バックアップを保全します。インシデント現場を直接削除または上書きしないでください。
4. データベースインシデントでは、まず現在の primary を特定し、その後フェイルオーバー、再参加、復元のいずれを行うか判断します。
5. 復旧後、`verify`、smoke check、実ログイン検証を実行します。

高 SLA 向け強化項目

- アプリケーションを multi-replica 化するか、アクティブノードの検証可能な自動移行プロセスを確立します。
- EMQX をクラスタ化するか、専用 VIP と HAProxy TCP / TLS でルーティングします。

- HAProxy TCP、専用 VIP、またはサービスクラスタリングで OPC UA の高可用入口を提供します。
- PostgreSQL 同期レプリケーションを使用するか、許容できる replication lag と RPO を明確に定義します。
- バックアップスケジュール、オフサイトレプリケーション、定期復元訓練を自動化します。
- コンテナ、ホスト、データベース、ログ、black-box probe を統一アラートに接続します。

関連手順: 運用 Runbook と 標準ポート一覧。

リファレンス

標準ポート一覧

標準的な 6 VM オフライン Tier0 Enterprise デプロイ向けのファイアウォールポート一覧。

次の表は Enterprise New の 6 VM オフラインデプロイを前提にしています。顧客側ファイアウォールでは、必要最小限の送信元範囲からの通信のみを許可してください。これらのポートを公開インターネットへ直接公開しないでください。

送信元	宛先	ポート	プロトコル	用途
制御ホスト	6 台の対象 VM	22	TCP	SSH / Ansible
3 台のアプリケーションノード相互	3 台のアプリケーションノード	2377	TCP	Docker Swarm 管理プレーン
3 台のアプリケーションノード相互	3 台のアプリケーションノード	7946	TCP/UDP	Swarm ノード検出
3 台のアプリケーションノード相互	3 台のアプリケーションノード	4789	UDP	Swarm overlay
3 台のアプリケーションノード	アクティブなアプリケーションノード	8088	TCP	HAProxy から Enterprise New バックエンドへ。アプリケーションネットワークセグメントのみ

送信元	宛先	ポート	プロトコル	用途
ビジネスネットワークセグメント	APP VIP	80	TCP	プラットフォーム HTTP 入口、`/healthz`、`/readyz`
ビジネスネットワークセグメント	APP VIP / アクティブなアプリケーションノード	1883	TCP	MQTT
ビジネスネットワークセグメント	APP VIP / アクティブなアプリケーションノード	8883	TCP	MQTT TLS
ビジネスネットワークセグメント	APP VIP / アクティブなアプリケーションノード	8083	TCP	MQTT WebSocket
ビジネスネットワークセグメント	APP VIP / アクティブなアプリケーションノード	8084	TCP	MQTT WebSocket TLS
ビジネスネットワークセグメント	APP VIP / アクティブなアプリケーションノード	4840	TCP	OPC UA Server
制御されたアプリケーションネットワークセグメント	アクティブなアプリケーションノード	8089	TCP	Launchpad
動的アプリケーションアク	アクティブなアプリケーシ	30000-30199	TCP	App Gateway / 動的アプリ

送信元	宛先	ポート	プロトコル	用途
セスネットワークセグメント	ョンノード			ケーションのホストポート
アプリケーションノード、DB ノード、Witness	DB VIP / DB ノード	5432	TCP	PostgreSQL / TimescaleDB
DB1、DB2、Witness	DB1、DB2、Witness	5432	TCP	Streaming replication / repmgr
アプリケーションノード	NFS server	2049	TCP/UDP	`NFS_ENABLED=true` の場合に使用

現在の compose デプロイでは、Redis、SourceFlow、EventFlow、Marimo、EMQX Dashboard は外部に公開されません。これらのサービスは Docker runtime ネットワーク内の backend からのみ呼び出されます。

リファレンス

運用 Runbook

標準 Tier0 Enterprise deployment の daily inspection、failover、backup、restore、cleanup、troubleshooting 手順。

この document は Enterprise New offline application を前提にしています。

日次点検

1. deployment package の root directory で次の commands を実行します。

```
./bin/verify.sh --site config/site.conf
./bin/smoke-business-emqx.sh --site config/site.conf
```

2. 次の項目を確認します。
   - 3 台の Swarm nodes が `Ready` である。
   - `APP VIP /healthz` と `/readyz` が `200` を返す。
   - `DB VIP:5432` にアクセスできる。
   - PostgreSQL primary、standby、witness topology が正常である。
   - `backend`、`redis`、`emqx`、`sourceflow`、`eventflow`、`marimo`、`opcua-server` が active application node ですべて running である。

Application Checks

1. active application node で次の commands を実行します。

```
cd /opt/tier0-enterprise/tier0-deploy
bash bin/compose.sh ps
bash bin/compose.sh logs --tail=200 backend
curl -fsS http://127.0.0.1:8088/healthz
curl -fsS http://127.0.0.1:8088/readyz
```

> ⓘ ノート

`bin/compose.sh` を迂回して raw `docker compose` commands を直接実行しないでください。この wrapper script は installer が生成した runtime environment を読み込みます。

2. Health-check の見方:
   - `healthz` failure: まず backend process、image、container logs を確認します。
   - `readyz` failure: DB VIP、Redis、database migration logs も確認します。
   - Home page に到達できるが login できない場合: `.env` の `ADMIN_INITIAL_PASSWORD` を確認し、その後 backend IAM migration logs と user tables を確認します。HTTP port だけで deployment success を判断しないでください。

Protocol Entry Checks

```
nc -vz <APP_VIP> 1883
nc -vz <APP_VIP> 8883
nc -vz <APP_VIP> 8083
nc -vz <APP_VIP> 8084
nc -vz <APP_VIP> 4840
```

ⓘ ノート
EMQX と OPC UA Server は active application node に固定されています。APP VIP failover が保証するのは `APP VIP:80` の HTTP entry のみです。Protocol ports は HAProxy TCP proxying 経由で route されないため、この design は protocol entries の high availability を保証しません。

Database Checks

Database nodes で次を実行します。

```
systemctl status tier0-postgresql
systemctl status tier0-repmgrd
sudo -iu postgres psql -Atqc 'select pg_is_in_recovery();'
```

```
sudo -iu postgres /usr/local/bin/repmgr -f /etc/repmgr.conf cluster show
```

- `pg_is_in_recovery=false` は primary を意味します。
- `pg_is_in_recovery=true` は standby を意味します。
- DB VIP は current primary のみに配置されている必要があります。
- standby replication が interrupted した場合、または witness が upstream errors を報告した場合は、まず site を保全してください。full deployment を直接再実行しないでください。

APP VIP Failover

1. test 前に、現在 VIP を保持している node を記録し、次を確認します。

```
curl -fsS http://<APP_VIP>/healthz
curl -fsS http://<APP_VIP>/readyz
```

2. 現在の VIP node で `keepalived` を停止した後、別の application node への VIP takeover を確認し、両方の URLs を再テストします。HAProxy backend は active application node の `8088/TCP` を指すため、3 台すべての application nodes が active node の port 8088 に到達できる必要があります。
3. `keepalived` を復旧した後、`verify.sh` と smoke checks を再実行します。drill 中に active application containers を停止しないでください。

DB Primary Failure Switchover

Failure drill の前に backup を作成し、replication が healthy であることを確認します。

```
./bin/backup-db.sh --site config/site.conf
```

現在の primary で `tier0-postgresql` を停止した後、`repmgr` が standby を promote し、DB VIP takeover することを確認します。target RTO は 2 分以内です。同じ architecture での過去の tests は約 59-64 秒でした。

⚠ 注意

- old primary を production topology に直接戻して起動してはいけません。まず PostgreSQL、repmgrd、keepalived を停止し、残っている DB VIP を削除します。その後 current new primary から basebackup を実行し、node を standby として register し、最後に witness を再接続します。
- failover 後に full installer を直接再実行しないでください。inventory 内の static primary/standby definitions が current topology を乱す可能性があります。

Backup and Restore

- Backup:

```
./bin/backup-db.sh --site config/site.conf
```

- Restore:

```
./bin/restore-db.sh --site config/site.conf --restore-file /backup/tier0-db/postgres-YYYYmmddTHHMMSS.dump
```

ⓘ ノート
restore 前に current primary を確認してください。restore 後は `verify.sh`、smoke checks、default-user path の実際の platform login verification を実行します。

Cleanup and Retest

```
./full-cleanup.sh --env install.env --yes
./install.sh --auto --yes
```

⚠ 注意

Full cleanup は 6 nodes 上の Tier0 runtime data、Docker/Swarm data、database backups を削除します。また control container が終了した後、control node の final cleanup も実行します。この entry は disposable であることが確認済みの test environments でのみ使用してください。customer data または incident site を保持する必要がある場合は、通常の `--cleanup` を使用し、先に backup を完了してください。

Common Issues

Backend Startup Failure

```
cd /opt/tier0-enterprise/tier0-deploy
bash bin/compose.sh logs --tail=300 backend
bash bin/compose.sh ps
```

ⓘ ノート
DB URL、DB VIP、Redis、migration、mounted directory permissions に注目してください。

EMQX Startup Failure

```
cd /opt/tier0-enterprise/tier0-deploy
bash bin/compose.sh logs --tail=200 emqx
```

ⓘ ノート
certificates、configuration files、volume permissions、および ports `1883/8883/8083/8084` が占有されていないか確認してください。

repmgrd Repeated Restarts

```
systemctl status tier0-repmgrd
tail -100 /var/log/repmgr/repmgr.log
```

```
sudo -iu postgres psql -Atqc 'show shared_preload_libraries;'
```

ⓘ ノート
`shared_preload_libraries` に `repmgr` が含まれていること、および systemd unit が `repmgrd --daemonize=false` を使用していることを確認してください。

Material Verification Failure

```
./bin/verify-materials.sh
```

⚠ 注意
SHA256 verification を skip して deployment を続行しないでください。delivery package を再取得するか、traceable process に従って materials を rebuild してください。

OpenAPI リファレンス付録

Tier0 OpenAPI v1 · 操作: 36 · スキーマ: 85

操作

API のフィールド名、例、元の説明は、同梱の英語版 OpenAPI 仕様に従います。

UNS

Unified Namespace のデータの読み取り、書き込み、管理。

POST /uns/attachments

Upload a UNS attachment
Uploads a multipart attachment for a UNS node. Supply a file and at least one of unsId or topic; a valid unsId takes precedence.

リクエストボディ

```
curl --request POST 'https://<tier0-host>/openapi/v1/uns/attachments' \
  --header 'x-api-key: <api-key>'
```

multipart/form-data

フィールド	型	必須	説明
unsId	integer (int64)	いいえ	Numeric owner node ID. Supply unsId or topic; a valid unsId takes precedence.
topic	string	いいえ	Full owner node path, used when a valid unsId is not supplied.
fileName	string	いいえ	Original file name.
sha256	string	いいえ	SHA-256 checksum of the file contents.
file	string (binary)	はい	Binary file contents. The multipart field name must be file.

レスポンス

200 400 401 403 404 409 413 500

成功レスポンスの例

200 · application/json

```
{
  "code": 0,
  "msg": "string",
  "data": {
    "unsId": 0,
    "fileName": "string",
    "filePath": "string",
    "fileUrl": "string",
    "contentType": "string"
  }
}
```

成功レスポンスのフィールド

200 · application/json

フィールド	型	必須	説明
code	integer	いいえ	Response code
msg	string	いいえ	Response message
data	object	いいえ	UnsAttachmentUploadResp. スキーマの章で参照先のスキーマを確認してください。

レスポンスステータスの意味

ステータス	意味	メディアタイプ
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–

ステータス	意味	メディアタイプ
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /uns/attachments/list

List UNS attachments
Lists attachments for a UNS node or topic. Temporary file URLs can expire and must then be requested again.

リクエストボディ

```
curl --request POST 'https://<tier0-host>/openapi/v1/uns/attachments/list' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"unsId":0,"topic":"string","page":0,"size":0,"includeFileUrl":false}'
```

application/json

フィールド	型	必須	説明
unsId	integer (int64)	いいえ	Numeric UNS node ID.
topic	string	いいえ	Full UNS topic path.
page	integer (int64)	いいえ	Page number, starting at 1.
size	integer (int64)	いいえ	Number of attachments per page.
includeFileUrl	boolean (boolean)	いいえ	Whether to return temporary file URLs. Defaults to true.

レスポンス

200 400 401 403 404 409 413 500

成功レスポンスの例

200 · application/json

```
{
  "code": 0,
  "msg": "string",
  "data": {
    "unsId": 0,
    "list": [
      "value"
    ],
    "total": 0,
    "page": 0,
    "size": 0
  }
}
```

成功レスポンスのフィールド

200 · application/json

フィールド	型	必須	説明
code	integer	いいえ	Response code
msg	string	いいえ	Response message
data	object	いいえ	UnsAttachmentListResp. スキーマの章で参照先のスキーマを確認してください。

レスポンスステータスの意味

ステータス	意味	メディアタイプ
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /uns/browse

Browse the UNS tree

Returns a UNS subtree. Use an empty path for the workspace root.

リクエストボディ

```
curl --request POST 'https://<tier0-host>/openapi/v1/uns/browse' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"path":"string","max_depth":0,"include_metadata":false,"include_leaf_value":false}'
```

application/json

フィールド	型	必須	説明
path	string	いいえ	Root UNS path. Empty means the workspace root.
max_depth	integer (int64)	いいえ	Maximum tree depth. Omit or use 0 for the server default.
include_metadata	boolean (boolean)	いいえ	Whether returned nodes should include metadata.
include_leaf_value	boolean (boolean)	いいえ	Whether returned leaf nodes should include their latest payload.

レスポンス

200 400 401 403 404 409 413 500

成功レスポンスの例

200 · application/json

```
{
  "code": 0,
  "msg": "string",
  "data": {
    "tree": [
      "value"
    ]
  }
}
```

成功レスポンスのフィールド

200 · application/json

フィールド	型	必須	説明
code	integer	いいえ	Response code
msg	string	いいえ	Response message

フィールド	型	必須	説明
data	object	いいえ	BrowseResp. スキーマの章で参照先のスキーマを確認してください。

レスポンスステータスの意味

ステータス	意味	メディアタイプ
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /uns/create

Create UNS nodes

Creates UNS nodes and nested subtrees. Check each item result and use the returned topic in subsequent calls.

リクエストボディ

```
curl --request POST 'https://<tier0-host>/openapi/v1/uns/create' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"namespace":
[{"name":"value","type":"value","children":"value","topicType":"value","fields":"value"}]}'
```

application/json

フィールド	型	必須	説明
namespace	array<object>	はい	UNS nodes to create. Nest children to create a subtree.

レスポンス

200 400 401 403 404 409 413 500

成功レスポンスの例

200 · application/json

```
{
  "code": 0,
  "msg": "string",
  "data": {
    "success": false,
    "results": [
      "value"
    ]
  }
}
```

成功レスポンスのフィールド

200 · application/json

フィールド	型	必須	説明
code	integer	いいえ	Response code
msg	string	いいえ	Response message
data	object	いいえ	CreateResp. スキーマの章で参照先のスキーマを確認してください。

レスポンスステータスの意味

ステータス	意味	メディアタイプ
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /uns/delete

Delete UNS nodes

Deletes the listed nodes sequentially. By default this is a soft delete; the batch is not rolled back when a later item fails.

リクエストボディ

```
curl --request POST 'https://<tier0-host>/openapi/v1/uns/delete' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"topics":["string"],"hard_delete":false}'
```

application/json

フィールド	型	必須	説明
topics	array<string>	はい	UNS topic or path node paths to delete.
hard_delete	boolean (boolean)	いいえ	Permanently delete instead of moving nodes to the recycle state.

レスポンス

200 400 401 403 404 409 413 500

成功レスポンスの例

200 · application/json

```
{
  "code": 0,
  "msg": "string",
  "data": {
    "success": false
  }
}
```

成功レスポンスのフィールド

200 · application/json

フィールド	型	必須	説明
code	integer	いいえ	Response code
msg	string	いいえ	Response message
data	object	いいえ	OpenapiEmptyResp. スキーマの章で参照先のスキーマを確認してください。

レスポンスステータスの意味

ステータス	意味	メディアタイプ
200	A successful response.	application/json

ステータス	意味	メディアタイプ
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /uns/history

Query UNS history
Queries historical topic values over an inclusive RFC 3339 time range. Results can be aggregated or sampled.

リクエストボディ

```
curl --request POST 'https://<tier0-host>/openapi/v1/uns/history' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"topics":["string"],"start_time":"string","end_time":"string","aggregation":
{"interval":"string","function":"string","field":"string","fields":["value"]},"page":0}'
```

application/json

フィールド	型	必須	説明
topics	array<string>	はい	UNS topic paths whose history should be queried.
start_time	string	はい	Inclusive query start in RFC 3339 format.
end_time	string	はい	Inclusive query end in RFC 3339 format.
aggregation	object	いいえ	HistoryAggregation. スキーマの章で参照先のスキーマを確認してください。
page	integer (int64)	いいえ	Page number, starting from 1.
size	integer (int64)	いいえ	Page size; values above the server maximum are capped.

レスポンス

200 400 401 403 404 409 413 500

成功レスポンスの例

200 · application/json

```
{
  "code": 0,
  "msg": "string",
  "data": {
    "success": false,
    "results": [
      "value"
    ],
    "page": 0,
    "size": 0,
    "total": 0
  }
}
```

成功レスポンスのフィールド

200 · application/json

フィールド	型	必須	説明
code	integer	いいえ	Response code
msg	string	いいえ	Response message
data	object	いいえ	HistoryResp. スキーマの章で参照先のスキーマを確認してください。

レスポンスステータスの意味

ステータス	意味	メディアタイプ
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–

ステータス	意味	メディアタイプ
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /uns/read

Read UNS values
Returns value, quality, and timeStamp (VQT) for each requested topic. A failed topic does not discard successful items in the same batch.

リクエストボディ

```
curl --request POST 'https://<tier0-host>/openapi/v1/uns/read' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"topics":["string"],"include_metadata":false,"include_leaf_value":false}'
```

application/json

フィールド	型	必須	説明
topics	array<string>	はい	UNS topic paths to read.
include_metadata	boolean (boolean)	いいえ	Whether each result should include UNS node metadata.
include_leaf_value	boolean (boolean)	いいえ	Whether leaf nodes should include their latest payload.

レスポンス

200 400 401 403 404 409 413 500

成功レスポンスの例

200 · application/json

```
{
  "code": 0,
  "msg": "string",
  "data": {
    "success": false,
    "results": [
      "value"
    ]
  }
}
```

成功レスポンスのフィールド

200 · application/json

フィールド	型	必須	説明
code	integer	いいえ	Response code
msg	string	いいえ	Response message
data	object	いいえ	ReadResp. スキーマの章で参照先のスキーマを確認してください。

レスポンスステータスの意味

ステータス	意味	メディアタイプ
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /uns/restore

Restore a deleted UNS node
Restores a soft-deleted UNS node. Permanently deleted nodes cannot be restored.

リクエストボディ

```
curl --request POST 'https://<tier0-host>/openapi/v1/uns/restore' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"path":"string"}'
```

application/json

フィールド	型	必須	説明
path	string	はい	Full UNS path of the node to restore. Use its path before deletion, not a node ID or HTTP URL.

レスポンス

200 400 401 403 404 409 413 500

成功レスポンスの例

200 · application/json

```
{
  "code": 0,
  "msg": "string",
  "data": {
    "success": false
  }
}
```

成功レスポンスのフィールド

200 · application/json

フィールド	型	必須	説明
code	integer	いいえ	Response code
msg	string	いいえ	Response message
data	object	いいえ	OpenapiEmptyResp. スキーマの章で参照先のスキーマを確認してください。

レスポンスステータスの意味

ステータス	意味	メディアタイプ
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /uns/search

Search UNS nodes
Searches UNS node names, aliases, and paths, with optional type and path filters.

リクエストボディ

```
curl --request POST 'https://<tier0-host>/openapi/v1/uns/search' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data
'{"keyword":"string","topicType":"string","path_prefix":"string","include_metadata":false,"include_
leaf_value":false}'
```

application/json

フィールド	型	必須	説明
keyword	string	いいえ	Keyword matched against UNS node names, aliases, and paths.
topicType	string	いいえ	Topic data category filter: STATE, ACTION, or METRIC.
path_prefix	string	いいえ	Only return nodes under this UNS path prefix.
include_metadata	boolean (boolean)	いいえ	Whether returned nodes should include metadata.
include_leaf_value	boolean (boolean)	いいえ	Whether returned leaf nodes should include their latest payload.
page	integer (int64)	いいえ	Page number, starting from 1.
size	integer (int64)	いいえ	Page size.

レスポンス

200 400 401 403 404 409 413 500

成功レスポンスの例

200 · application/json

```
{
  "code": 0,
  "msg": "string",
  "data": {
    "objects": [
      "value"
    ],
    "page": 0,
    "size": 0,
    "total": 0
  }
}
```


成功レスポンスのフィールド

200 · application/json

フィールド	型	必須	説明
code	integer	いいえ	Response code
msg	string	いいえ	Response message
data	object	いいえ	SearchResp. スキーマの章で参照先のスキーマを確認してください。

レスポンスステータスの意味

ステータス	意味	メディアタイプ
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /uns/unsBindFlow

Bind a flow to a UNS node
Binds a flow ID to a UNS node ID.

リクエストボディ

```
curl --request POST 'https://<tier0-host>/openapi/v1/uns/unsBindFlow' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"unsId":0,"flowId":0}'
```

application/json

フィールド	型	必須	説明
unsId	integer (int64)	はい	UNS node ID to bind.
flowId	integer (int64)	はい	Flow ID to bind.

レスポンス

200 400 401 403 404 409 413 500

成功レスポンスの例

200 · application/json

```
{
  "code": 0,
  "msg": "string",
  "data": {
    "success": false,
    "unsId": 0,
    "flowId": 0
  }
}
```

成功レスポンスのフィールド

200 · application/json

フィールド	型	必須	説明
code	integer	いいえ	Response code
msg	string	いいえ	Response message
data	object	いいえ	UnsBindFlowResp. スキーマの章で参照先のスキーマを確認してください。

レスポンスステータスの意味

ステータス	意味	メディアタイプ
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json

ステータス	意味	メディアタイプ
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /uns/update

Update a UNS node

Updates an existing UNS node. Empty strings, an empty fields array, and an empty extendProperties object retain existing values.

リクエストボディ

```
curl --request POST 'https://<tier0-host>/openapi/v1/uns/update' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data
'{"path":"string","name":"string","description":"string","displayName":"string","extendProperties":
{}}'
```

application/json

フィールド	型	必須	説明
path	string	はい	Existing UNS topic or path node to update.
name	string	いいえ	New node name.
description	string	いいえ	Updated node description.
displayName	string	いいえ	Updated display name.
extendProperties	object	いいえ	Updated custom metadata key-value pairs.

フィールド	型	必須	説明
fields	array<object>	いいえ	Updated payload schema fields.
alias	string	いいえ	Updated topic alias.
enableHistory	boolean (boolean)	いいえ	Supply true or false to change history storage. Omit to keep the existing setting.
updateMask	array<string>	いいえ	Compatibility field ignored by the current Enterprise implementation. It cannot be used to clear fields.

レスポンス

200 400 401 403 404 409 413 500

成功レスポンスの例

200 · application/json

```
{
  "code": 0,
  "msg": "string",
  "data": {
    "success": false
  }
}
```

成功レスポンスのフィールド

200 · application/json

フィールド	型	必須	説明
code	integer	いいえ	Response code
msg	string	いいえ	Response message
data	object	いいえ	OpenapiEmptyResp. スキーマの章で参照先のスキーマを確認してください。

レスポンスステータスの意味

ステータス	意味	メディアタイプ
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain

ステータス	意味	メディアタイプ
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /uns/write

Write UNS values

Writes values to existing TOPIC nodes; wildcards are not supported. `value` must match the node payload schema. **Batch results:** check each `data.results` item, not only HTTP 200. **Retry:** confirm the outcome before retrying a write.

リクエストボディ

```
curl --request POST 'https://<tier0-host>/openapi/v1/uns/write' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"writes":
[{"topic":"value","value":"value","timeStamp":"value"}],"qos":0,"retain":false}'
```

application/json

フィールド	型	必須	説明
writes	array<object>	はい	Write operations to apply.
qos	integer (int64)	いいえ	MQTT QoS: 0, 1, or 2.
retain	boolean (boolean)	いいえ	Whether the MQTT message should be retained.

レスポンス

200 400 401 403 404 409 413 500

成功レスポンスの例

200 · application/json

```
{
  "code": 0,
  "msg": "string",
  "data": {
    "success": false,
    "results": [
      "value"
    ]
  }
}
```


成功レスポンスのフィールド

200 · application/json

フィールド	型	必須	説明
code	integer	いいえ	Response code
msg	string	いいえ	Response message
data	object	いいえ	WriteResp. スキーマの章で参照先のスキーマを確認してください。

レスポンスステータスの意味

ステータス	意味	メディアタイプ
200	Inspect the success flag of each `data.results` item. HTTP 200 alone does not confirm that every write succeeded.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

ファイルリソース

保存されたファイルのアップロード、ダウンロード、管理。

POST /assets/files

Create a direct-upload form
Issues an object-store upload form; it does not transfer file contents. Submit every returned post field unchanged with the file.

リクエストボディ

```
curl --request POST 'https://<tier0-host>/openapi/v1/assets/files' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data
'{"fileName":"string","contentType":"string","size":0,"business":"string","useBy":"string"}'
```

application/json

フィールド	型	必須	説明
fileName	string	はい	Original file name.
contentType	string	いいえ	MIME content type.
size	integer (int64)	はい	File size in bytes. Business validation requires a value greater than 0 and at most 1 GiB.
business	string	いいえ	Business domain or feature associated with the file.
useBy	string	いいえ	Optional usage category: user, workspace, or platform.
visibility	string	いいえ	File visibility, such as private or public.
appInstanceId	string	いいえ	Associated application instance ID.
sessionId	string	いいえ	Caller session ID.

レスポンス

200 400 401 403 404 409 413 500

成功レスポンスの例

200 · application/json

```
{
  "code": 0,
  "data": {
    "fileId": 0,
    "filePath": "string",
    "fileUrl": "string",
    "uploadUrl": "string",
    "expiresAt": 0
  },
  "msg": "string"
}
```


成功レスポンスのフィールド

200 · application/json

フィールド	型	必須	説明
code	integer	いいえ	Response code
data	object	いいえ	AssetFileUploadResp. スキーマの章で参照先のスキーマを確認してください。
msg	string	いいえ	Response message

レスポンスステータスの意味

ステータス	意味	メディアタイプ
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /assets/files/delete

Delete a file

Deletes a file identified by its server-issued storage path.

リクエストボディ

```
curl --request POST 'https://<tier0-host>/openapi/v1/assets/files/delete' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"filePath":"string"}'
```

application/json

フィールド	型	必須	説明
filePath	string	はい	Server-issued storage path returned by an upload API.

レスポンス

200 400 401 403 404 409 413 500

成功レスポンスの例

200 · application/json

```
{
  "code": 0,
  "data": {
    "deleted": false
  },
  "msg": "string"
}
```

成功レスポンスのフィールド

200 · application/json

フィールド	型	必須	説明
code	integer	いいえ	Response code
data	object	いいえ	AssetFileDeleteResp. スキーマの章で参照先のスキーマを確認してください。
msg	string	いいえ	Response message

レスポンスステータスの意味

ステータス	意味	メディアタイプ
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain

ステータス	意味	メディアタイプ
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

GET /assets/files/download

Download a private file
Returns file bytes directly. Do not forward x-api-key or other credentials when a client follows a redirect to a cross-origin storage URL.

リクエストパラメーター

名前	場所	型	必須	説明
filePath	query	string	はい	Server-issued storage path returned by an upload API.
responseContentDisposition	query	string	いいえ	Optional Content-Disposition value for the download response.

レスポンス

302 400 401 403 404 409 413 500

レスポンスステータスの意味

ステータス	意味	メディアタイプ
302	Redirect to the final download URL.	–
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–

ステータス	意味	メディアタイプ
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /assets/files/multipart/abort

Abort a multipart upload

Cancels a multipart upload session.

リクエストボディ

```
curl --request POST 'https://<tier0-host>/openapi/v1/assets/files/multipart/abort' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"fileKey":"string","uploadId":"string"}'
```

application/json

フィールド	型	必須	説明
fileKey	string	はい	Object-store key used for the multipart upload.
uploadId	string	はい	Object-store multipart upload session ID.

レスポンス

200 400 401 403 404 409 413 500

成功レスポンスの例

200 · application/json

```
{
  "code": 0,
  "data": {
    "aborted": 1
  },
  "msg": "string"
}
```

成功レスポンスのフィールド

200 · application/json

フィールド	型	必須	説明
code	integer	いいえ	Response code
data	object	いいえ	AssetFileMultipartAbortResp. スキーマの章で参照先のスキーマを確認してください。
msg	string	いいえ	Response message

レスポンスステータスの意味

ステータス	意味	メディアタイプ
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /assets/files/multipart/complete

Complete a multipart upload
Completes a multipart upload after every uploaded part number and ETag have been supplied.

リクエストボディ

```
curl --request POST 'https://<tier0-host>/openapi/v1/assets/files/multipart/complete' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"fileKey":"string","uploadId":"string","parts":[{"partNumber":"value","etag":"value"}]}'
```

application/json

フィールド	型	必須	説明
fileKey	string	はい	Object-store key used for the multipart upload.
uploadId	string	はい	Object-store multipart upload session ID.
parts	array<object>	はい	Uploaded part numbers and their ETags.

レスポンス

200 400 401 403 404 409 413 500

成功レスポンスの例

200 · application/json

```
{
  "code": 0,
  "data": {
    "filePath": "string",
    "fileUrl": "string",
    "sizeBytes": 0,
    "expiresAt": 0
  },
  "msg": "string"
}
```

成功レスポンスのフィールド

200 · application/json

フィールド	型	必須	説明
code	integer	いいえ	Response code
data	object	いいえ	AssetFileMultipartCompleteResp. スキーマの章で参照先のスキーマを確認してください。
msg	string	いいえ	Response message

レスポンスステータスの意味

ステータス	意味	メディアタイプ
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain

ステータス	意味	メディアタイプ
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /assets/files/multipart/init

Initialize a multipart upload
Creates a multipart upload session and returns its ID, part size, and part count.

リクエストボディ

```
curl --request POST 'https://<tier0-host>/openapi/v1/assets/files/multipart/init' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"fileName":"string","contentType":"string","size":0,"partSize":0,"business":"string"}'
```

application/json

フィールド	型	必須	説明
fileName	string	はい	Original file name.
contentType	string	いいえ	File MIME type, for example application/octet-stream.
size	integer (int64)	はい	Total file size in bytes, greater than 0 and at most 1 GiB.
partSize	integer (int64)	いいえ	Preferred part size in bytes. The server may normalize it.
business	string	いいえ	Business domain or feature associated with the file.
useBy	string	いいえ	Optional usage category: user, workspace, or platform.
visibility	string	いいえ	File visibility: private or public.
appInstanceId	string	いいえ	Associated application instance ID.
sessionId	string	いいえ	Caller session ID.
bizId	string	いいえ	Associated business identifier.

レスポンス

200 400 401 403 404 409 413 500

成功レスポンスの例

200 · application/json

```
{
  "code": 0,
  "data": {
    "fileKey": "string",
    "filePath": "string",
    "uploadId": "string",
    "partSize": 0,
    "partCount": 0
  },
  "msg": "string"
}
```

成功レスポンスのフィールド

200 · application/json

フィールド	型	必須	説明
code	integer	いいえ	Response code
data	object	いいえ	AssetFileMultipartInitResp. スキーマの章で参照先のスキーマを確認してください。
msg	string	いいえ	Response message

レスポンスステータスの意味

ステータス	意味	メディアタイプ
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–

ステータス	意味	メディアタイプ
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /assets/files/multipart/part-urls

Create multipart upload URLs
Returns presigned PUT URLs for the requested one-based part numbers.

リクエストボディ

```
curl --request POST 'https://<tier0-host>/openapi/v1/assets/files/multipart/part-urls' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"fileKey":"string","uploadId":"string","partNumbers":[0]}'
```

application/json

フィールド	型	必須	説明
fileKey	string	はい	Object-store key used for the multipart upload.
uploadId	string	はい	Object-store multipart upload session ID.
partNumbers	array<integer (int32)>	はい	One-based part numbers to sign.

レスポンス

200 400 401 403 404 409 413 500

成功レスポンスの例

200 · application/json

```
{
  "code": 0,
  "data": {
    "partUrls": [
      "value"
    ]
  },
  "msg": "string"
}
```

成功レスポンスのフィールド

200 · application/json

フィールド	型	必須	説明
code	integer	いいえ	Response code
data	object	いいえ	AssetFileMultipartPartUrlsResp. スキーマの章で参照先のスキーマを確認してください。
msg	string	いいえ	Response message

レスポンスステータスの意味

ステータス	意味	メディアタイプ
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

GET /assets/files/public

Download a public file

Downloads bytes for a file explicitly marked public. An unguessable path is not authorization; do not mark sensitive files public.

リクエストパラメーター

名前	場所	型	必須	説明
filePath	query	string	はい	Server-issued storage path returned by an upload API.
responseContentDisposition	query	string	いいえ	Optional Content-Disposition value for the download response.

レスポンス

200 400 404 409 413 500

レスポンスステータスの意味

ステータス	意味	メディアタイプ
200	Returns file bytes directly. Content-Type depends on the file type; no JSON envelope is used.	–
400	Correct the request parameters or Content-Type before retrying.	application/json
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

GET /assets/files/url

Create a temporary file URL
Creates a temporary URL for a file.

リクエストパラメーター

名前	場所	型	必須	説明
filePath	query	string	はい	Server-issued storage path returned by an upload API.
expiredSec	query	integer (int64)	いいえ	Requested temporary URL lifetime in seconds.

レスポンス

200 400 401 403 404 409 413 500

成功レスポンスの例

200 · application/json

```
{
  "code": 0,
  "data": {
    "fileUrl": "string",
    "expiresAt": 0
  },
  "msg": "string"
}
```

成功レスポンスのフィールド

200 · application/json

フィールド	型	必須	説明
code	integer	いいえ	Response code
data	object	いいえ	AssetFileUrlResp. スキーマの章で参照先のスキーマを確認してください。
msg	string	いいえ	Response message

レスポンスステータスの意味

ステータス	意味	メディアタイプ
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /assets/upload

Create a legacy asset upload URL
Legacy compatibility endpoint. The current Enterprise implementation returns an empty uploadUrl; use /assets/files for new integrations.

リクエストボディ

```
curl --request POST 'https://<tier0-host>/openapi/v1/assets/upload' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"fileName":"string","sizeBytes":0,"contentType":"string","size":0,"sha256":"string"}'
```

application/json

フィールド	型	必須	説明
fileName	string	はい	Original file name.
sizeBytes	integer (int64)	はい	File size in bytes.
contentType	string	いいえ	File MIME type, for example application/octet-stream.
size	integer (int64)	いいえ	File size in bytes.
sha256	string	いいえ	Optional SHA-256 checksum of the file content.
storageKey	string	いいえ	Optional object-store key. Omit it to let the server generate one.
isPublic	boolean (boolean)	いいえ	Whether the file should be accessible through the public download endpoint.
business	string	いいえ	Business domain or feature that owns the uploaded file.
useBy	string	いいえ	Usage scenario or owner category for the uploaded file.

レスポンス

200 400 401 403 404 409 413 500

成功レスポンスの例

200 · application/json

```
{
  "code": 0,
  "msg": "string",
  "data": {
    "uploadUrl": "string",
    "filePath": "string",
    "fileUrl": "string"
  }
}
```

成功レスポンスのフィールド

200 · application/json

フィールド	型	必須	説明
code	integer	いいえ	Response code
msg	string	いいえ	Response message

フィールド	型	必須	説明
data	object	いいえ	AssetUploadResp. スキーマの章で参照先のスキーマを確認してください。

レスポンスステータスの意味

ステータス	意味	メディアタイプ
200	Legacy upload response. The current Enterprise implementation returns an empty uploadUrl; use /assets/files for new integrations.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

認証

現在の OpenAPI の ID と権限の確認。

POST `/auth/whoami`

Inspect the current API key identity
Returns the current API-key identity and its effective permissions. Call this before integration work to verify access.

レスポンス

200 400 401 403 404 409 413 500

成功レスポンスの例

200 · application/json

```
{
  "code": 0,
  "data": {
    "userID": 0,
    "userName": "string",
    "email": "string",
    "workspaceID": 0,
    "workspaceName": "string"
  },
  "msg": "string"
}
```


成功レスポンスのフィールド

200 · application/json

フィールド	型	必須	説明
code	integer	いいえ	Response code
data	object	いいえ	AuthWhoamiResp. スキーマの章で参照先のスキーマを確認してください。
msg	string	いいえ	Response message

レスポンスステータスの意味

ステータス	意味	メディアタイプ
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

Flow

Source Flow と Event Flow の定義の管理。

POST /flow/create

Create a flow
Creates a SourceFlow or EventFlow and returns its numeric resource ID.

リクエストボディ

```
curl --request POST 'https://<tier0-host>/openapi/v1/flow/create' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"flowName":"string","flowType":"string","description":"string","template":"string"}'
```

application/json

フィールド	型	必須	説明
flowName	string	はい	Flow name, up to 63 Unicode characters.
flowType	string	はい	Flow category: SourceFlow or EventFlow.
description	string	いいえ	Flow description.
template	string	いいえ	Optional flow template JSON.

レスポンス

200 400 401 403 404 409 413 500

成功レスポンスの例

200 · application/json

```
{
  "code": 0,
  "data": {
    "id": 0,
    "brokerID": "string"
  },
  "msg": "string"
}
```

成功レスポンスのフィールド

200 · application/json

フィールド	型	必須	説明
code	integer	いいえ	Response code
data	object	いいえ	FlowCreateResp. スキーマの章で参照先のスキーマを確認してください。

フィールド	型	必須	説明
msg	string	いいえ	Response message

レスポンスステータスの意味

ステータス	意味	メディアタイプ
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /flow/delete

Delete flows

Deletes the listed flows sequentially; earlier deletions are not rolled back when a later item fails.

リクエストボディ

```
curl --request POST 'https://<tier0-host>/openapi/v1/flow/delete' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"ids":[0]}'
```

application/json

フィールド	型	必須	説明
ids	array<integer (int64)>	はい	Flow IDs to delete.

レスポンス

200 400 401 403 404 409 413 500

成功レスポンスの例

200 · application/json

```
{
  "code": 0,
  "data": {
    "success": false
  },
  "msg": "string"
}
```

成功レスポンスのフィールド

200 · application/json

フィールド	型	必須	説明
code	integer	いいえ	Response code
data	object	いいえ	FlowEmptyResp. スキーマの章で参照先のスキーマを確認してください。
msg	string	いいえ	Response message

レスポンスステータスの意味

ステータス	意味	メディアタイプ
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /flow/deploy

Deploy a flow
Deploys a complete Node-RED canvas JSON string to a flow.

リクエストボディ

```
curl --request POST 'https://<tier0-host>/openapi/v1/flow/deploy' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"id":0,"flowsJson":"string"}'
```

application/json

フィールド	型	必須	説明
id	integer (int64)	はい	Flow ID to deploy.
flowsJson	string	はい	Node-RED canvas JSON to deploy.

レスポンス

200 400 401 403 404 409 413 500

成功レスポンスの例

200 · application/json

```
{
  "code": 0,
  "data": {
    "flowId": "string"
  },
  "msg": "string"
}
```

成功レスポンスのフィールド

200 · application/json

フィールド	型	必須	説明
code	integer	いいえ	Response code
data	object	いいえ	FlowDeployResp. スキーマの章で参照先のスキーマを確認してください。
msg	string	いいえ	Response message

レスポンスステータスの意味

ステータス	意味	メディアタイプ
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json

ステータス	意味	メディアタイプ
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /flow/flowdata

Get flow canvas data

Returns the Node-RED canvas revision and node objects for a flow.

リクエストボディ

```
curl --request POST 'https://<tier0-host>/openapi/v1/flow/flowdata' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"id":0}'
```

application/json

フィールド	型	必須	説明
id	integer (int64)	はい	Flow ID.

レスポンス

200 400 401 403 404 409 413 500

成功レスポンスの例

200 · application/json

```
{
  "code": 0,
  "data": {
    "rev": "string",
    "flows": [
      "value"
    ]
  },
  "msg": "string"
}
```

成功レスポンスのフィールド

200 · application/json

フィールド	型	必須	説明
code	integer	いいえ	Response code
data	object	いいえ	FlowDataResp. スキーマの章で参照先のスキーマを確認してください。
msg	string	いいえ	Response message

レスポンスステータスの意味

ステータス	意味	メディアタイプ
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /flow/get

Get a flow
Returns flow metadata for a numeric flow ID.

リクエストボディ

```
curl --request POST 'https://<tier0-host>/openapi/v1/flow/get' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"id":0}'
```

application/json

フィールド	型	必須	説明
id	integer (int64)	はい	Flow ID.

レスポンス

200 400 401 403 404 409 413 500

成功レスポンスの例

200 · application/json

```
{
  "code": 0,
  "data": {
    "id": 0,
    "flowId": "string",
    "flowName": "string",
    "flowType": "string",
    "flowStatus": "string"
  },
  "msg": "string"
}
```

成功レスポンスのフィールド

200 · application/json

フィールド	型	必須	説明
code	integer	いいえ	Response code
data	object	いいえ	FlowInfo. スキーマの章で参照先のスキーマを確認してください。
msg	string	いいえ	Response message

レスポンスステータスの意味

ステータス	意味	メディアタイプ
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json

ステータス	意味	メディアタイプ
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /flow/list

List flows

Lists flows, optionally filtered by a keyword or flow category.

リクエストボディ

```
curl --request POST 'https://<tier0-host>/openapi/v1/flow/list' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"keyword":"string","flowType":"string"}'
```

application/json

フィールド	型	必須	説明
keyword	string	いいえ	Keyword matched against flow names and descriptions.
flowType	string	いいえ	Flow category filter: SourceFlow or EventFlow.

レスポンス

200 400 401 403 404 409 413 500

成功レスポンスの例

200 · application/json

```
{
  "code": 200,
  "msg": "success",
  "data": {
    "list": [
      {
        "id": 101,
        "flowId": "flow-101",
        "flowName": "Line 1 collector",
        "flowType": "source",
        "flowStatus": "draft",
        "template": "[]",
        "description": "Collects line telemetry",
        "isFavorite": 2,
        "currentVersionId": 0,
        "currentVersionName": "",
        "currentVersionType": "",
        "createdTime": 1788134400000,
        "updatedTime": 1788134400000
      }
    ]
  }
}
```


成功レスポンスのフィールド

`200 · application/json`

フィールド	型	必須	説明
code	integer	いいえ	Response code
data	object	いいえ	FlowListResp. スキーマの章で参照先のスキーマを確認してください。
msg	string	いいえ	Response message

レスポンスステータスの意味

ステータス	意味	メディアタイプ
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–

ステータス	意味	メディアタイプ
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /flow/nodes

List available flow node types
Returns the Node-RED node sets available for a flow category.

リクエストボディ

```
curl --request POST 'https://<tier0-host>/openapi/v1/flow/nodes' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"flowType":"string"}'
```

application/json

フィールド	型	必須	説明
flowType	string	はい	Flow category whose Node-RED node catalog should be returned.

レスポンス

200 400 401 403 404 409 413 500

成功レスポンスの例

200 · application/json

```
{
  "code": 0,
  "data": {
    "nodes": [
      "value"
    ]
  },
  "msg": "string"
}
```

成功レスポンスのフィールド

200 · application/json

フィールド	型	必須	説明
code	integer	いいえ	Response code
data	object	いいえ	FlowNodesResp. スキーマの章で参照先のスキーマを確認してください。

フィールド	型	必須	説明
msg	string	いいえ	Response message

レスポンスステータスの意味

ステータス	意味	メディアタイプ
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /flow/update

Update a flow

Updates flow metadata and settings for a numeric flow ID.

リクエストボディ

```
curl --request POST 'https://<tier0-host>/openapi/v1/flow/update' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"id":0,"flowName":"string","description":"string","template":"string","isFavorite":0}'
```

application/json

フィールド	型	必須	説明
id	integer (int64)	はい	Flow ID to update.
flowName	string	いいえ	Updated flow name.
description	string	いいえ	Updated flow description.
template	string	いいえ	Updated flow template JSON.

フィールド	型	必須	説明
isFavorite	integer (int64)	いいえ	Favorite state: 1 for favorite, 2 for not favorite.

レスポンス

200 400 401 403 404 409 413 500

成功レスポンスの例

200 · application/json

```
{
  "code": 0,
  "data": {
    "success": false
  },
  "msg": "string"
}
```

成功レスポンスのフィールド

200 · application/json

フィールド	型	必須	説明
code	integer	いいえ	Response code
data	object	いいえ	FlowEmptyResp. スキーマの章で参照先のスキーマを確認してください。
msg	string	いいえ	Response message

レスポンスステータスの意味

ステータス	意味	メディアタイプ
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

プラットフォームとプロジェクト

プラットフォーム情報の読み取りとプロジェクトの管理。

POST /info

Get service information

Returns the service name, version, reported capabilities, and configured MQTT broker.

リクエストボディ

```
curl --request POST 'https://<tier0-host>/openapi/v1/info' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{}'
```

`application/json`

OpenAPI では、このリクエストボディのオブジェクトフィールドが宣言されていません。

レスポンス

200 400 401 403 404 409 413 500

レスポンスステータスの意味

ステータス	意味	メディアタイプ
200	A successful response.	–
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /launchpad/{projectName}/getMembers

List project members
Lists Launchpad project members. URL-encode projectName when it contains special characters.

リクエストパラメーター

名前	場所	型	必須	説明
projectName	path	string	はい	Project name in the Launchpad URL. URL-encode special characters.

リクエストボディ

```
curl --request POST 'https://<tier0-host>/openapi/v1/launchpad/{projectName}/getMembers' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"roleKey":"string","roles":
["string"],"updatedAtStart":"string","updatedAtEnd":"string","page":0}'
```

application/json

フィールド	型	必須	説明
roleKey	string	いいえ	Role key filter.
roles	array<string>	いいえ	Role filters or the returned role list, depending on the model.
updatedAtStart	string	いいえ	Inclusive update-time lower bound in RFC 3339 format.
updatedAtEnd	string	いいえ	Inclusive update-time upper bound in RFC 3339 format.
page	integer (int64)	いいえ	Page number, starting at 1.
size	integer (int64)	いいえ	Number of members per page.

レスポンス

200 400 401 403 404 409 413 500

成功レスポンスの例

200 · application/json

```
{
  "code": 0,
  "data": {
    "list": [
      "value"
    ],
    "total": 0,
    "page": 0,
    "size": 0
  },
  "msg": "string"
}
```


成功レスポンスのフィールド

200 · application/json

フィールド	型	必須	説明
code	integer	いいえ	Response code
data	object	いいえ	LaunchpadMembersResp. スキーマの章で参照先のスキーマを確認してください。
msg	string	いいえ	Response message

レスポンスステータスの意味

ステータス	意味	メディアタイプ
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /platform/getMembers

List platform members
Lists platform members with optional keyword, role, status, and update-time filters.

リクエストボディ

```
curl --request POST 'https://<tier0-host>/openapi/v1/platform/getMembers' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"keyword":"string","roleKey":"string","roles":["string"],"statuses":
["string"],"updatedAtStart":"string"}'
```

application/json

フィールド	型	必須	説明
keyword	string	いいえ	Search keyword.
roleKey	string	いいえ	Role key filter.
roles	array<string>	いいえ	Role filters or the returned role list, depending on the model.
statuses	array<string>	いいえ	Member status filters: active or disabled.
updatedAtStart	string	いいえ	Inclusive update-time lower bound in RFC 3339 format.
updatedAtEnd	string	いいえ	Inclusive update-time upper bound in RFC 3339 format.
page	integer (int64)	いいえ	Page number, starting at 1.
size	integer (int64)	いいえ	Number of members per page.

レスポンス

200 400 401 403 404 409 413 500

成功レスポンスの例

200 · application/json

```
{
  "code": 0,
  "data": {
    "list": [
      "value"
    ],
    "total": 0,
    "page": 0,
    "size": 0
  },
  "msg": "string"
}
```

成功レスポンスのフィールド

200 · application/json

フィールド	型	必須	説明
code	integer	いいえ	Response code
data	object	いいえ	PlatformMembersResp. スキーマの章で参照先のスキーマを確認してください。
msg	string	いいえ	Response message

レスポンスステータスの意味

ステータス	意味	メディアタイプ
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

MQTT 認証

MQTT 認証情報の作成と削除。

POST /mqtt-auth/create

Create an MQTT credential
Creates a normal MQTT credential. The returned password is available only once and must be stored securely immediately.

リクエストボディ

```
curl --request POST 'https://<tier0-host>/openapi/v1/mqtt-auth/create' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"name":"string","description":"string","clientIDRandomSuffixEnabled":false}'
```

application/json

フィールド	型	必須	説明
name	string	はい	MQTT credential name.
description	string	いいえ	Description of the MQTT credential.
clientIDRandomSuffixEnabled	boolean	いいえ	Whether to append a random suffix to the MQTT client ID.

レスポンス

200 400 401 403 404 409 413 500

成功レスポンスの例

200 · application/json

```
{
  "code": 0,
  "data": {
    "id": 0,
    "name": "string",
    "clientID": "string",
    "username": "string",
    "password": "string"
  },
  "msg": "string"
}
```

成功レスポンスのフィールド

200 · application/json

フィールド	型	必須	説明
code	integer	いいえ	Response code
data	object	いいえ	MqttAuthCreateResp. スキーマの章で参照先のスキーマを確認してください。
msg	string	いいえ	Response message

レスポンスステータスの意味

ステータス	意味	メディアタイプ
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain

ステータス	意味	メディアタイプ
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /mqtt-auth/delete

Delete an MQTT credential
Idempotently deletes a normal MQTT credential. System credentials are not deleted.

リクエストボディ

```
curl --request POST 'https://<tier0-host>/openapi/v1/mqtt-auth/delete' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"id":0}'
```

application/json

フィールド	型	必須	説明
id	integer (int64)	はい	Resource ID.

レスポンス

200 400 401 403 404 409 413 500

成功レスポンスの例

200 · application/json

```
{
  "code": 0,
  "data": {
    "success": false
  },
  "msg": "string"
}
```

成功レスポンスのフィールド

200 · application/json

フィールド	型	必須	説明
code	integer	いいえ	Response code
data	object	いいえ	MqttAuthDeleteResp. スキーマの章で参照先のスキーマを確認してください。
msg	string	いいえ	Response message

レスポンスステータスの意味

ステータス	意味	メディアタイプ
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

スキーマ

API のフィールド名、例、元の説明は、同梱の英語版 OpenAPI 仕様に従います。

AssetFileDeleteReq

フィールド	型	必須	説明
filePath	string	はい	Server-issued storage path returned by an upload API.

AssetFileDeleteResp

フィールド	型	必須	説明
deleted	boolean	はい	Whether the file was deleted.

AssetFileMultipartAbortReq

フィールド	型	必須	説明
fileKey	string	はい	Object-store key used for the multipart upload.
uploadId	string	はい	Object-store multipart upload session ID.

AssetFileMultipartAbortResp

フィールド	型	必須	説明
aborted	integer (int32)	はい	Abort result: 1 = aborted (including idempotent replay success), 2 = not aborted.

AssetFileMultipartCompleteReq

フィールド	型	必須	説明
fileKey	string	はい	Object-store key used for the multipart upload.
uploadId	string	はい	Object-store multipart upload session ID.
parts	array<object>	はい	Uploaded part numbers and their ETags.

AssetFileMultipartCompleteResp

フィールド	型	必須	説明
filePath	string	はい	Server-issued storage path returned by an upload API.
fileUrl	string	いいえ	File access URL.
sizeBytes	integer (int64)	はい	File size in bytes.
expiresAt	integer (int64)	いいえ	Expiration time, in Unix milliseconds.

AssetFileMultipartInitReq

フィールド	型	必須	説明
fileName	string	はい	Original file name.
contentType	string	いいえ	File MIME type, for example application/octet-stream.
size	integer (int64)	はい	Total file size in bytes, greater than 0 and at most 1 GiB.
partSize	integer (int64)	いいえ	Preferred part size in bytes. The server may normalize it.

フィールド	型	必須	説明
business	string	いいえ	Business domain or feature associated with the file.
useBy	string	いいえ	Optional usage category: user, workspace, or platform.
visibility	string	いいえ	File visibility: private or public.
appInstanceId	string	いいえ	Associated application instance ID.
sessionId	string	いいえ	Caller session ID.
bizId	string	いいえ	Associated business identifier.

AssetFileMultipartInitResp

フィールド	型	必須	説明
fileKey	string	はい	Object-store key used for the multipart upload.
filePath	string	はい	Server-issued storage path returned by an upload API.
uploadId	string	はい	Object-store multipart upload session ID.
partSize	integer (int64)	はい	Part size in bytes; the server may normalize it.
partCount	integer (int64)	はい	Total number of upload parts.
expiresAt	integer (int64)	はい	Expiration time, in Unix milliseconds.

AssetFileMultipartPart

フィールド	型	必須	説明
partNumber	integer (int32)	はい	Part number, starting at 1.
etag	string	はい	ETag returned after uploading this part.

AssetFileMultipartPartUrl

フィールド	型	必須	説明
partNumber	integer (int32)	はい	Part number, starting at 1.
url	string	はい	Presigned PUT URL for this part.
expiresAt	integer (int64)	はい	Expiration time, in Unix milliseconds.

AssetFileMultipartPartUrlsReq

フィールド	型	必須	説明
fileKey	string	はい	Object-store key used for the multipart upload.
uploadId	string	はい	Object-store multipart upload session ID.
partNumbers	array<integer (int32)>	はい	One-based part numbers to sign.

AssetFileMultipartPartUrlsResp

フィールド	型	必須	説明
partUrls	array<object>	はい	Presigned URLs for the requested parts.

AssetFileUploadReq

フィールド	型	必須	説明
fileName	string	はい	Original file name.
contentType	string	いいえ	MIME content type.
size	integer (int64)	はい	File size in bytes. Business validation requires a value greater than 0 and at most 1 GiB.
business	string	いいえ	Business domain or feature associated with the file.
useBy	string	いいえ	Optional usage category: user, workspace, or platform.
visibility	string	いいえ	File visibility, such as private or public.
appInstanceId	string	いいえ	Associated application instance ID.
sessionId	string	いいえ	Caller session ID.

AssetFileUploadResp

フィールド	型	必須	説明
fileId	integer (int64)	いいえ	File ID.
filePath	string	はい	Server-issued storage path returned by an upload API.

フィールド	型	必須	説明
fileUrl	string	いいえ	File access URL.
uploadUrl	string	はい	Upload URL; the legacy Enterprise endpoint currently returns an empty string.
expiresAt	integer (int64)	いいえ	Expiration time, in Unix milliseconds.

AssetFileUrlResp

フィールド	型	必須	説明
fileUrl	string	はい	File access URL.
expiresAt	integer (int64)	いいえ	Expiration time, in Unix milliseconds.

AssetUploadReq

AssetUploadReq

フィールド	型	必須	説明
fileName	string	はい	Original file name.
sizeBytes	integer (int64)	はい	File size in bytes.
contentType	string	いいえ	File MIME type, for example application/octet-stream.
size	integer (int64)	いいえ	File size in bytes.
sha256	string	いいえ	Optional SHA-256 checksum of the file content.
storageKey	string	いいえ	Optional object-store key. Omit it to let the server generate one.
isPublic	boolean (boolean)	いいえ	Whether the file should be accessible through the public download endpoint.
business	string	いいえ	Business domain or feature that owns the uploaded file.
useBy	string	いいえ	Usage scenario or owner category for the uploaded file.

AssetUploadResp

AssetUploadResp

フィールド	型	必須	説明
uploadUrl	string	はい	Upload URL; the legacy Enterprise endpoint currently returns an empty string.
filePath	string	はい	Server-issued storage path returned by an upload API.
fileUrl	string	いいえ	File access URL.

AuthWhoamiResp

フィールド	型	必須	説明
userID	integer (int64)	はい	Numeric ID of the API key owner.
userName	string	はい	User name.
email	string	はい	Email address.
workspaceID	integer (int64)	はい	Platform workspace ID.
workspaceName	string	はい	Workspace name; may be empty in Enterprise.
apiKeyName	string	はい	Name of the current API key.
keyPrefix	string	はい	API key prefix.
permissions	array<string>	はい	Effective permissions of the current identity.
roles	array<string>	はい	Role filters or the returned role list, depending on the model.
keyType	string	はい	API key ownership type, for example personal.

BrowseReq

BrowseReq

フィールド	型	必須	説明
path	string	いいえ	Root UNS path. Empty means the workspace root.
max_depth	integer (int64)	いいえ	Maximum tree depth. Omit or use 0 for the server default.
include_metadata	boolean (boolean)	いいえ	Whether returned nodes should include metadata.
include_leaf_value	boolean (boolean)	いいえ	Whether returned leaf nodes should include their latest payload.

BrowseResp

BrowseResp

フィールド	型	必須	説明
tree	array<object>	はい	UNS node tree.

CreateResp

CreateResp

フィールド	型	必須	説明
success	boolean (boolean)	はい	Whether the operation succeeded.
results	array<object>	はい	Per-item results. Check success and error for each item.

FlowCreateReq

フィールド	型	必須	説明
flowName	string	はい	Flow name, up to 63 Unicode characters.
flowType	string	はい	Flow category: SourceFlow or EventFlow.
description	string	いいえ	Flow description.
template	string	いいえ	Optional flow template JSON.

FlowCreateResp

フィールド	型	必須	説明
id	integer (int64)	はい	Resource ID.
brokerID	string	はい	ID of the MQTT broker configuration node injected when the flow is created.

FlowDataResp

フィールド	型	必須	説明
rev	string	はい	Node-RED canvas revision; may be an empty string.
flows	array<object>	はい	Array of Node-RED canvas node objects.

FlowDeleteReq

フィールド	型	必須	説明
ids	array<integer (int64)>	はい	Flow IDs to delete.

FlowDeployReq

フィールド	型	必須	説明
id	integer (int64)	はい	Flow ID to deploy.
flowsJson	string	はい	Node-RED canvas JSON to deploy.

FlowDeployResp

フィールド	型	必須	説明
flowId	string	はい	Flow identifier.

FlowEmptyResp

フィールド	型	必須	説明
success	boolean	はい	Whether the operation succeeded.

FlowGetReq

フィールド	型	必須	説明
id	integer (int64)	はい	Flow ID.

FlowInfo

フィールド	型	必須	説明
id	integer (int64)	はい	Resource ID.
flowId	string	はい	Flow identifier.
flowName	string	はい	Flow name.
flowType	string	はい	Response flow type: source or event.
flowStatus	string	はい	Current flow status, for example draft.
template	string	はい	Flow template JSON, encoded as a string.
description	string	はい	Description.

フィールド	型	必須	説明
isFavorite	integer (int64)	はい	Favorite state: 1 for favorite, 2 for not favorite.
currentVersionId	integer (int64)	はい	Current version ID; 0 if unset.
currentVersionName	string	はい	Current version name; empty if unset.
currentVersionType	string	はい	Current version type; empty if unset.
createdTime	integer (int64)	はい	Creation time, in Unix milliseconds.
updatedTime	integer (int64)	はい	Last update time, in Unix milliseconds.

FlowListReq

フィールド	型	必須	説明
keyword	string	いいえ	Keyword matched against flow names and descriptions.
flowType	string	いいえ	Flow category filter: SourceFlow or EventFlow.

FlowListResp

フィールド	型	必須	説明
list	array<object>	はい	Result list.

FlowNodesReq

フィールド	型	必須	説明
flowType	string	はい	Flow category whose Node-RED node catalog should be returned.

FlowNodesResp

フィールド	型	必須	説明
nodes	array<object>	はい	Available Node-RED node sets.

FlowUpdateReq

フィールド	型	必須	説明
id	integer (int64)	はい	Flow ID to update.
flowName	string	いいえ	Updated flow name.
description	string	いいえ	Updated flow description.

フィールド	型	必須	説明
template	string	いいえ	Updated flow template JSON.
isFavorite	integer (int64)	いいえ	Favorite state: 1 for favorite, 2 for not favorite.

HistoryAggregation

HistoryAggregation

フィールド	型	必須	説明
interval	string	はい	Positive aggregation window, such as 1m, 5m, 1h, or 1d.
function	string	いいえ	Aggregation function: avg, min, max, sum, count, first, or last.
field	string	いいえ	First-level payload field to aggregate.
fields	array<object>	いいえ	Per-field aggregation settings.

HistoryAggregationField

HistoryAggregationField

フィールド	型	必須	説明
name	string	はい	Payload field to aggregate.
function	string	いいえ	Function for this field; defaults to the outer aggregation function.

HistoryQueryMeta

HistoryQueryMeta

フィールド	型	必須	説明
aggregated	boolean (boolean)	はい	Whether the result was aggregated.
sampled	boolean (boolean)	はい	Whether the result was sampled.
rawTotal	integer (int64)	はい	Total number of raw records.
returnedTotal	integer (int64)	はい	Number of records returned in this response.
requestedIntervalMs	integer (int64)	いいえ	Requested aggregation interval in milliseconds.
actualIntervalMs	integer (int64)	いいえ	Actual aggregation interval in milliseconds.
strategy	string	いいえ	Processing strategy used for the query.

HistoryReq

HistoryReq

フィールド	型	必須	説明
topics	array<string>	はい	UNS topic paths whose history should be queried.
start_time	string	はい	Inclusive query start in RFC 3339 format.
end_time	string	はい	Inclusive query end in RFC 3339 format.
aggregation	object	いいえ	HistoryAggregation. スキーマの章で参照先のスキーマを確認してください。
page	integer (int64)	いいえ	Page number, starting from 1.
size	integer (int64)	いいえ	Page size; values above the server maximum are capped.

HistoryResp

HistoryResp

フィールド	型	必須	説明
success	boolean (boolean)	はい	Whether the operation succeeded.
results	array<object>	はい	Per-item results. Check success and error for each item.
page	integer (int64)	はい	Page number, starting at 1.
size	integer (int64)	はい	File size in bytes.
total	integer (int64)	はい	Total number of matching records.

InfoReq

```
{
  "type": "object",
  "properties": {}
}
```

LaunchpadGetMembersReq

フィールド	型	必須	説明
roleKey	string	いいえ	Role key filter.
roles	array<string>	いいえ	Role filters or the returned role list, depending on the model.

フィールド	型	必須	説明
updatedAtStart	string	いいえ	Inclusive update-time lower bound in RFC 3339 format.
updatedAtEnd	string	いいえ	Inclusive update-time upper bound in RFC 3339 format.
page	integer (int64)	いいえ	Page number, starting at 1.
size	integer (int64)	いいえ	Number of members per page.

LaunchpadMemberResp

フィールド	型	必須	説明
memberId	string	はい	Member identifier.
userId	string	はい	User ID.
userName	string	いいえ	User name.
email	string	いいえ	Email address.
roles	array<object>	はい	Role filters or the returned role list, depending on the model.
updatedAt	string	いいえ	Last update time.

LaunchpadMemberRoleAppResp

フィールド	型	必須	説明
appId	string	はい	Application ID, represented as a string.
appName	string	はい	Application name.

LaunchpadMemberRoleResp

フィールド	型	必須	説明
roleId	string	はい	Role ID, represented as a string.
roleKey	string	はい	Role key filter.
roleName	string	いいえ	Role display name.
description	string	いいえ	Description.
memberCount	integer (int64)	はい	Number of members assigned to the role.
apps	array<object>	いいえ	Applications associated with the role.

LaunchpadMembersResp

フィールド	型	必須	説明
list	array<object>	はい	Result list.
total	integer (int64)	はい	Total number of matching records.
page	integer (int64)	はい	Page number, starting at 1.
size	integer (int64)	はい	File size in bytes.

MqttAuthCreateReq

フィールド	型	必須	説明
name	string	はい	MQTT credential name.
description	string	いいえ	Description of the MQTT credential.
clientIDRandomSuffixEnabled	boolean	いいえ	Whether to append a random suffix to the MQTT client ID.

MqttAuthCreateResp

フィールド	型	必須	説明
id	integer (int64)	はい	Resource ID.
name	string	はい	Name.
clientID	string	はい	MQTT client ID.
username	string	はい	MQTT user name.
password	string	はい	Plaintext password. Save it now; it cannot be retrieved again.
clientIDRandomSuffixEnabled	boolean	はい	Whether to append a random suffix to the MQTT client ID.

MqttAuthDeleteReq

フィールド	型	必須	説明
id	integer (int64)	はい	Resource ID.

MqttAuthDeleteResp

フィールド	型	必須	説明
success	boolean	はい	Whether the operation succeeded.

NamespaceNode

NamespaceNode

フィールド	型	必須	説明
name	string	はい	Node name, up to 63 Unicode characters. In a nested request, this is the name under its parent.
type	string	いいえ	Node type: PATH or TOPIC.
children	array<object>	いいえ	Child nodes to create under this node.
topicType	string	いいえ	Topic data category: STATE, ACTION, or METRIC.
fields	array<object>	いいえ	Payload schema fields for TOPIC nodes.
description	string	いいえ	Node description, up to 512 Unicode characters.
displayName	string	いいえ	Display name shown in clients.
extendProperties	object	いいえ	Custom metadata key-value pairs.
alias	string	いいえ	Optional topic alias.
enableHistory	boolean (boolean)	いいえ	Whether history storage is enabled for this topic.

NodeCreateReq

NodeCreateReq

フィールド	型	必須	説明
namespace	array<object>	はい	UNS nodes to create. Nest children to create a subtree.

NodeDeleteReq

NodeDeleteReq

フィールド	型	必須	説明
topics	array<string>	はい	UNS topic or path node paths to delete.
hard_delete	boolean (boolean)	いいえ	Permanently delete instead of moving nodes to the recycle state.

NodeRedNodeSet

フィールド	型	必須	説明
id	string	はい	Resource ID.
name	string	はい	Name.
types	array<string>	はい	Node-RED node types provided by the node set.
enabled	boolean	はい	Whether the node set is enabled.
module	string	はい	Node-RED module name.
version	string	はい	Service or node module version.

NodeRestoreReq

NodeRestoreReq

フィールド	型	必須	説明
path	string	はい	Full UNS path of the node to restore. Use its path before deletion, not a node ID or HTTP URL.

NodeUpdateReq

NodeUpdateReq

フィールド	型	必須	説明
path	string	はい	Existing UNS topic or path node to update.
name	string	いいえ	New node name.
description	string	いいえ	Updated node description.

フィールド	型	必須	説明
displayName	string	いいえ	Updated display name.
extendProperties	object	いいえ	Updated custom metadata key-value pairs.
fields	array<object>	いいえ	Updated payload schema fields.
alias	string	いいえ	Updated topic alias.
enableHistory	boolean (boolean)	いいえ	Supply true or false to change history storage. Omit to keep the existing setting.
updateMask	array<string>	いいえ	Compatibility field ignored by the current Enterprise implementation. It cannot be used to clear fields.

OpenapiCreateResult

OpenapiCreateResult

フィールド	型	必須	説明
success	boolean (boolean)	はい	Whether the operation succeeded.
topic	string	はい	Full UNS topic path.
error	object	いいえ	OpenapiItemError. スキーマの章で参照先のスキーマを確認してください。

OpenapiEmptyResp

OpenapiEmptyResp

フィールド	型	必須	説明
success	boolean (boolean)	はい	Whether the operation succeeded.

OpenapiHistoryItem

OpenapiHistoryItem

フィールド	型	必須	説明
success	boolean (boolean)	はい	Whether the operation succeeded.
topic	string	はい	Full UNS topic path.
result	object	いいえ	OpenapiHistoryResult. スキーマの章で参照先のスキーマを確認してください。
error	object	いいえ	OpenapiItemError. スキーマの章で参照先のスキーマを確認してください。

OpenapiHistoryResult

OpenapiHistoryResult

フィールド	型	必須	説明
values	array<object>	はい	Historical VQT records.
meta	object	いいえ	HistoryQueryMeta. スキーマの章で参照先のスキーマを確認してください。

OpenapiHistoryValue

OpenapiHistoryValue

フィールド	型	必須	説明
value	object	いいえ	Payload value.
quality	string	はい	Data quality: Good, GoodNoData, Uncertain, or Bad.
timeStamp	integer (int64)	はい	Unix timestamp in milliseconds. Writes use server time when omitted.

OpenapiItemError

OpenapiItemError

フィールド	型	必須	説明
code	integer (int64)	はい	Status code; in an item error, identifies the failure.
message	string	はい	Item-level error message.

OpenapiNodeInfo

OpenapiNodeInfo

フィールド	型	必須	説明
id	integer (int64)	はい	Resource ID.
path	string	はい	Full UNS node path.
name	string	はい	Name.
type	string	はい	Node type: PATH or TOPIC.
topicType	string	はい	Topic category: STATE, ACTION, or METRIC.
description	string	いいえ	Description.
displayName	string	いいえ	Display name.
extendProperties	object	いいえ	Custom node metadata as key-value pairs.
fields	array<object>	いいえ	Payload field definitions.
children	array<object>	いいえ	Child nodes under this node.
alias	string	いいえ	Node alias.
enableHistory	boolean (boolean)	いいえ	Whether history storage is enabled.
bindFlowID	integer (int64)	いいえ	Bound Source Flow ID. 0 means no Source Flow is bound.
bindEventFlowID	integer (int64)	いいえ	Bound Event Flow ID. 0 means no Event Flow is bound.
payload	object	いいえ	OpenapiVQT. スキーマの章で参照先のスキーマを確認してください。

OpenapiReadItem

OpenapiReadItem

フィールド	型	必須	説明
success	boolean (boolean)	はい	Whether the operation succeeded.
topic	string	はい	Full UNS topic path.

フィールド	型	必須	説明
result	object	いいえ	OpenapiVQT. スキーマの章で参照先のスキーマを確認してください。
error	object	いいえ	OpenapiItemError. スキーマの章で参照先のスキーマを確認してください。
metadata	object	いいえ	OpenapiNodeInfo. スキーマの章で参照先のスキーマを確認してください。

OpenapiVQT

OpenapiVQT

フィールド	型	必須	説明
value	object	いいえ	Payload value.
quality	string	はい	Data quality: Good, GoodNoData, Uncertain, or Bad.
timeStamp	integer (int64)	はい	Unix timestamp in milliseconds. Writes use server time when omitted.

OpenapiWriteResult

OpenapiWriteResult

フィールド	型	必須	説明
success	boolean (boolean)	はい	Whether the operation succeeded.
topic	string	はい	Full UNS topic path.
error	object	いいえ	OpenapiItemError. スキーマの章で参照先のスキーマを確認してください。

PlatformGetMembersReq

フィールド	型	必須	説明
keyword	string	いいえ	Search keyword.
roleKey	string	いいえ	Role key filter.
roles	array<string>	いいえ	Role filters or the returned role list, depending on the model.
statuses	array<string>	いいえ	Member status filters: active or disabled.
updatedAtStart	string	いいえ	Inclusive update-time lower bound in RFC 3339 format.

フィールド	型	必須	説明
updatedAtEnd	string	いいえ	Inclusive update-time upper bound in RFC 3339 format.
page	integer (int64)	いいえ	Page number, starting at 1.
size	integer (int64)	いいえ	Number of members per page.

PlatformMemberResp

フィールド	型	必須	説明
userId	string	はい	User ID.
userName	string	いいえ	User name.
nickName	string	いいえ	User nickname.
email	string	いいえ	Email address.
status	string	はい	Platform member status: active or disabled.
roles	array<object>	はい	Role filters or the returned role list, depending on the model.
updatedAt	string	はい	Last update time.

PlatformMemberRoleResp

フィールド	型	必須	説明
roleId	string	はい	Role ID, represented as a string.
roleKey	string	はい	Role key filter.
roleName	string	いいえ	Role display name.
description	string	いいえ	Description.

PlatformMembersResp

フィールド	型	必須	説明
list	array<object>	はい	Result list.
total	integer (int64)	はい	Total number of matching records.
page	integer (int64)	はい	Page number, starting at 1.
size	integer (int64)	はい	File size in bytes.

ReadReq

ReadReq

フィールド	型	必須	説明
topics	array<string>	はい	UNS topic paths to read.
include_metadata	boolean (boolean)	いいえ	Whether each result should include UNS node metadata.
include_leaf_value	boolean (boolean)	いいえ	Whether leaf nodes should include their latest payload.

ReadResp

ReadResp

フィールド	型	必須	説明
success	boolean (boolean)	はい	Whether the operation succeeded.
results	array<object>	はい	Per-item results. Check success and error for each item.

SchemaField

SchemaField

フィールド	型	必須	説明
name	string	はい	Payload field name.
type	string	はい	Payload field type: INTEGER, LONG, DOUBLE, FLOAT, STRING, BOOLEAN, or DATETIME.
unit	string	いいえ	Optional engineering unit.

SearchReq

SearchReq

フィールド	型	必須	説明
keyword	string	いいえ	Keyword matched against UNS node names, aliases, and paths.
topicType	string	いいえ	Topic data category filter: STATE, ACTION, or METRIC.
path_prefix	string	いいえ	Only return nodes under this UNS path prefix.

フィールド	型	必須	説明
include_metadata	boolean (boolean)	いいえ	Whether returned nodes should include metadata.
include_leaf_value	boolean (boolean)	いいえ	Whether returned leaf nodes should include their latest payload.
page	integer (int64)	いいえ	Page number, starting from 1.
size	integer (int64)	いいえ	Page size.

SearchResp

SearchResp

フィールド	型	必須	説明
objects	array<object>	はい	Matching UNS nodes.
page	integer (int64)	はい	Page number, starting at 1.
size	integer (int64)	はい	File size in bytes.
total	integer (int64)	はい	Total number of matching records.

UnsAttachmentInfo

UnsAttachmentInfo

フィールド	型	必須	説明
unsId	integer (int64)	はい	Numeric UNS node ID.
fileName	string	いいえ	Original file name.
filePath	string	はい	Server-issued storage path returned by an upload API.
fileUrl	string	いいえ	File access URL.
contentType	string	いいえ	File MIME type, for example application/octet-stream.
size	integer (int64)	いいえ	File size in bytes.
sha256	string	いいえ	SHA-256 checksum of the file contents.
createdAt	integer (int64)	いいえ	Creation time, in Unix milliseconds.
fileUrlExpiresAt	integer (int64)	いいえ	File URL expiration time, in Unix milliseconds.

UnsAttachmentListReq

UnsAttachmentListReq

フィールド	型	必須	説明
unsId	integer (int64)	いいえ	Numeric UNS node ID.
topic	string	いいえ	Full UNS topic path.
page	integer (int64)	いいえ	Page number, starting at 1.
size	integer (int64)	いいえ	Number of attachments per page.
includeFileUrl	boolean (boolean)	いいえ	Whether to return temporary file URLs. Defaults to true.

UnsAttachmentListResp

UnsAttachmentListResp

フィールド	型	必須	説明
unsId	integer (int64)	はい	Numeric UNS node ID.
list	array<object>	はい	Result list.
total	integer (int64)	はい	Total number of matching records.
page	integer (int64)	はい	Page number, starting at 1.
size	integer (int64)	はい	File size in bytes.

UnsAttachmentUploadReq

UnsAttachmentUploadReq

フィールド	型	必須	説明
unsId	integer (int64)	いいえ	Numeric owner node ID. Supply unsId or topic; a valid unsId takes precedence.
topic	string	いいえ	Full owner node path, used when a valid unsId is not supplied.
fileName	string	いいえ	Original file name.
sha256	string	いいえ	SHA-256 checksum of the file contents.
file	string (binary)	はい	Binary file contents. The multipart field name must be file.

UnsAttachmentUploadResp

UnsAttachmentUploadResp

フィールド	型	必須	説明
unsId	integer (int64)	はい	Numeric UNS node ID.
fileName	string	いいえ	Original file name.
filePath	string	はい	Server-issued storage path returned by an upload API.
fileUrl	string	いいえ	File access URL.
contentType	string	いいえ	File MIME type, for example application/octet-stream.
size	integer (int64)	いいえ	File size in bytes.
sha256	string	いいえ	SHA-256 checksum of the file contents.
createdAt	integer (int64)	いいえ	Creation time, in Unix milliseconds.
fileUrlExpiresAt	integer (int64)	いいえ	File URL expiration time, in Unix milliseconds.

UnsBindFlowReq

UnsBindFlowReq

フィールド	型	必須	説明
unsId	integer (int64)	はい	UNS node ID to bind.
flowId	integer (int64)	はい	Flow ID to bind.

UnsBindFlowResp

UnsBindFlowResp

フィールド	型	必須	説明
success	boolean (boolean)	はい	Whether the operation succeeded.
unsId	integer (int64)	はい	Numeric UNS node ID.
flowId	integer (int64)	はい	Flow identifier.

WriteItem

WriteItem

フィールド	型	必須	説明
topic	string	はい	UNS topic path to write.
value	object	いいえ	Payload value to write.
timeStamp	integer (int64)	いいえ	Optional Unix timestamp in milliseconds. Omit it to use server time.

WriteReq

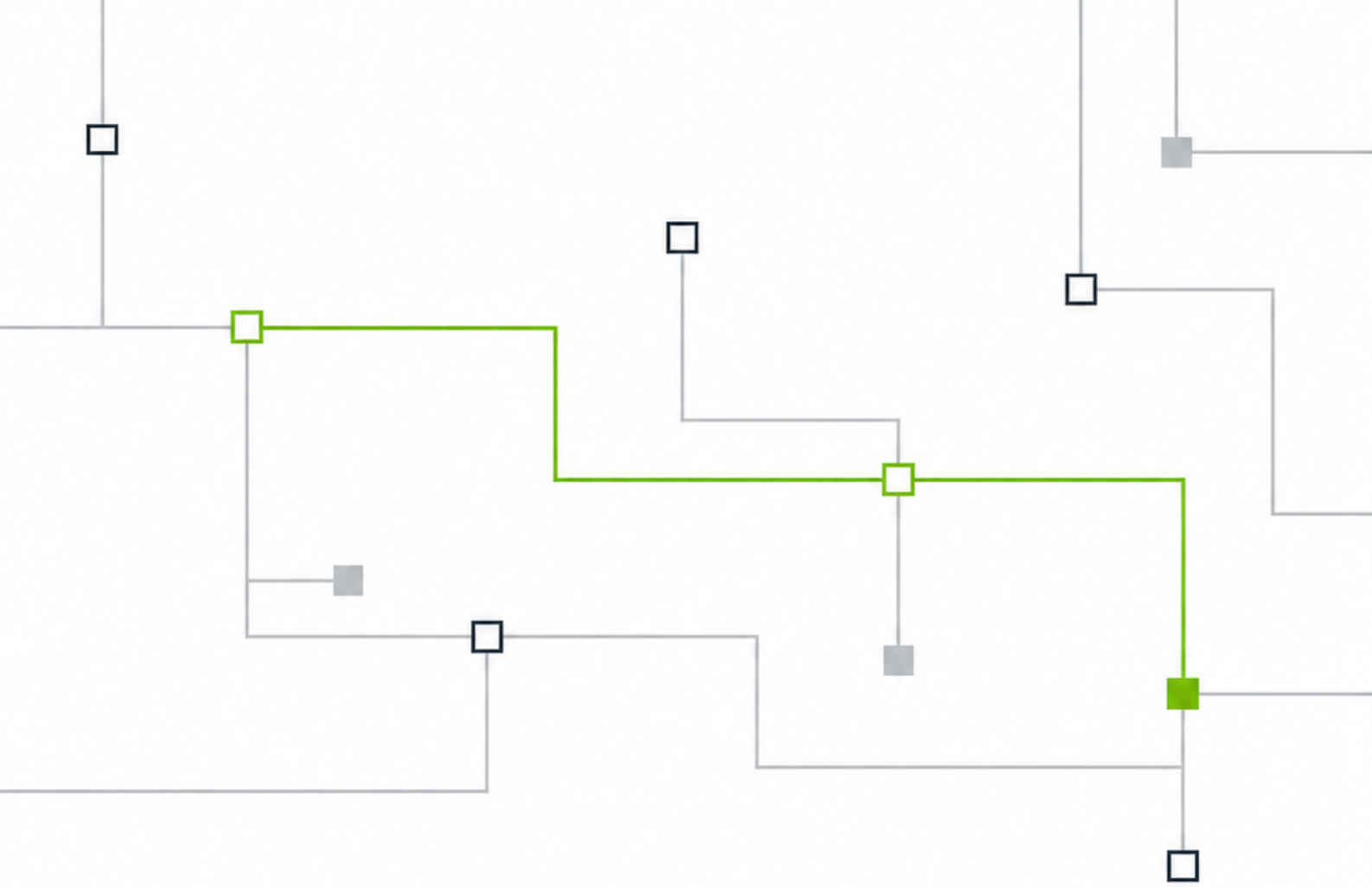
WriteReq

フィールド	型	必須	説明
writes	array<object>	はい	Write operations to apply.
qos	integer (int64)	いいえ	MQTT QoS: 0, 1, or 2.
retain	boolean (boolean)	いいえ	Whether the MQTT message should be retained.

WriteResp

WriteResp

フィールド	型	必須	説明
success	boolean (boolean)	はい	Whether the operation succeeded.
results	array<object>	はい	Per-item results. Check success and error for each item.



TIER 0

tier0.app